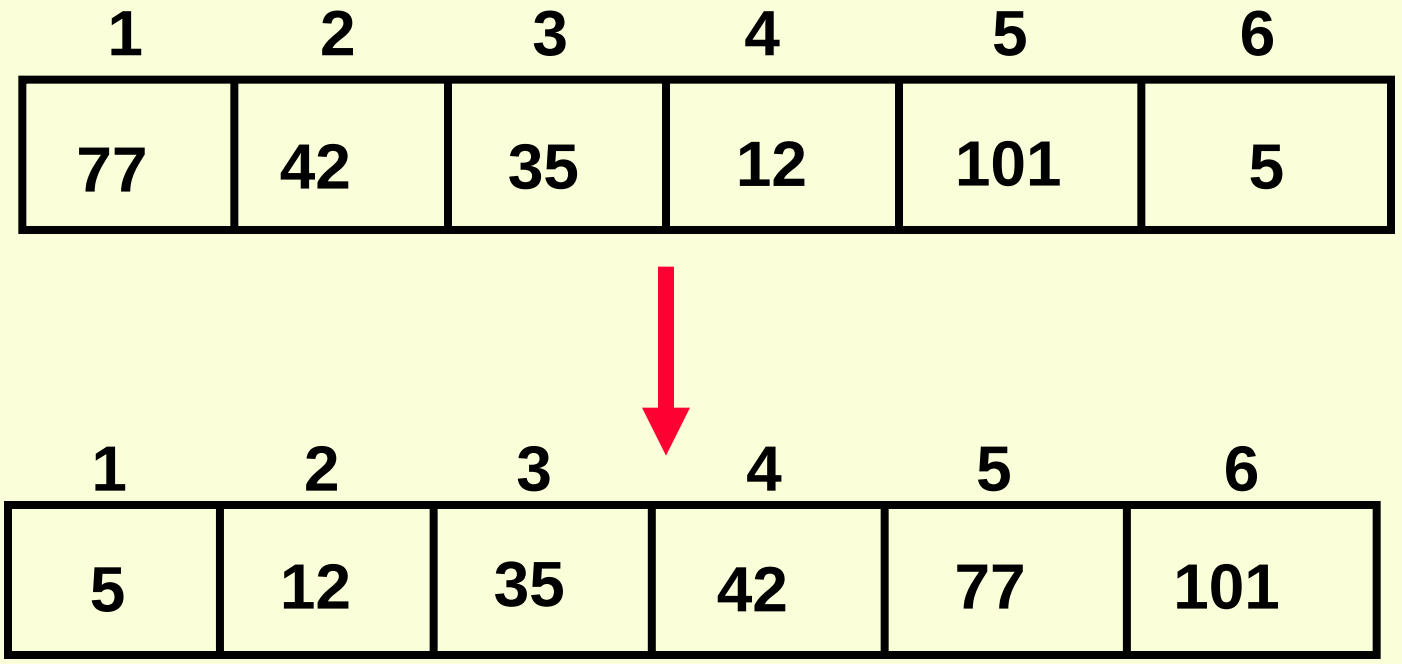
Lecture 16

Bubble Sort
Merge Sort

Bubble Sort

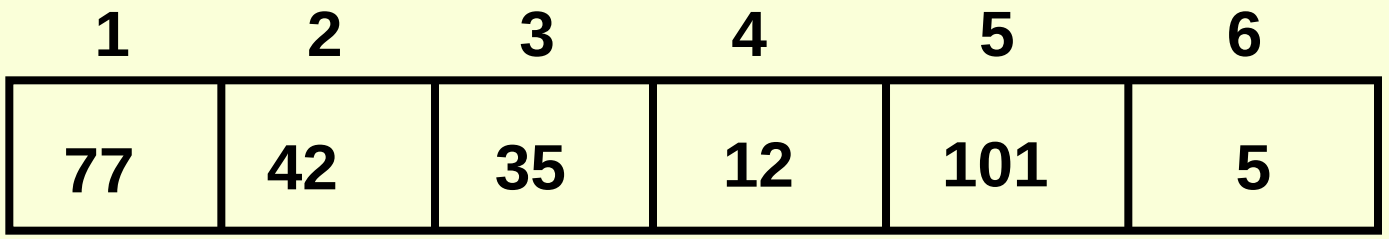
Sorting

- **Sorting takes an unordered collection and makes it an ordered one.**



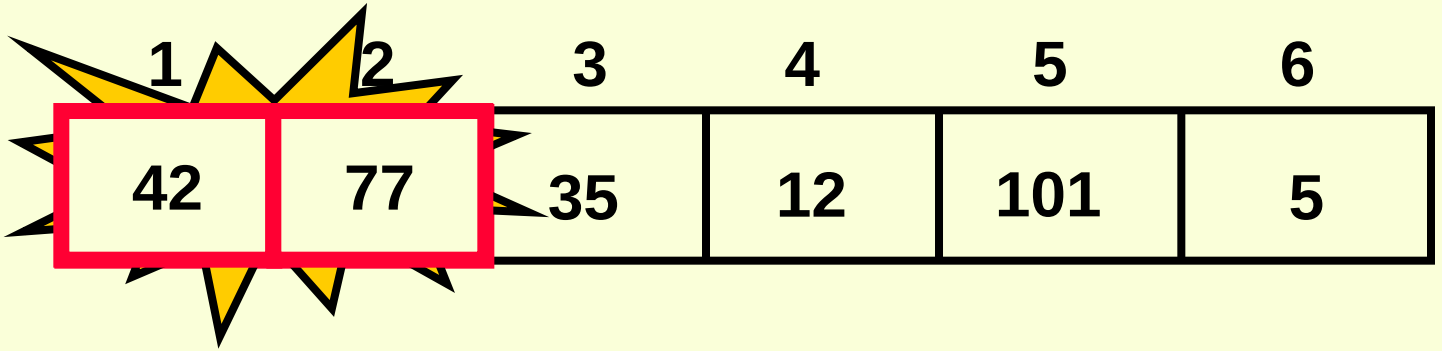
"Bubbling Up" the Largest Element

- Traverse a collection of elements
  - Move from the front to the end
  - "Bubble" the largest value to the end using pair-wise comparisons and swapping



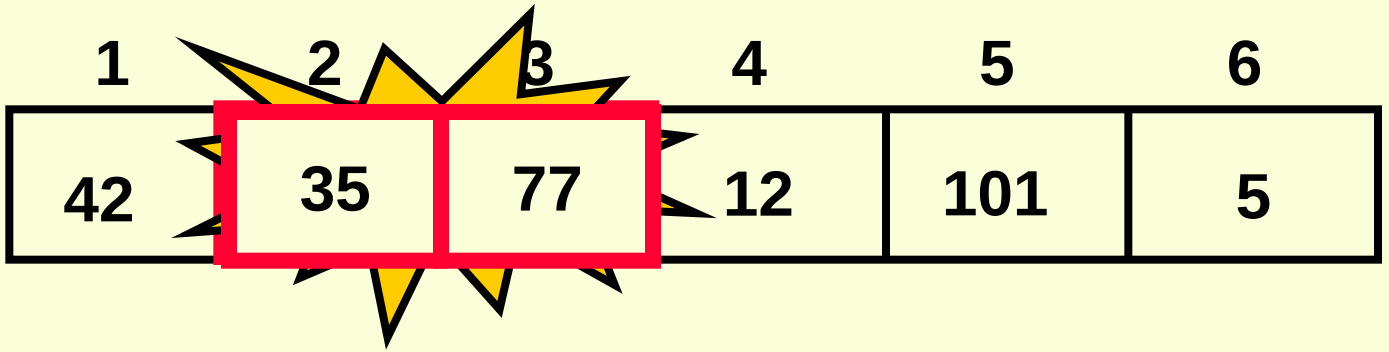
"Bubbling Up" the Largest Element

- Traverse a collection of elements
  - Move from the front to the end
  - "Bubble" the largest value to the end using pair-wise comparisons and swapping



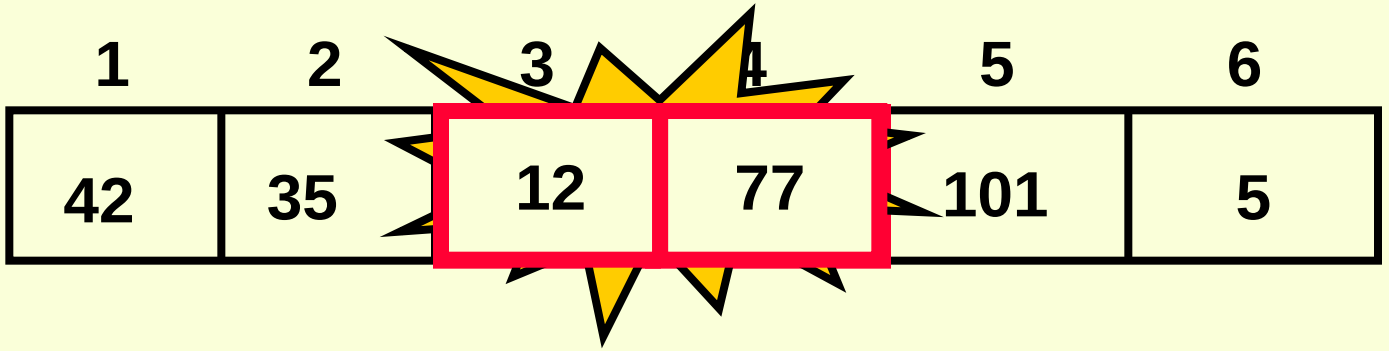
"Bubbling Up" the Largest Element

- Traverse a collection of elements
  - Move from the front to the end
  - "Bubble" the largest value to the end using pair-wise comparisons and swapping



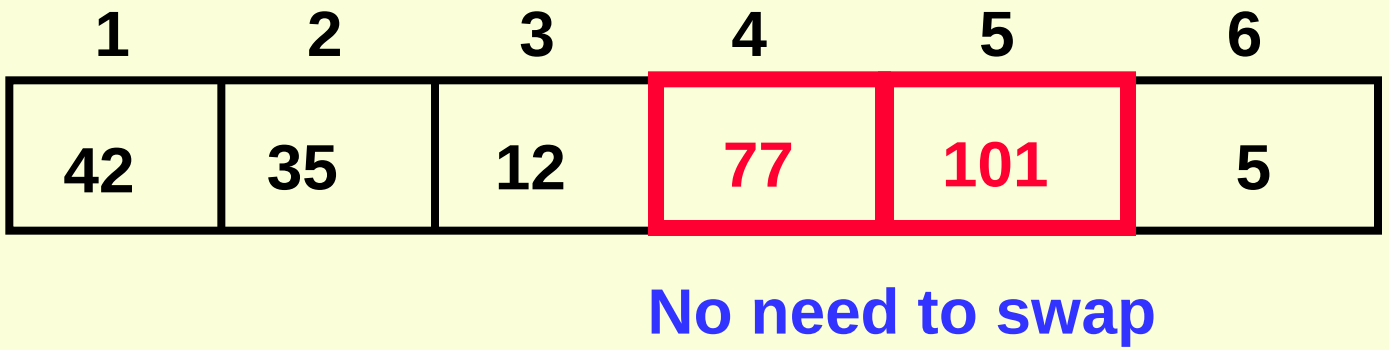
"Bubbling Up" the Largest Element

- Traverse a collection of elements
  - Move from the front to the end
  - "Bubble" the largest value to the end using pair-wise comparisons and swapping



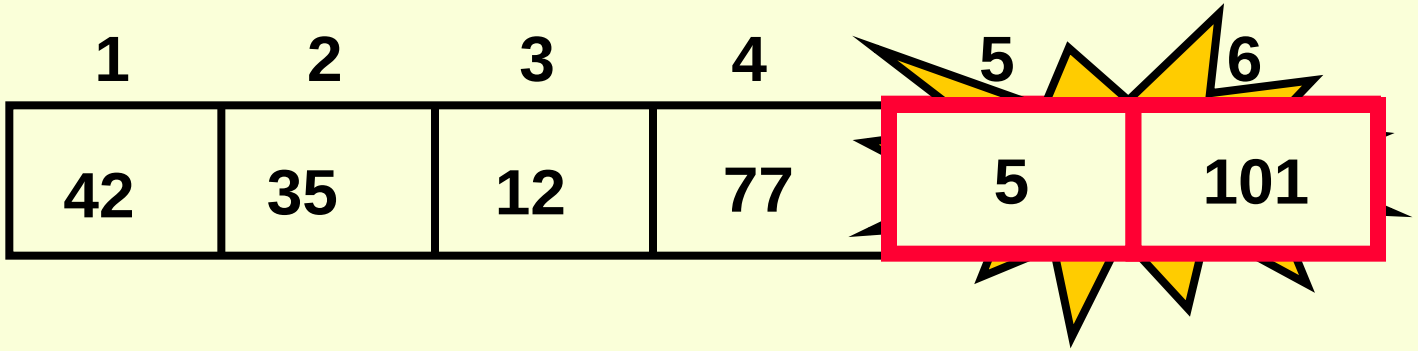
"Bubbling Up" the Largest Element

- Traverse a collection of elements
  - Move from the front to the end
  - "Bubble" the largest value to the end using pair-wise comparisons and swapping



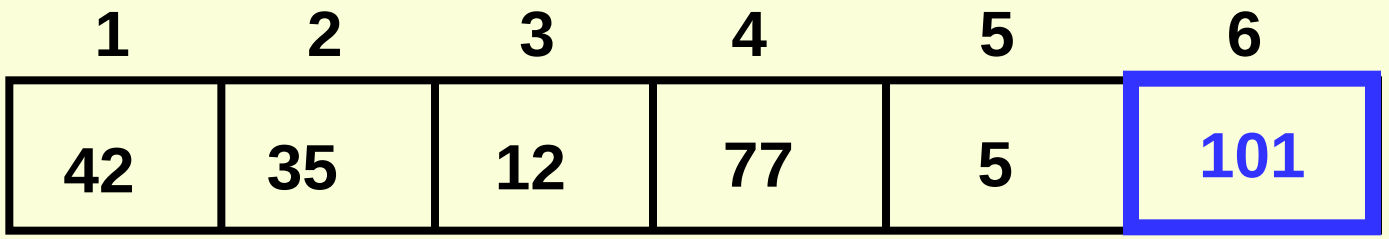
"Bubbling Up" the Largest Element

- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the largest value to the end using pair-wise comparisons and swapping



"Bubbling Up" the Largest Element

- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the largest value to the end using pair-wise comparisons and swapping



Largest value correctly placed

The “Bubble Up” Algorithm

```
index <- 1
last_compare_at <- n - 1

loop
  exitif(index > last_compare_at)
  if(A[index] > A[index + 1]) then
    Swap(A[index], A[index + 1])
  endif
  index <- index + 1
endloop
```

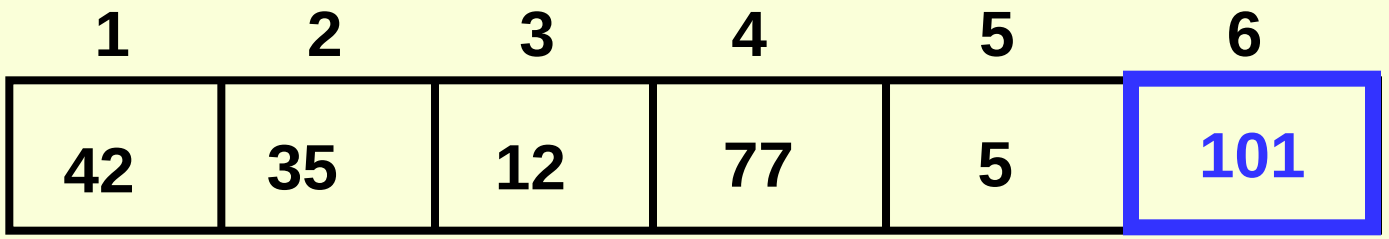
LB

No, Swap isn't built in.

```
Procedure Swap(a, b isoftype in/out Num)
   t isoftype Num
   t <- a
   a <- b
   b <- t
endprocedure // Swap
```

Items of Interest

- **Notice that only the largest value is correctly placed**
- **All other values are still out of order**
- **So we need to repeat this process**

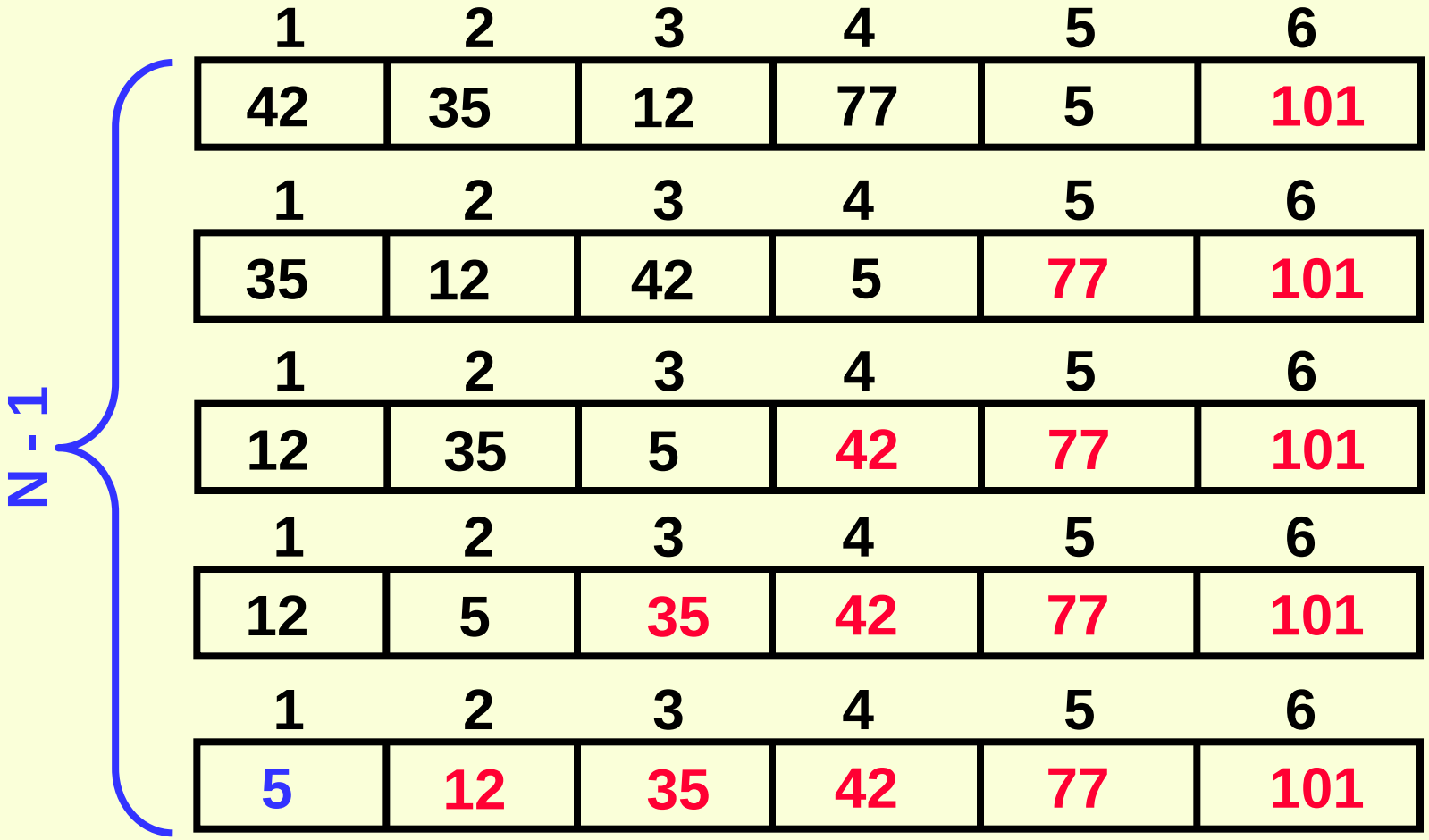


Largest value correctly placed

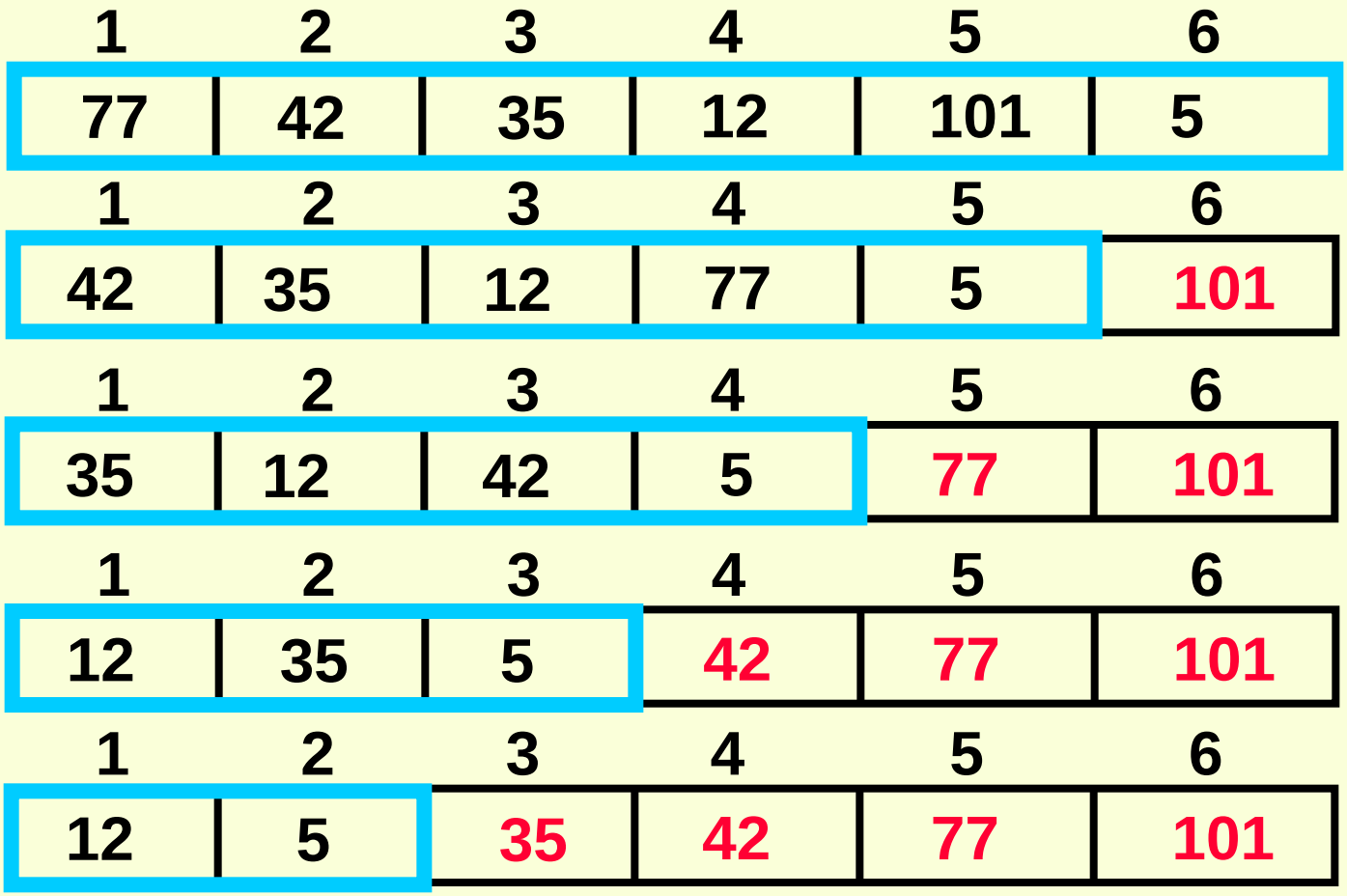
Repeat “Bubble Up” How Many Times?

- If we have N elements…
- And if each time we bubble an element, we place it in its correct location…
- Then we repeat the “bubble up” process N – 1 times.
- This guarantees we’ll correctly place all N elements.

“Bubbling” All the Elements



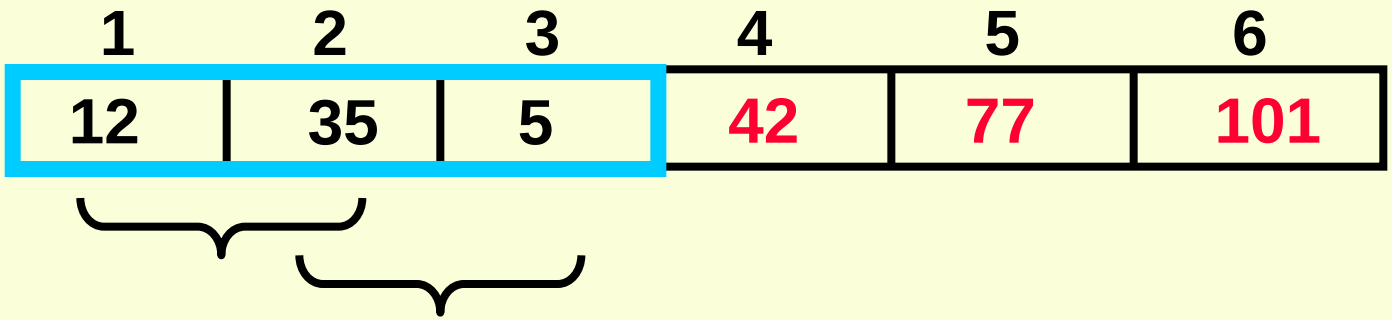
Reducing the Number of Comparisons



Reducing the Number of Comparisons

- On the Nth "bubble up", we only need to do MAX-N comparisons.

- For example:
  - This is the 4th "bubble up"
  - MAX is 6
  - Thus we have 2 comparisons to do



Putting It All Together

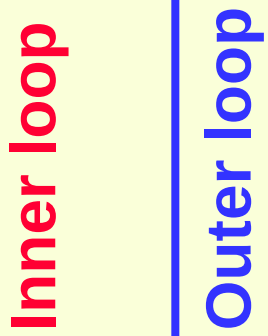
```
N is … // Size of Array

Arr_Type definesa Array[1..N] of Num

Procedure Swap(n1, n2 isoftype in/out Num)
  temp isoftype Num
  temp <- n1
  n1   <- n2
  n2   <- temp
endprocedure // Swap
```

```
procedure Bubblesort(A isoftype in/out Arr_Type)
  to_do, index isoftype Num
  to_do <- N - 1

  loop
    exitif(to_do = 0)
    index <- 1
    loop
      exitif(index > to_do)
      if(A[index] > A[index + 1]) then
        Swap(A[index], A[index + 1])
      endif
      index <- index + 1
    endloop
    to_do <- to_do - 1
  endloop
endprocedure // Bubblesort
```





Already Sorted Collections?

- What if the collection was already sorted?
- What if only a few elements were out of place and after a couple of “bubble ups,” the collection was sorted?

- We want to be able to detect this and “stop early”!

1	2	3	4	5	6
5	12	35	42	77	101

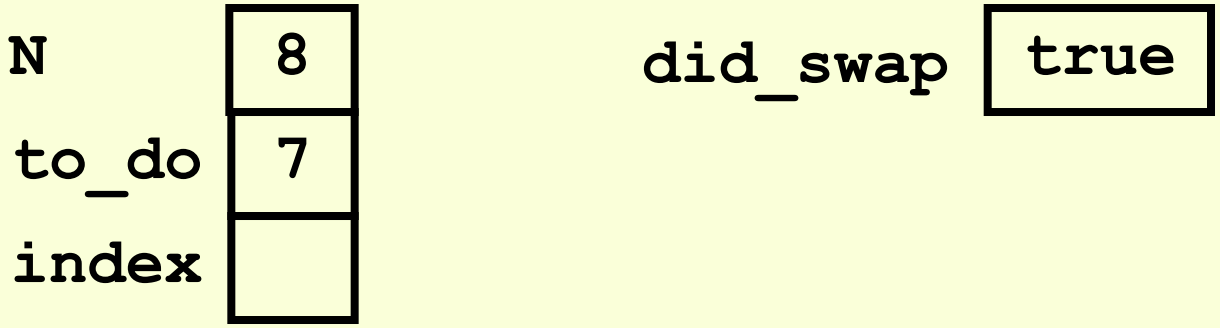
Using a Boolean "Flag"

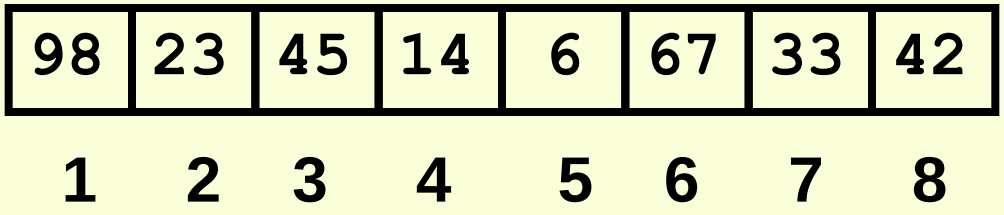
- We can use a boolean variable to determine if any swapping occurred during the "bubble up."
- If no swapping occurred, then we know that the collection is already sorted!
- This boolean "flag" needs to be reset after each "bubble up."

```
did_swap isoftype Boolean
did_swap <- true

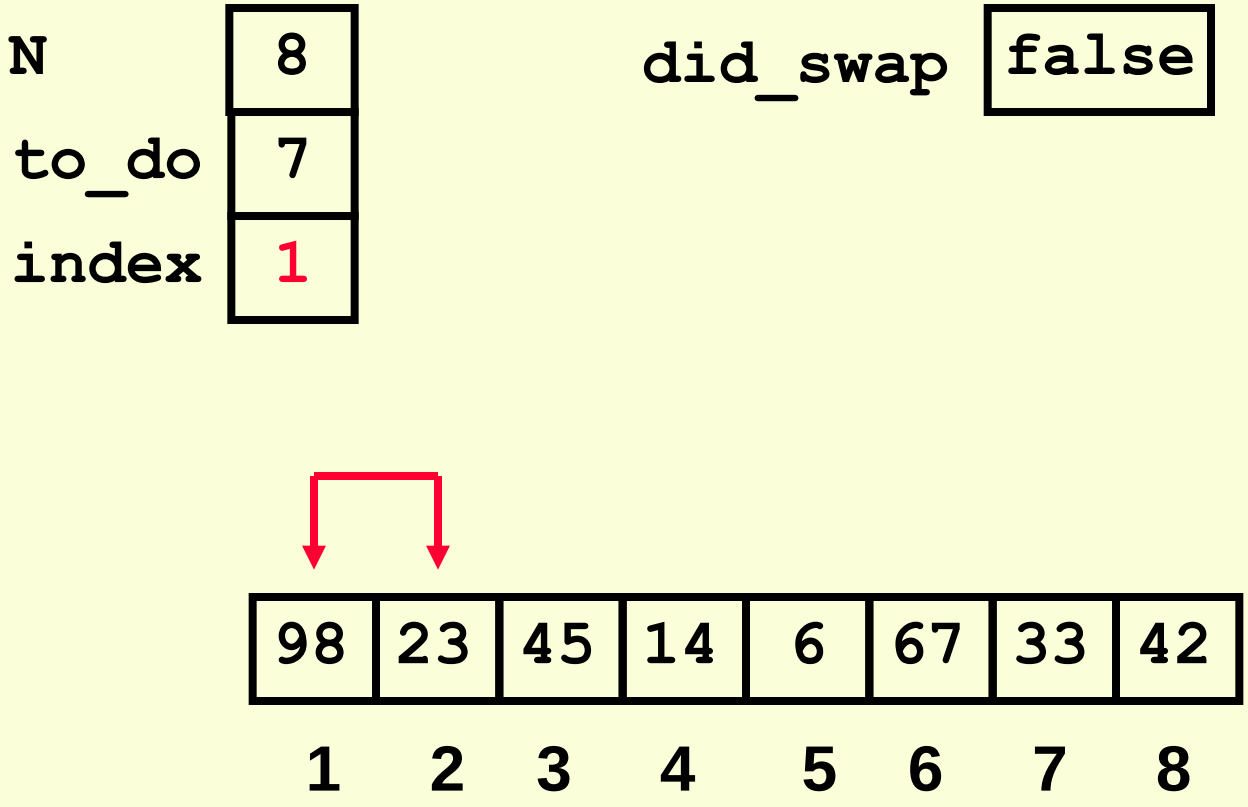
loop
  exitif ((to_do = 0) OR NOT(did_swap))
  index <- 1
  did_swap <- false
  loop
    exitif(index > to_do)
    if(A[index] > A[index + 1]) then
      Swap(A[index], A[index + 1])
      did_swap <- true
    endif
    index <- index + 1
  endloop
  to_do <- to_do - 1
endloop
```

An Animated Example

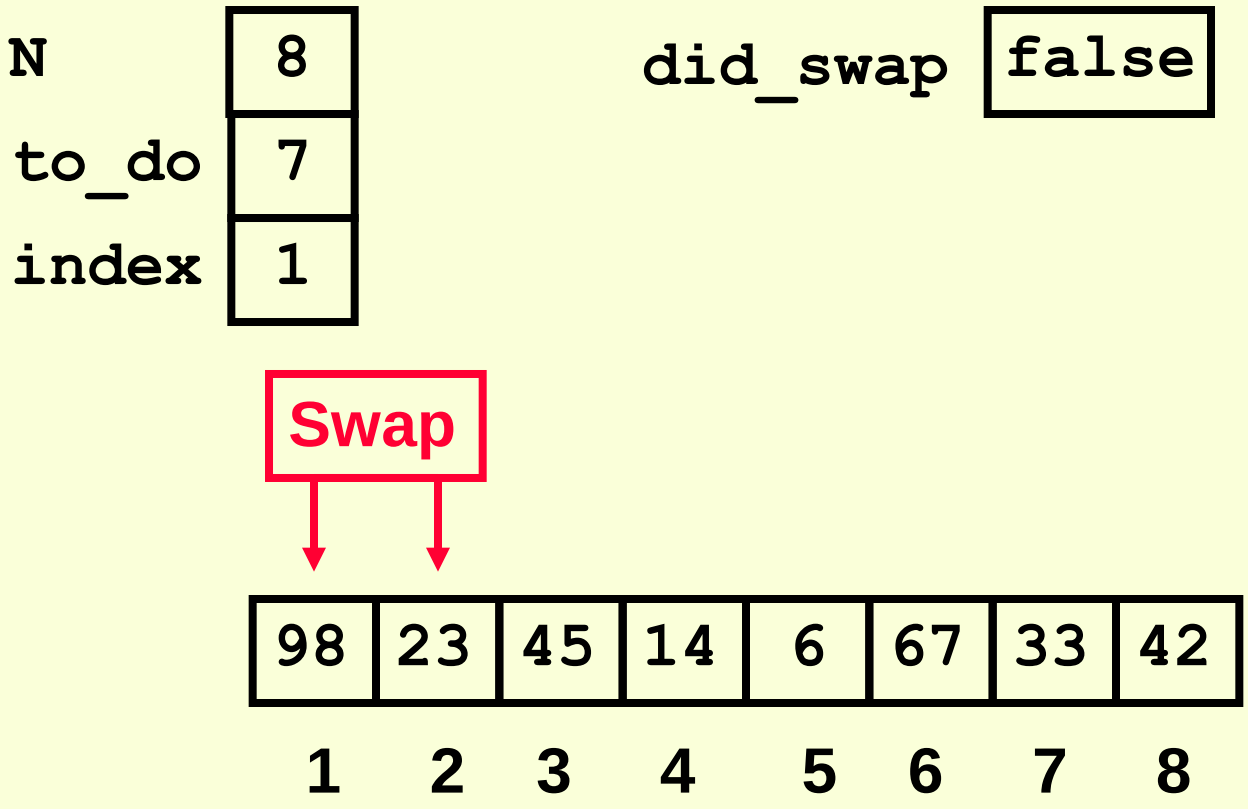




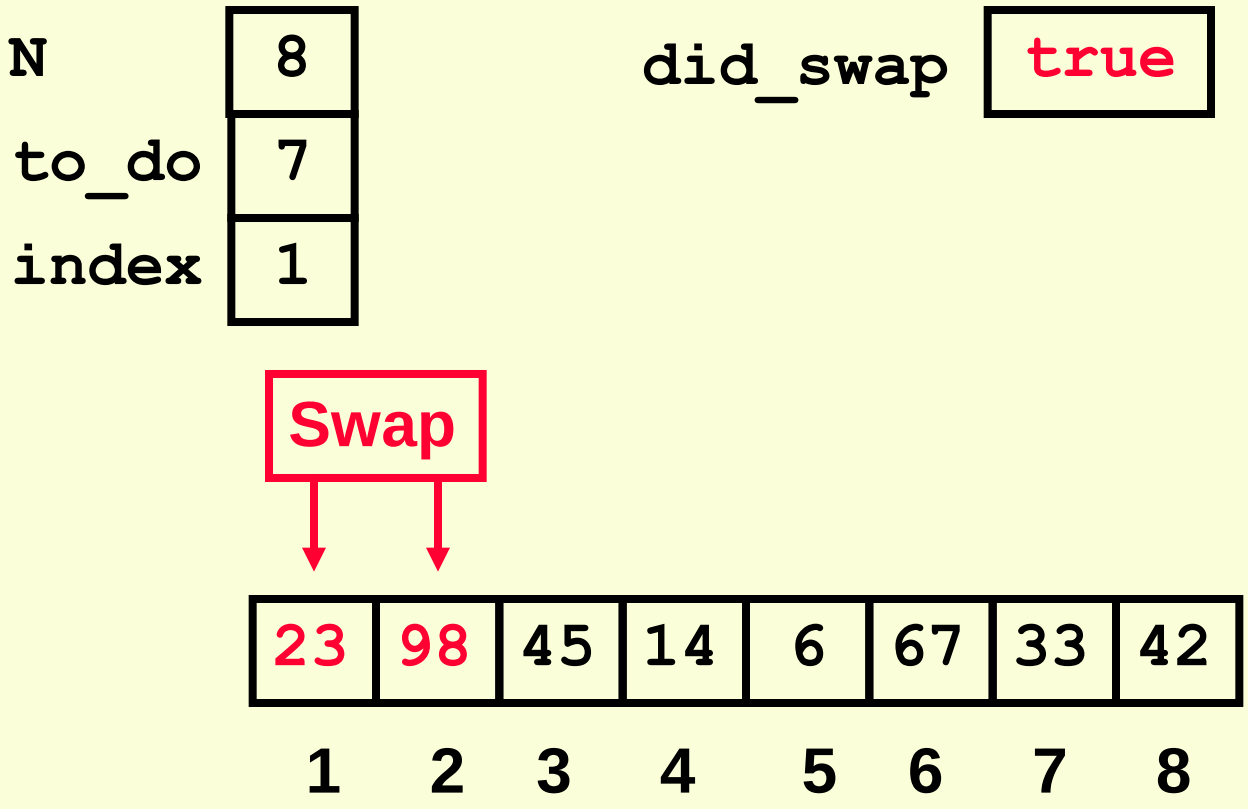
An Animated Example



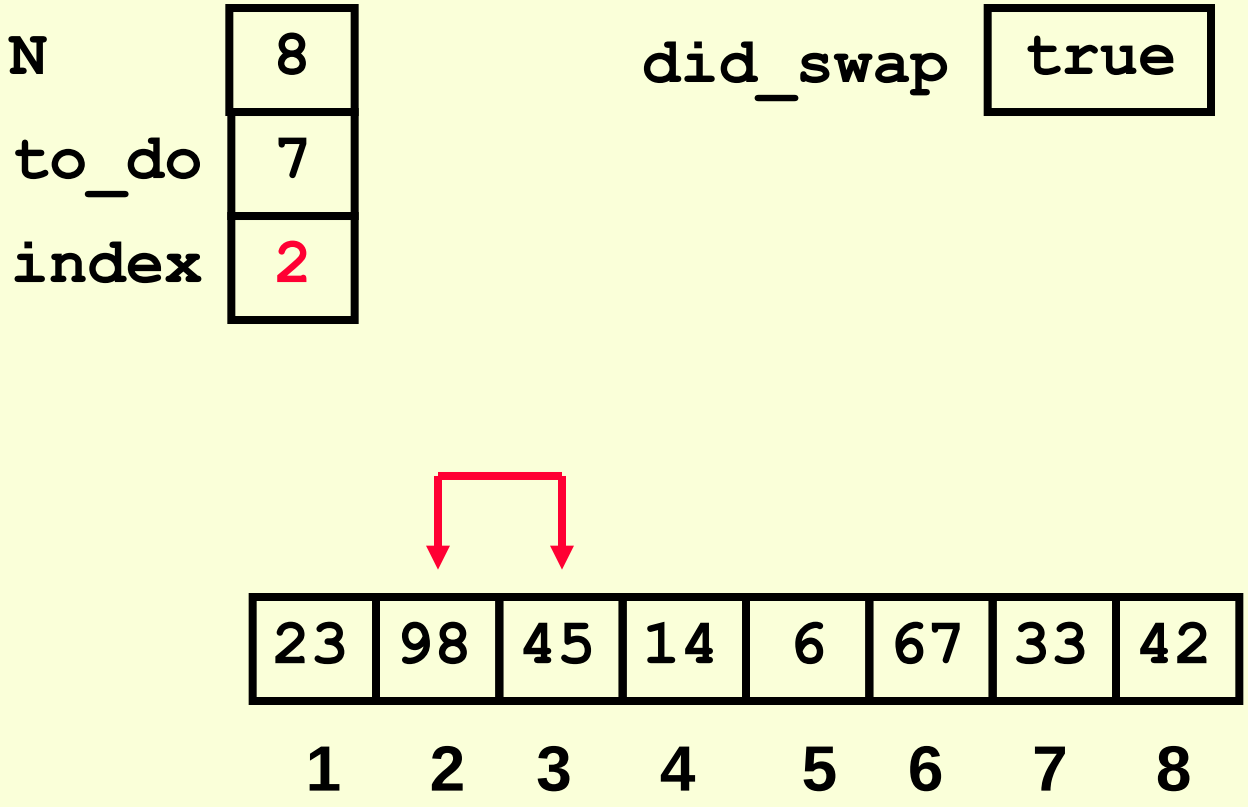
An Animated Example



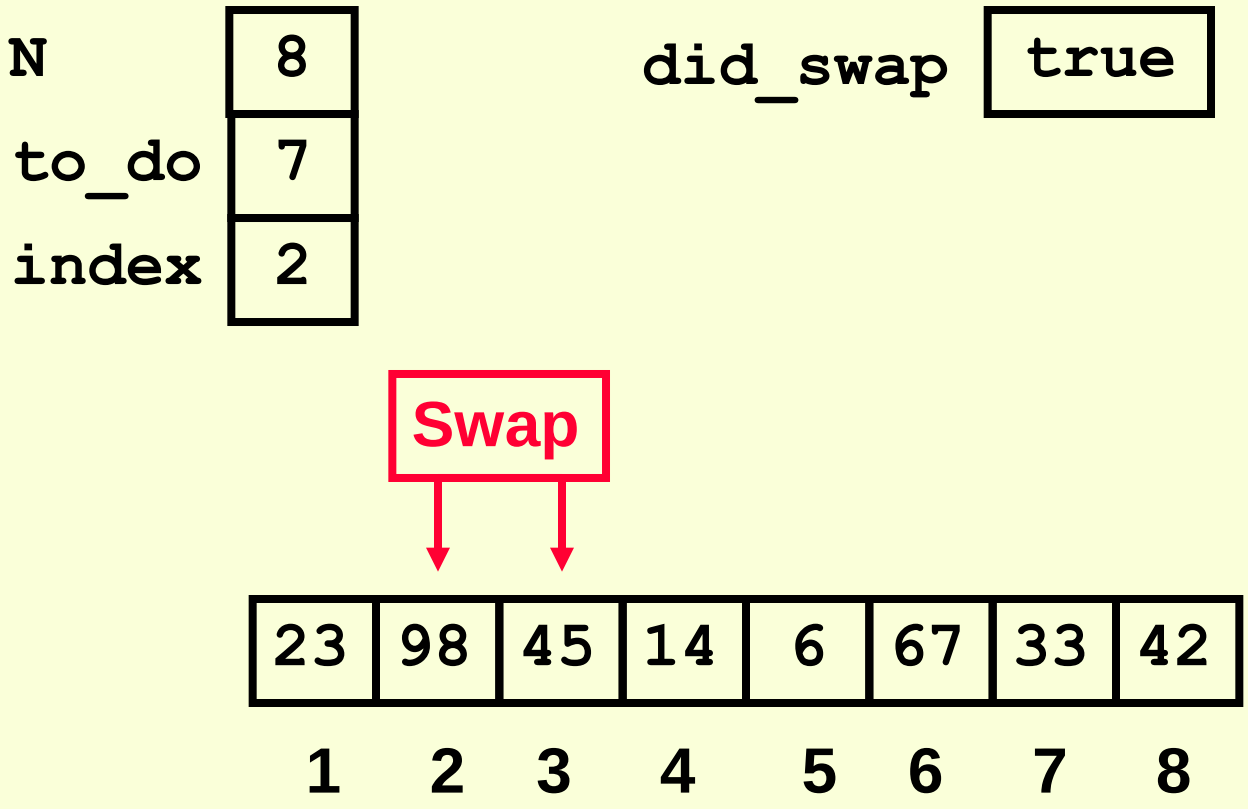
An Animated Example



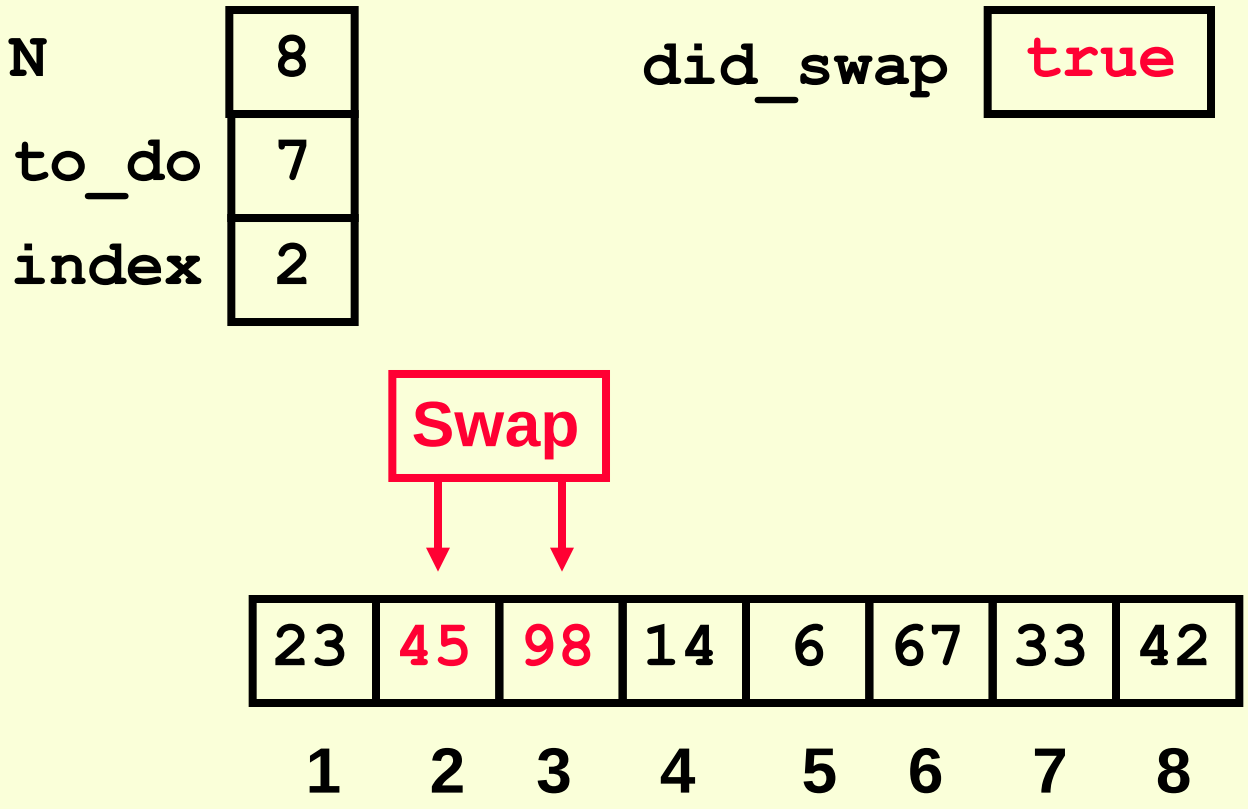
An Animated Example



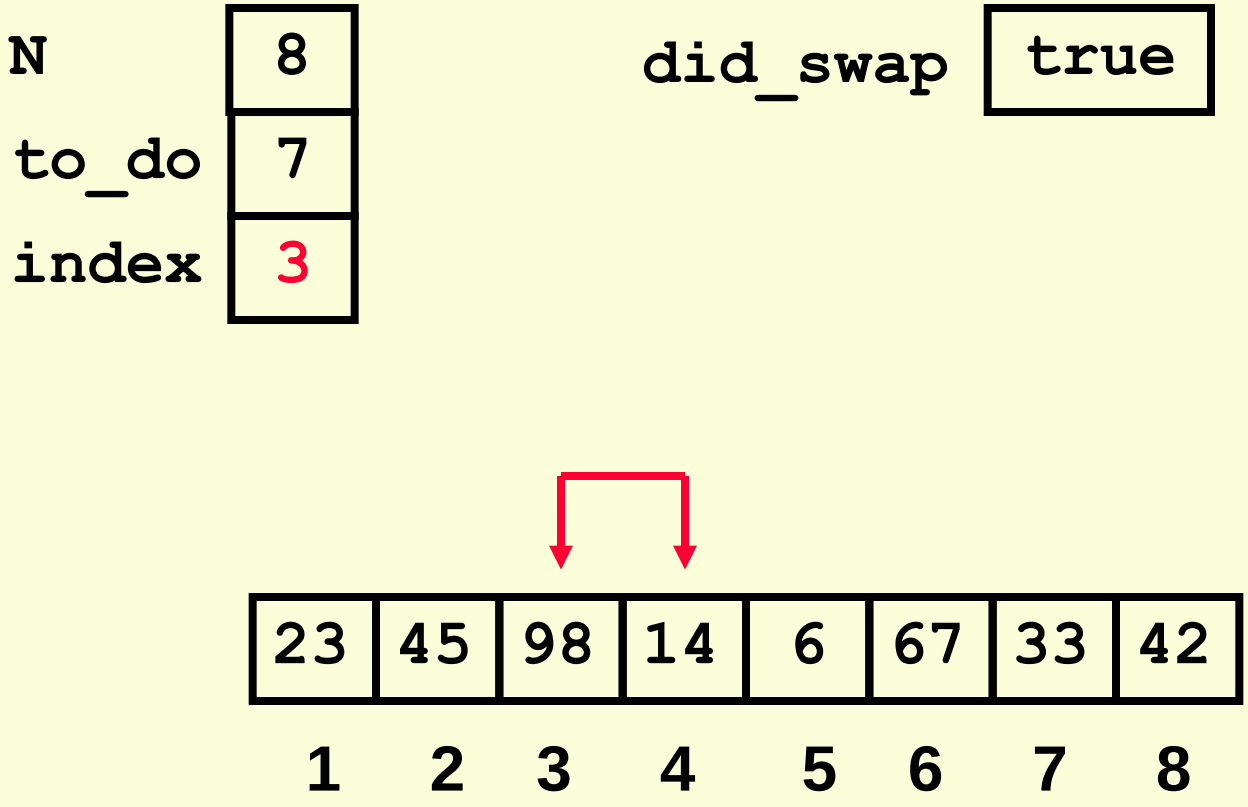
An Animated Example



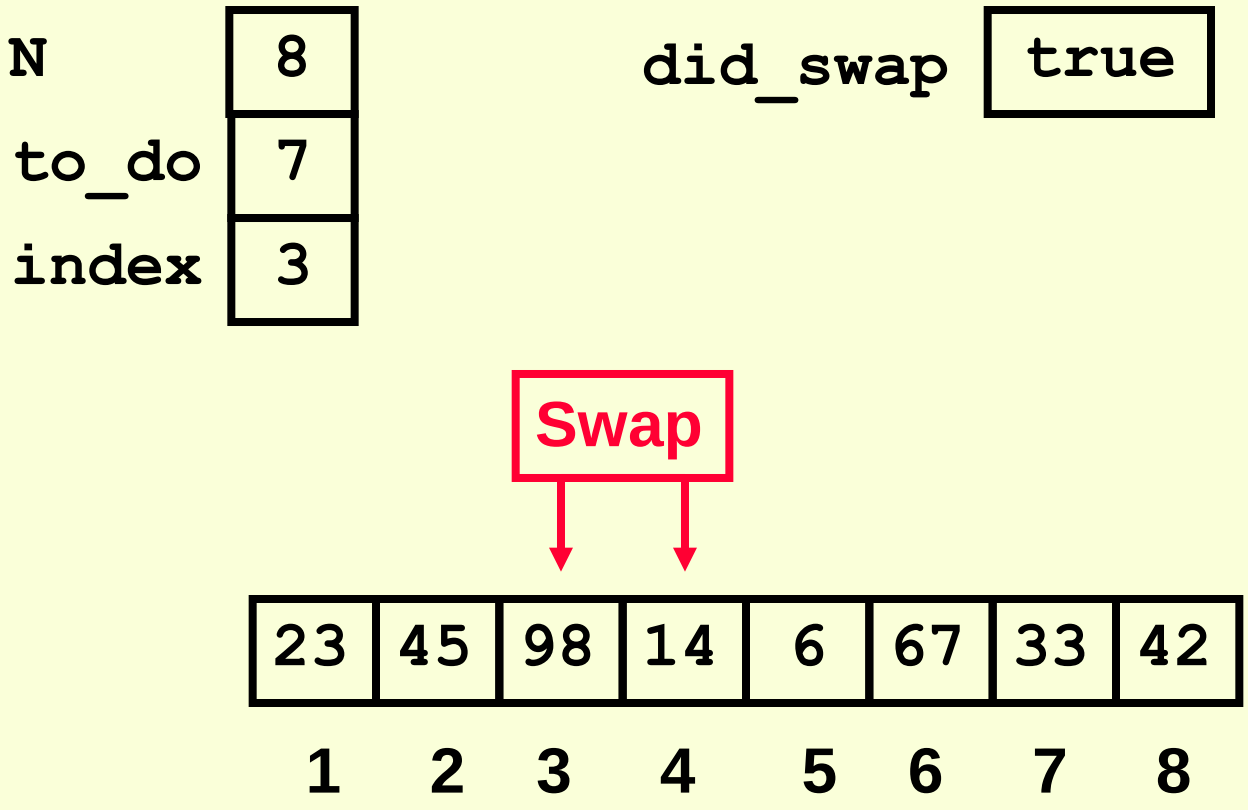
An Animated Example



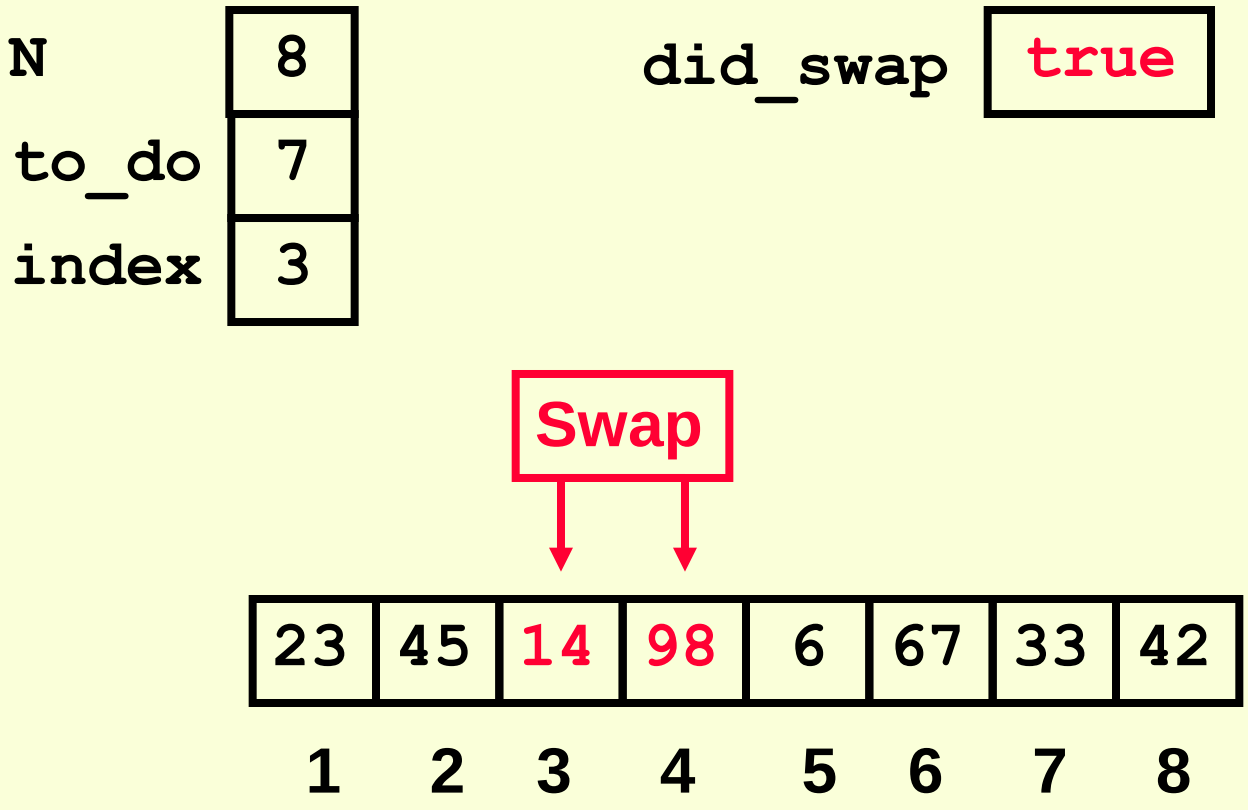
An Animated Example



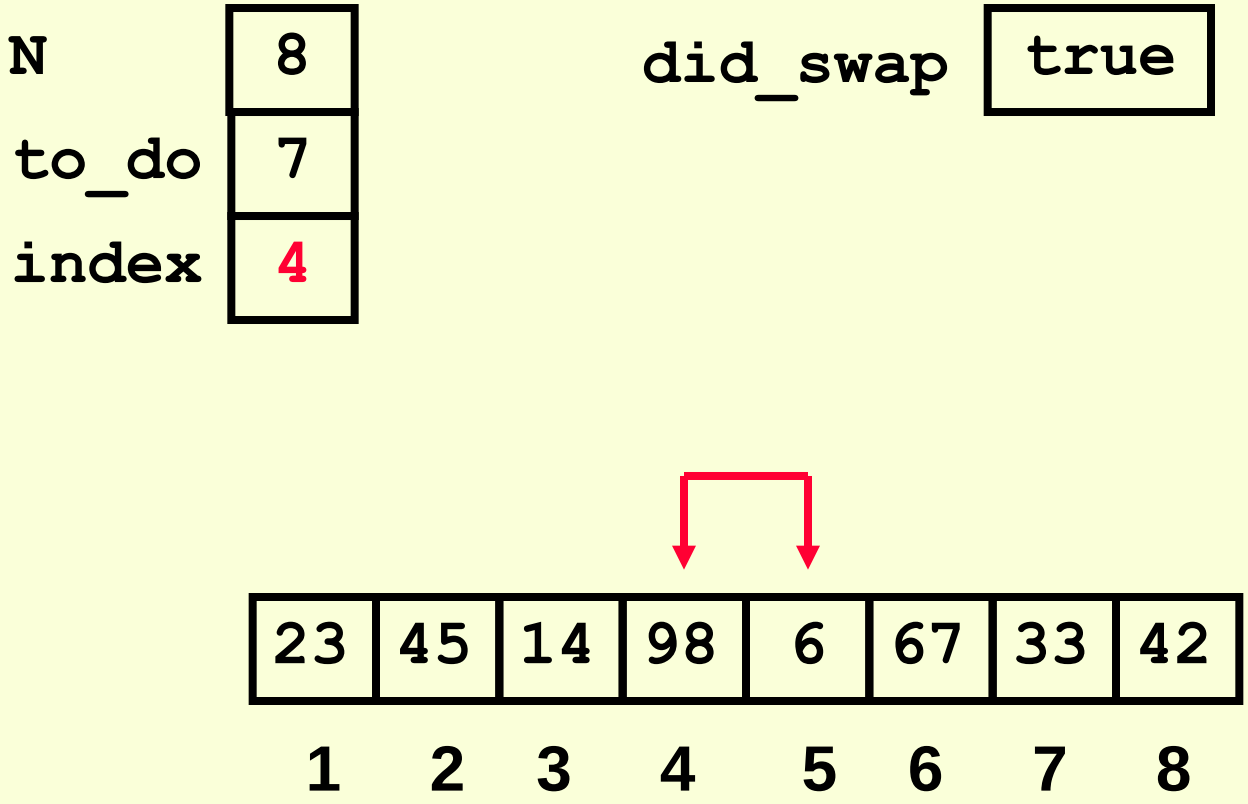
An Animated Example



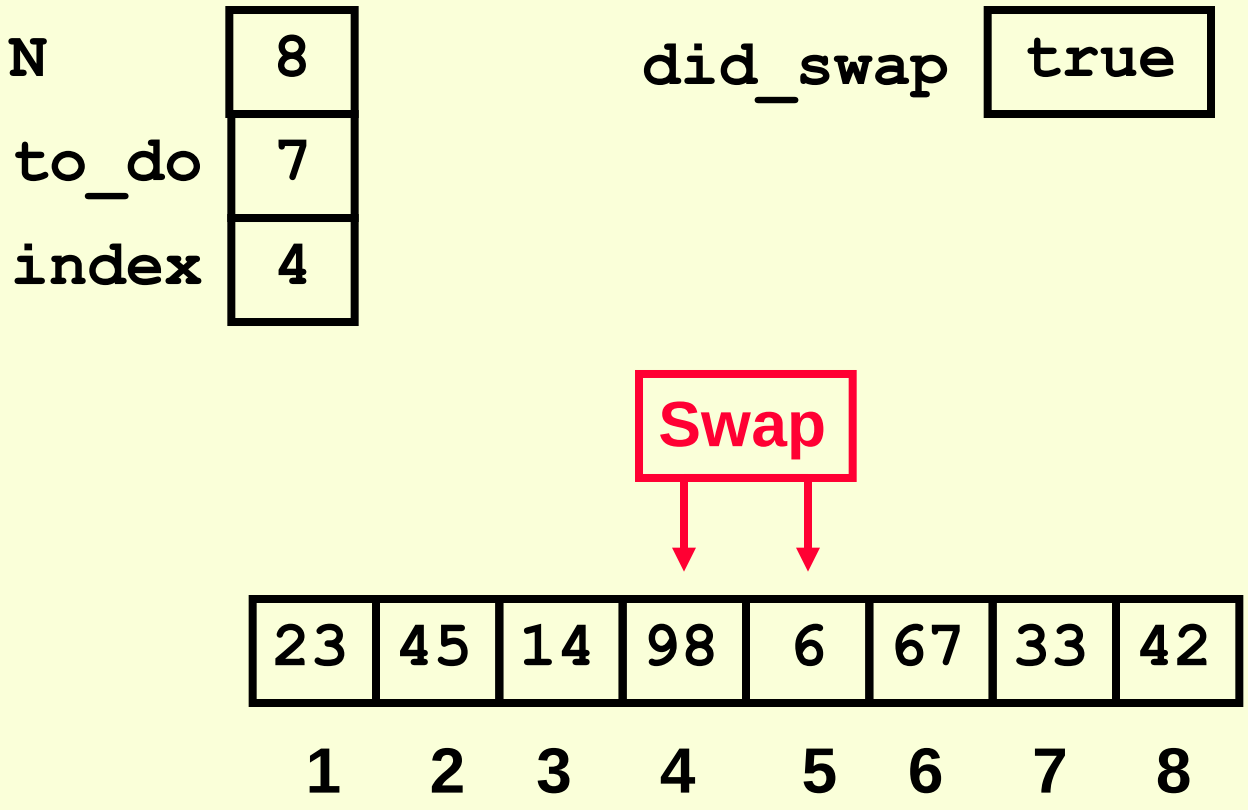
An Animated Example



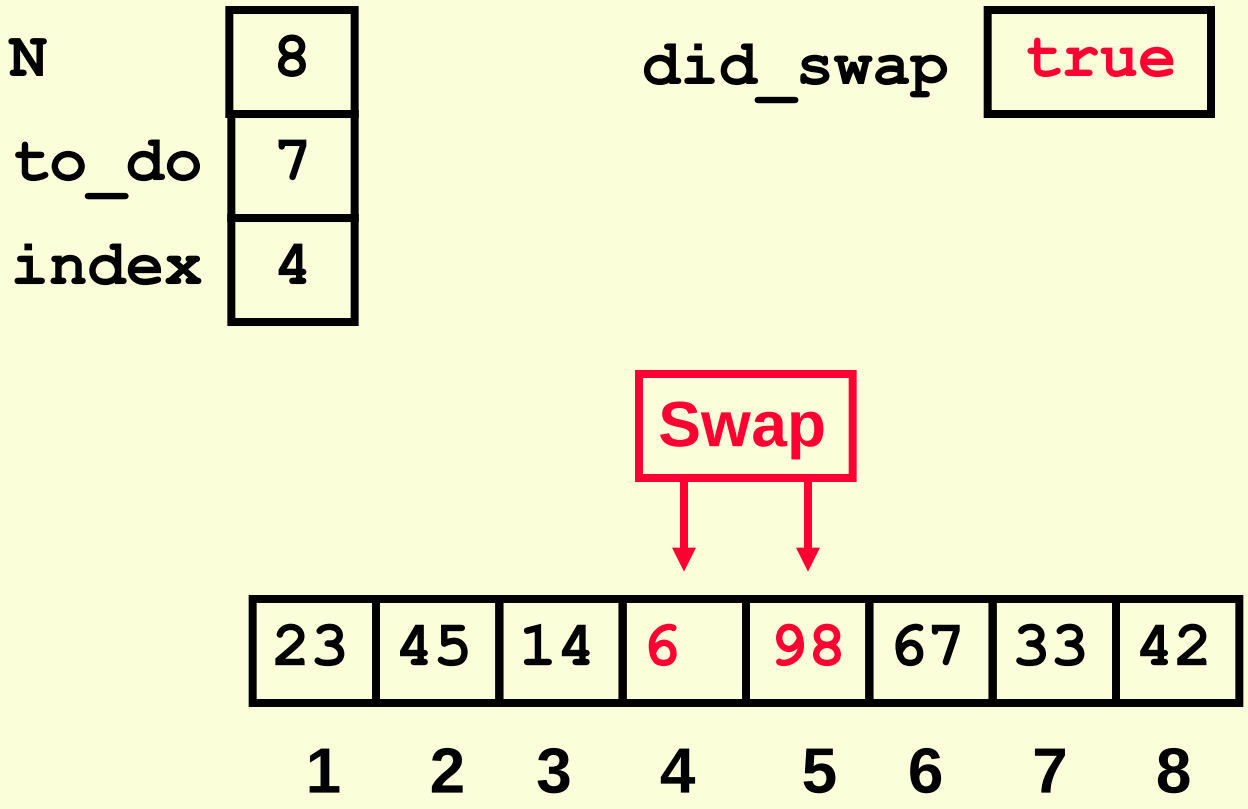
An Animated Example



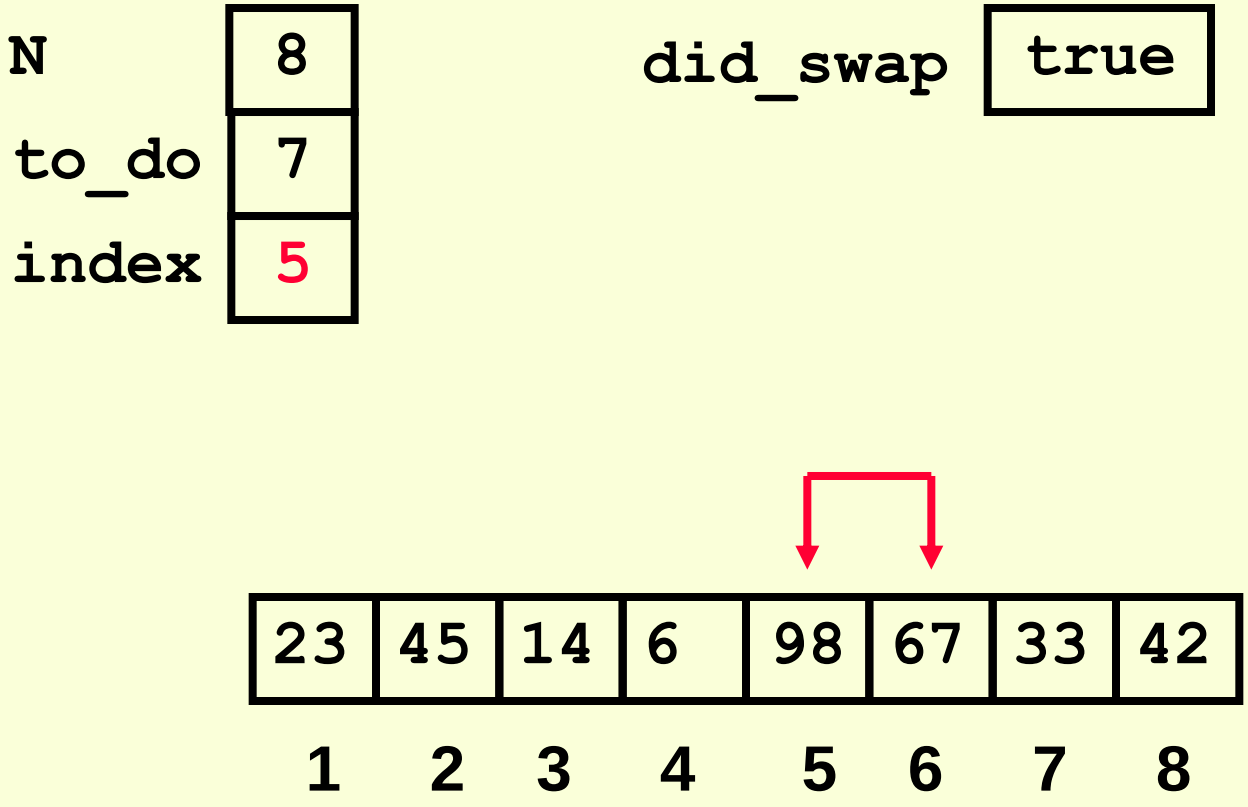
An Animated Example



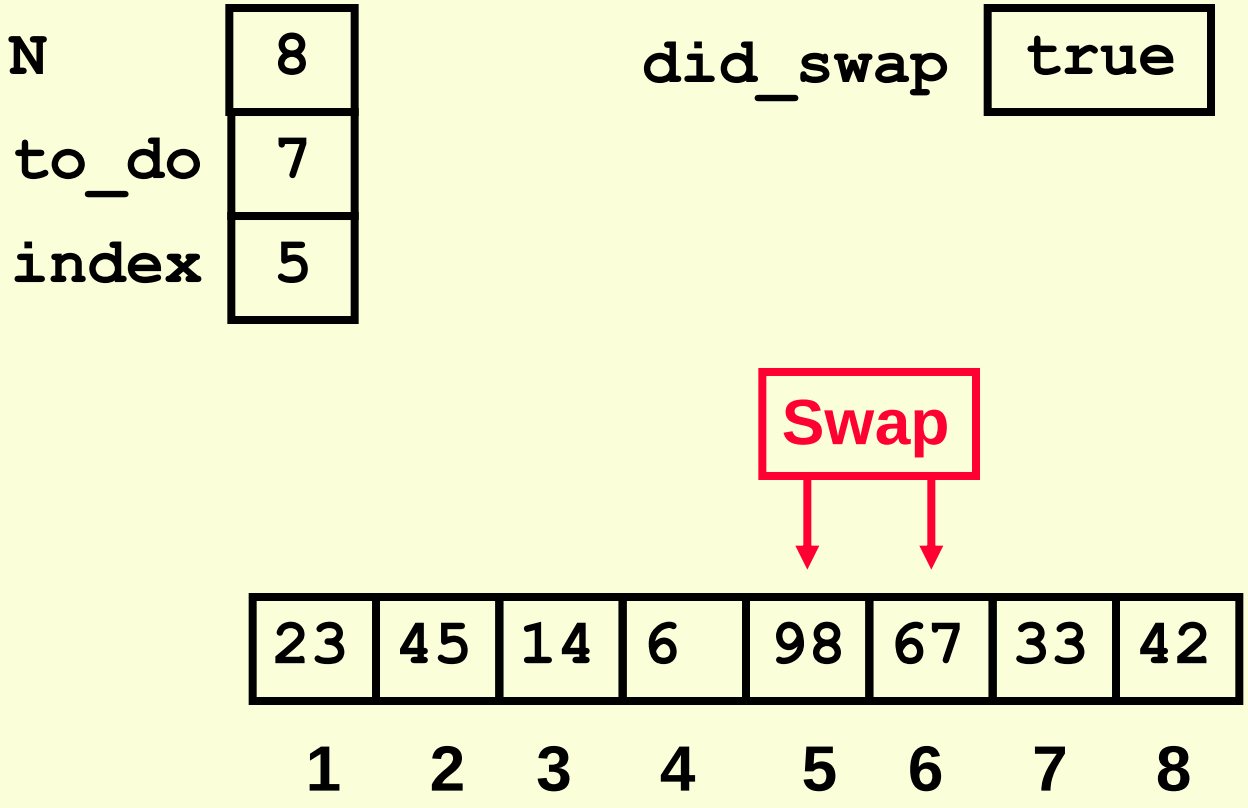
An Animated Example



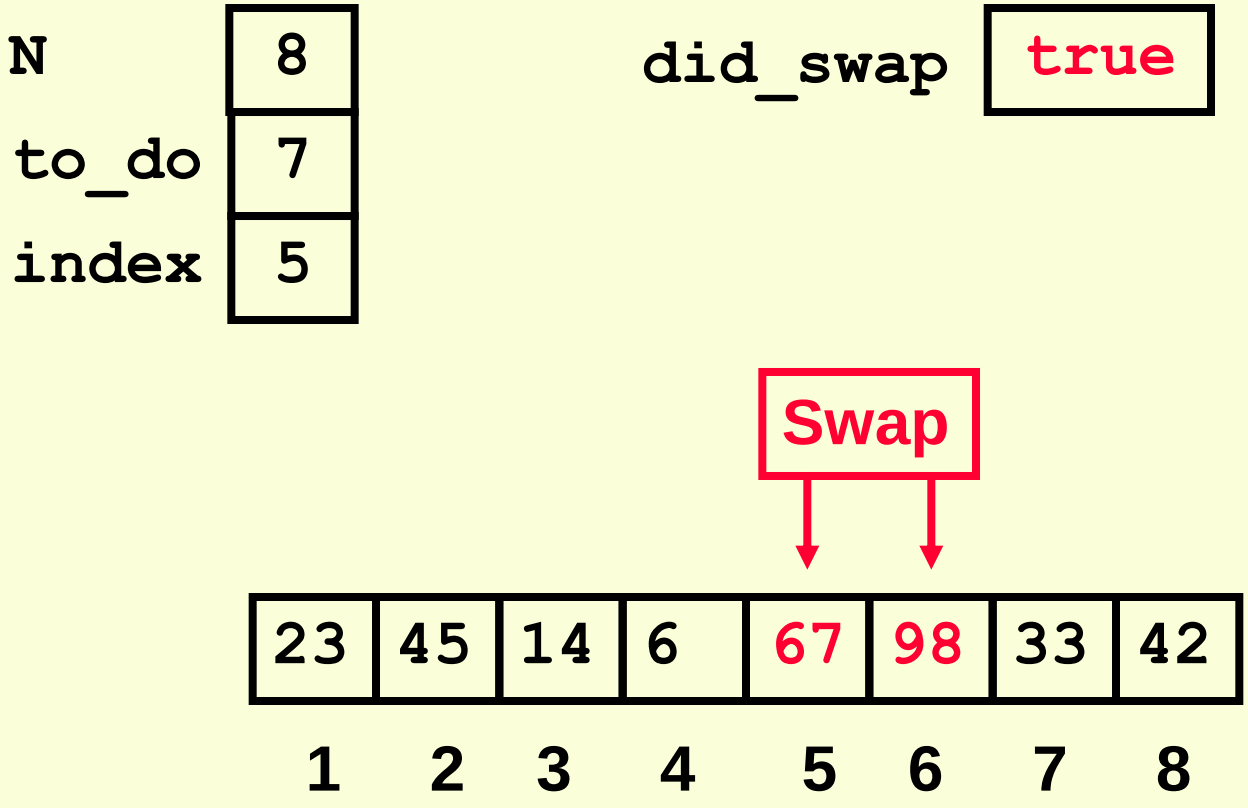
An Animated Example



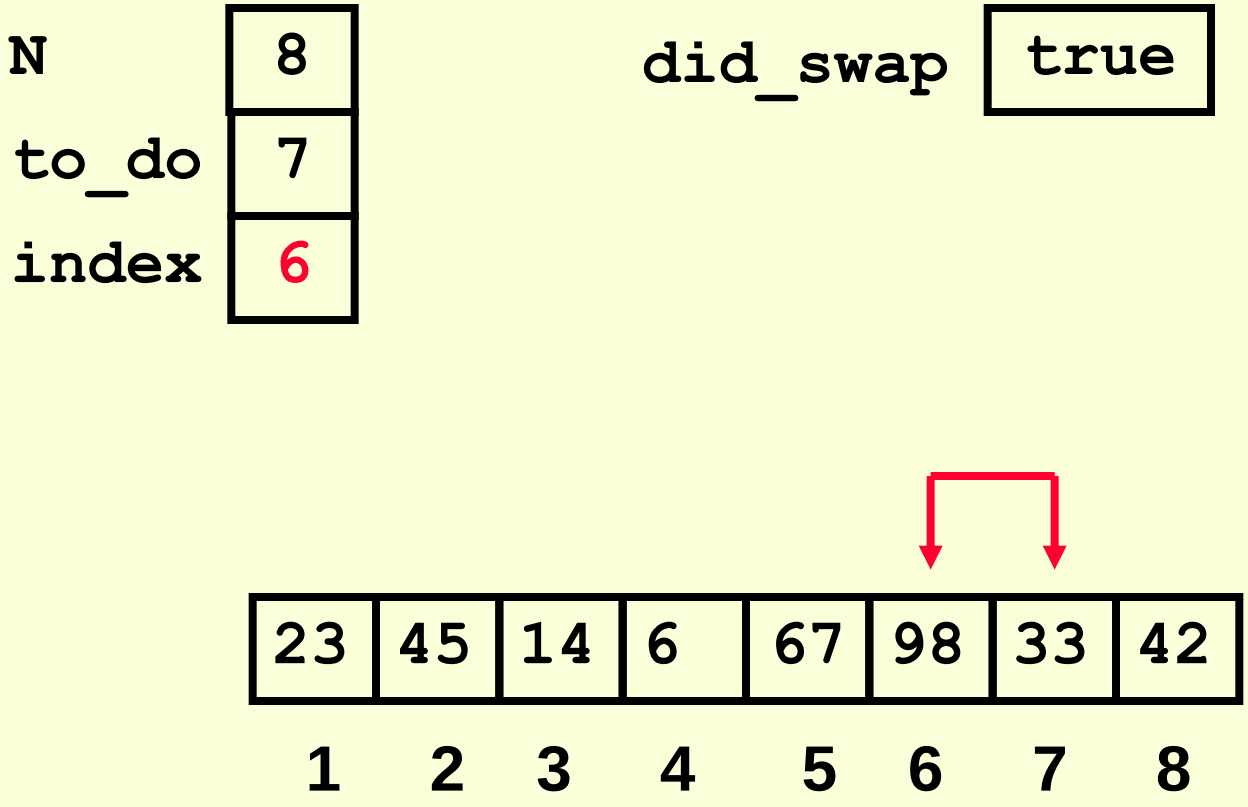
An Animated Example



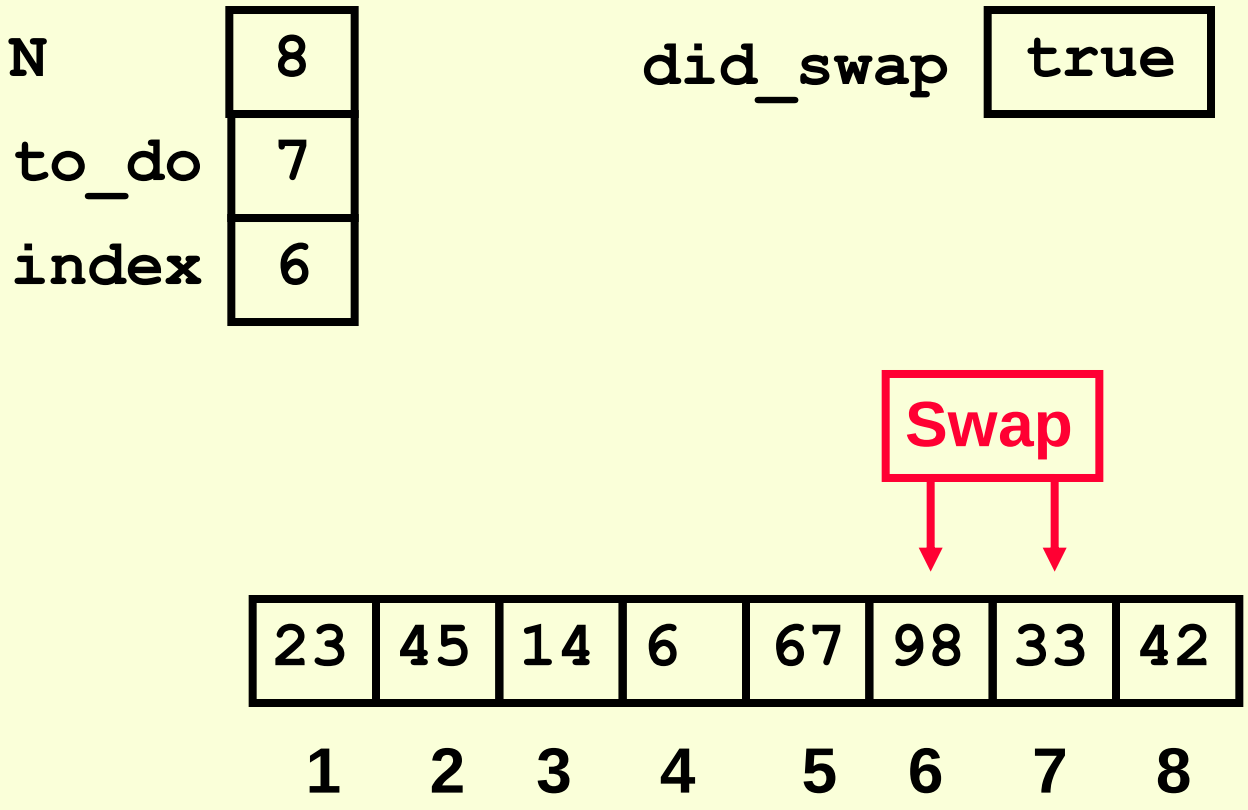
An Animated Example



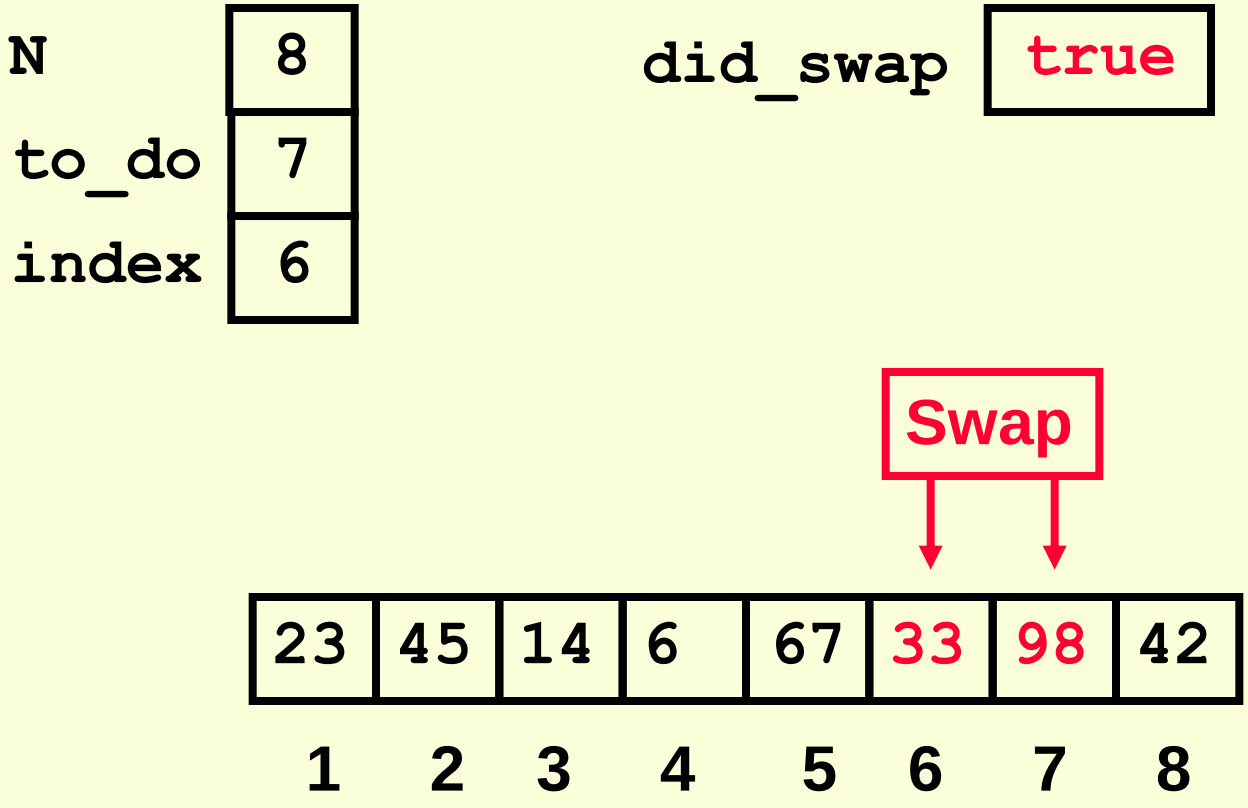
An Animated Example



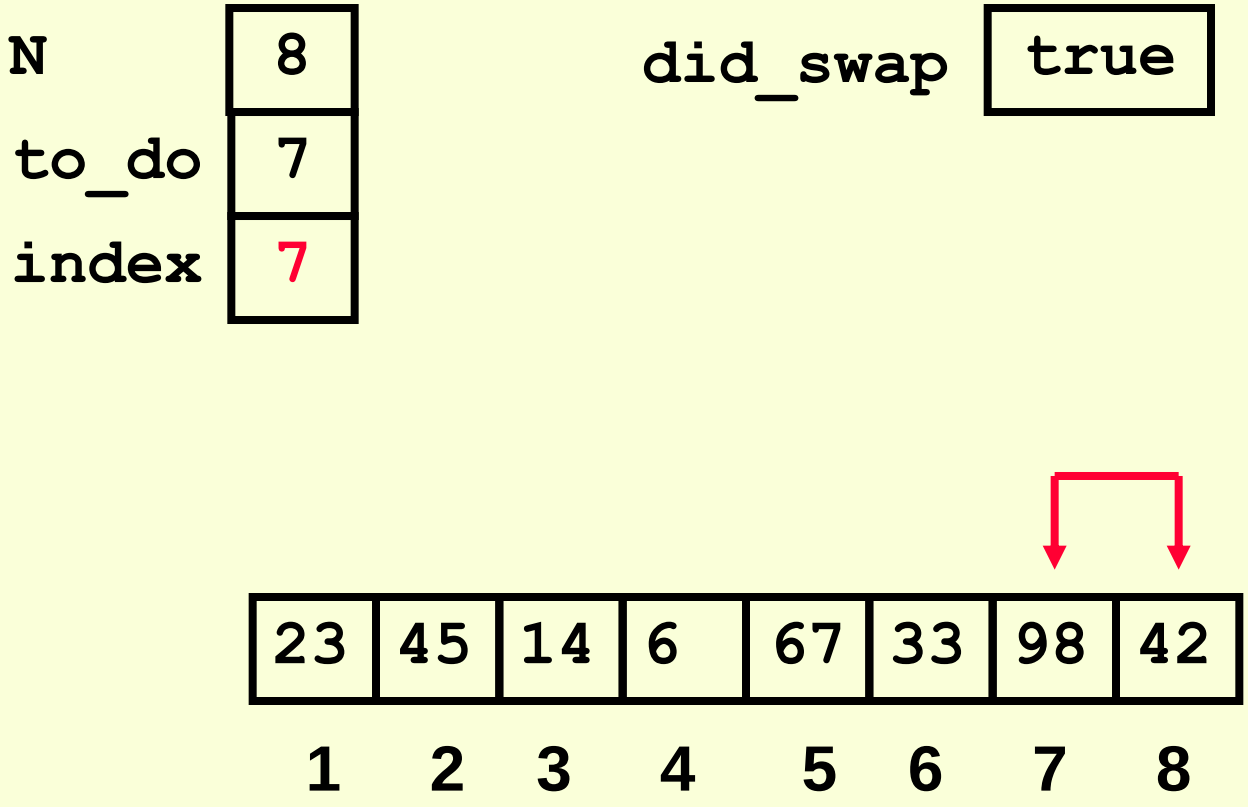
An Animated Example



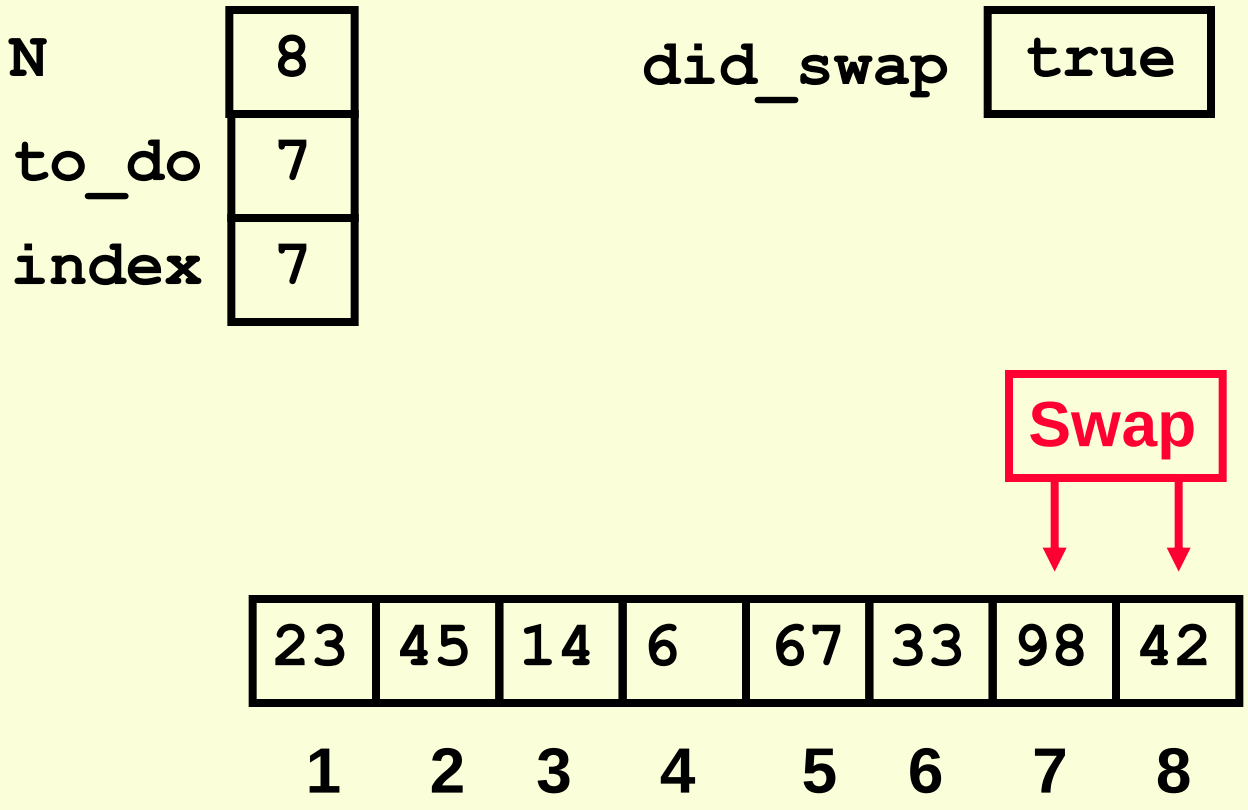
An Animated Example



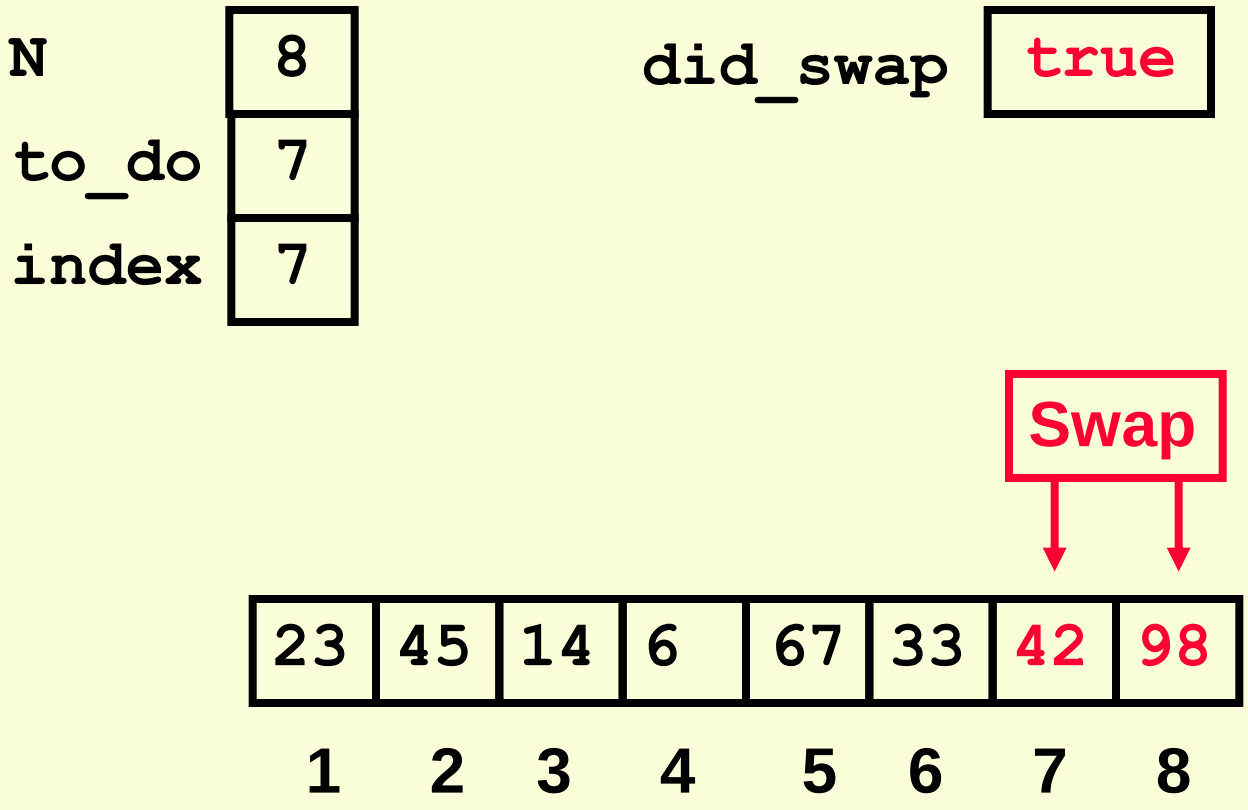
An Animated Example



An Animated Example



An Animated Example

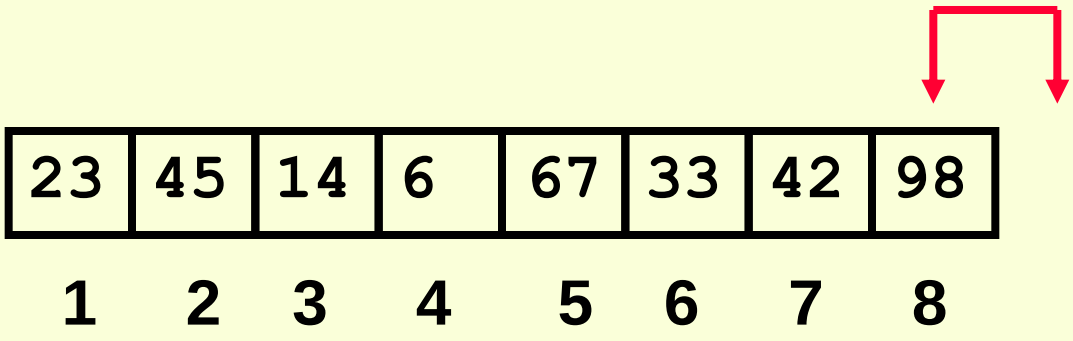


After First Pass of Outer Loop

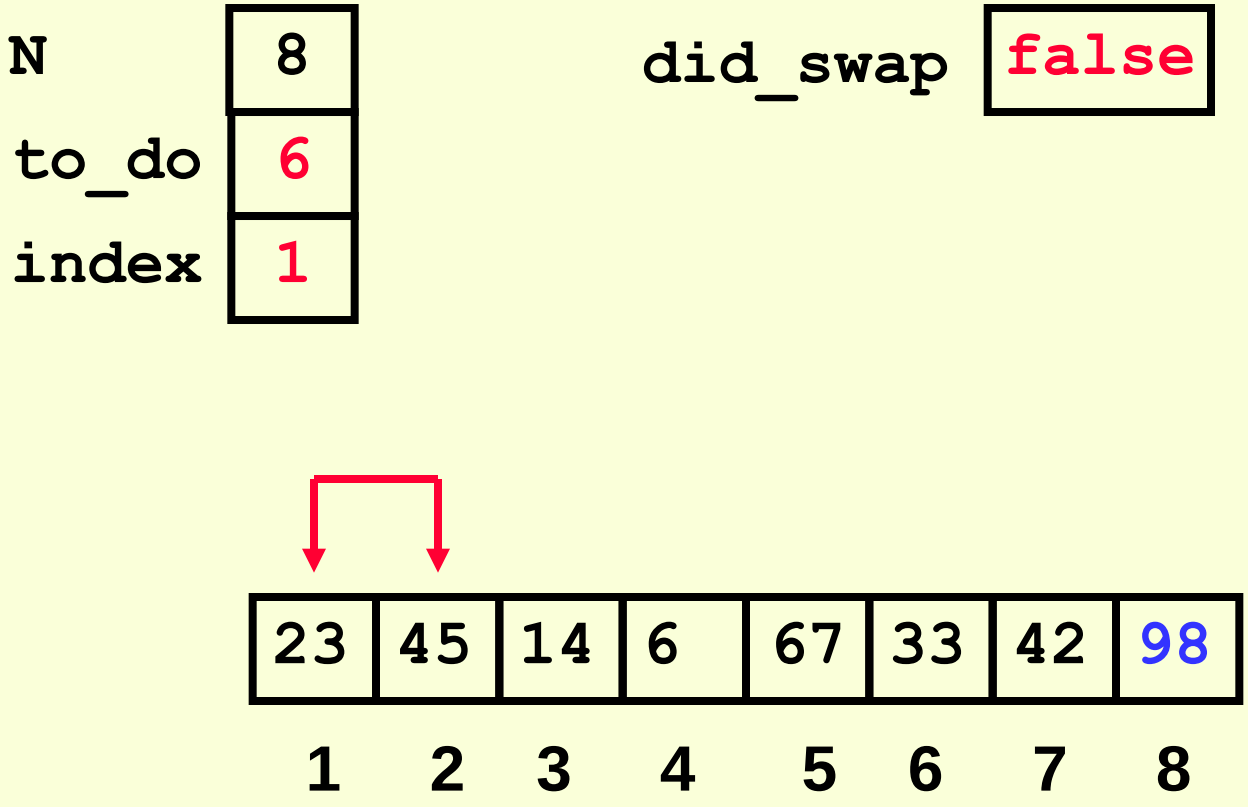
N	8
to_do	7
index	8

did_swap	true

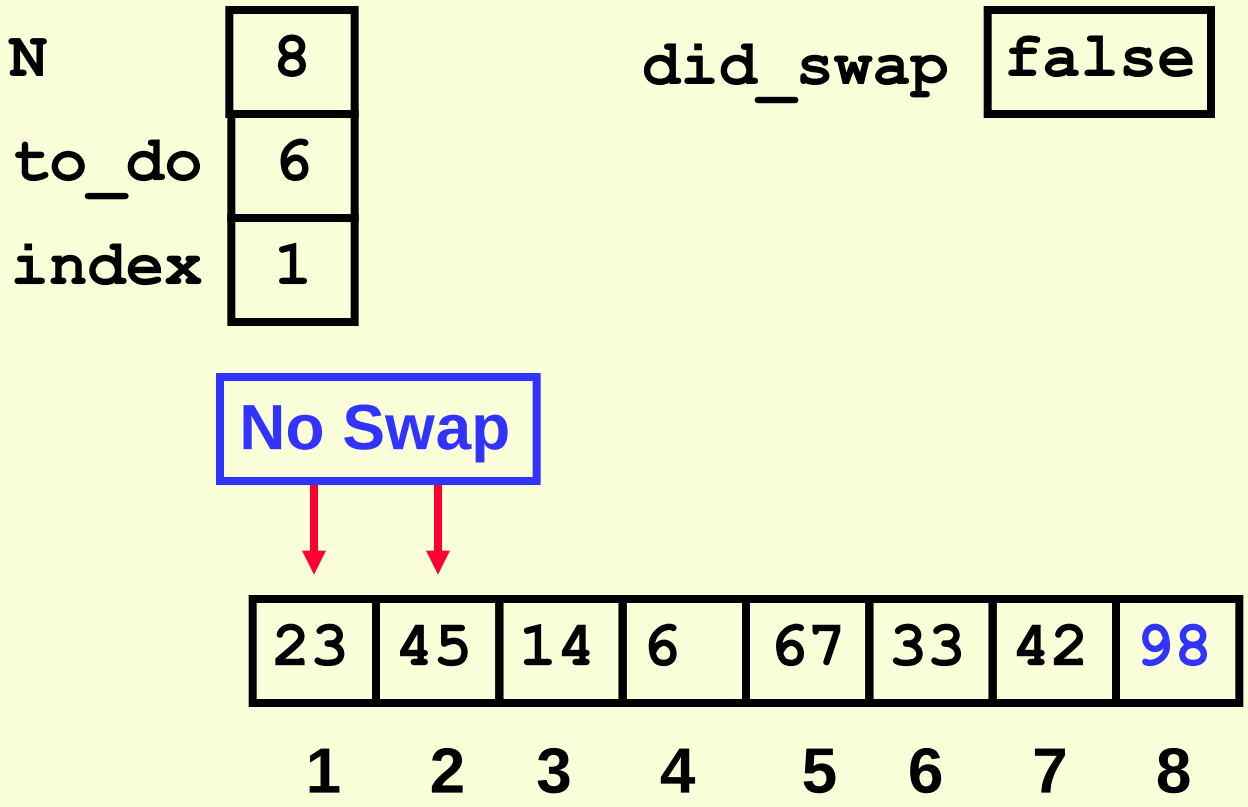
Finished first “Bubble Up”



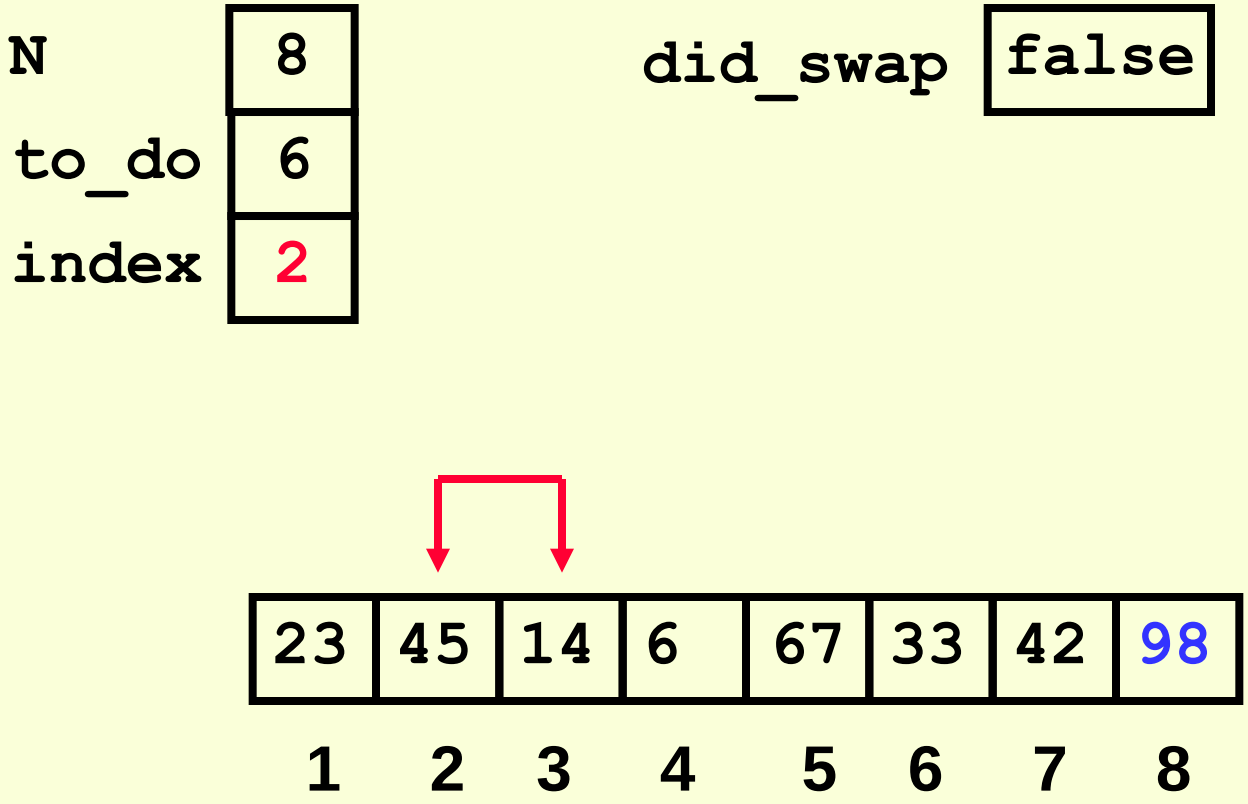
The Second “Bubble Up”



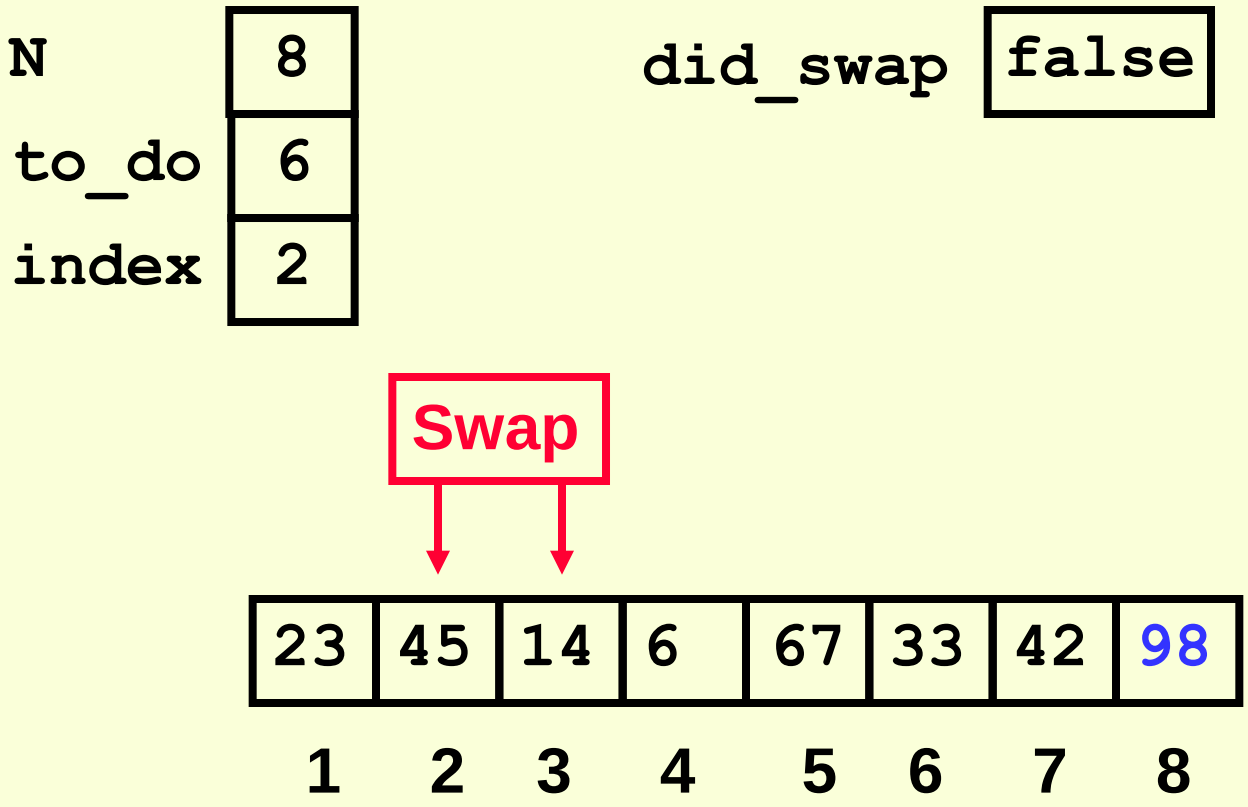
The Second “Bubble Up”



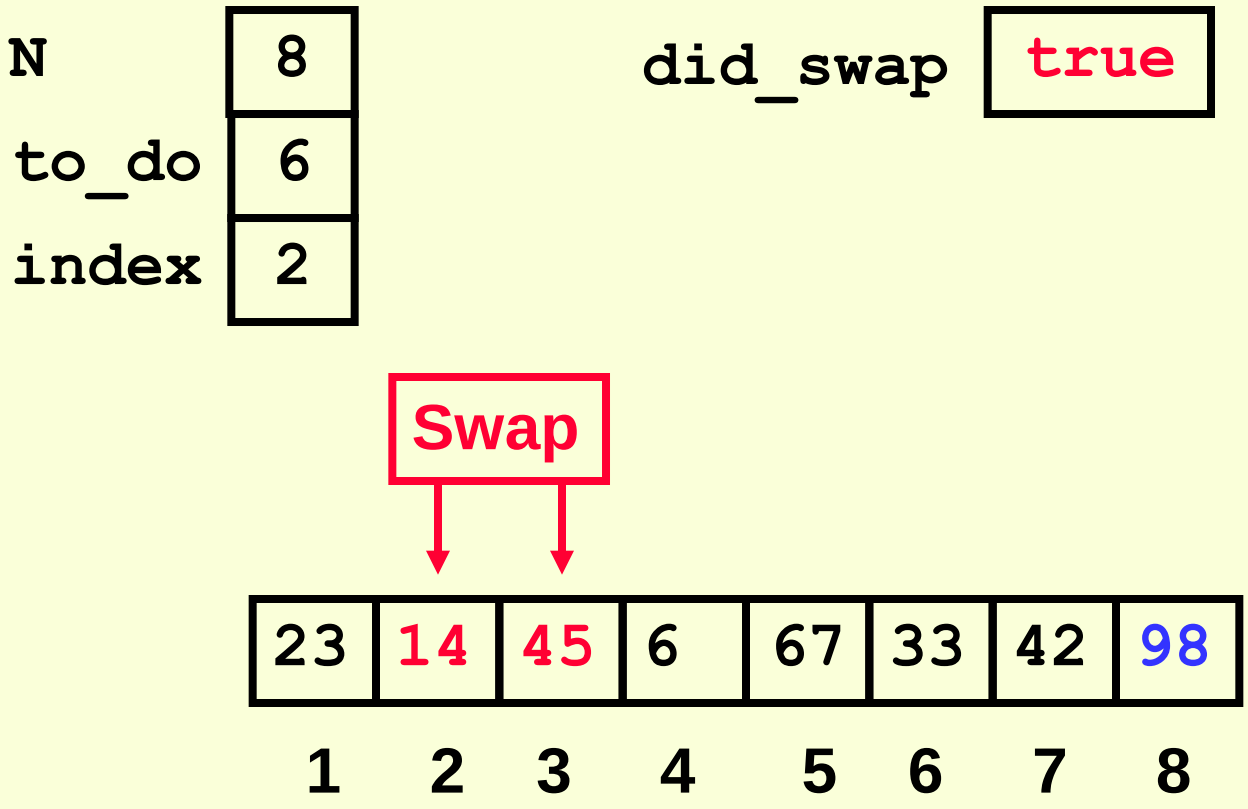
The Second “Bubble Up”



The Second “Bubble Up”



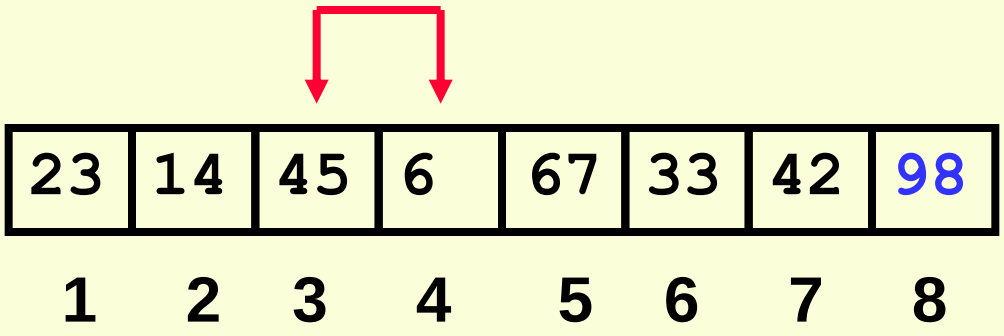
The Second “Bubble Up”



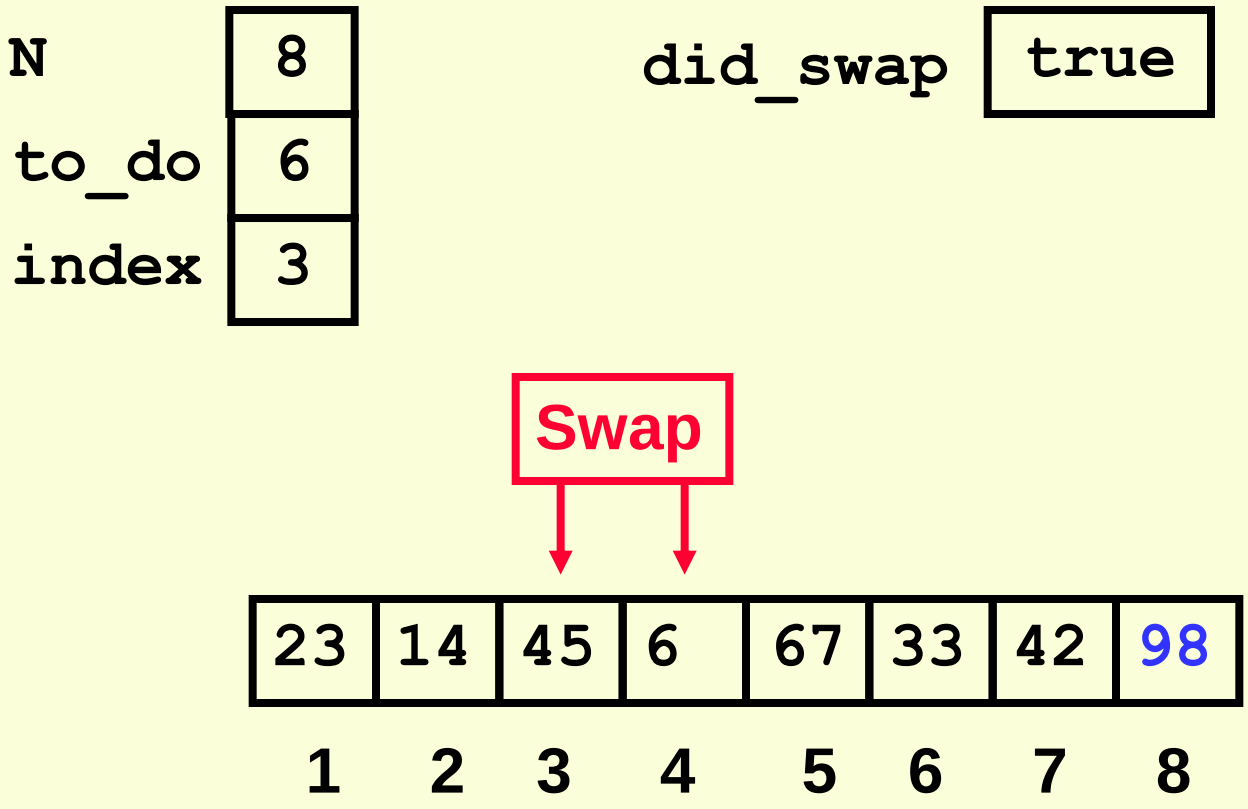
The Second “Bubble Up”

N	8
to_do	6
index	3

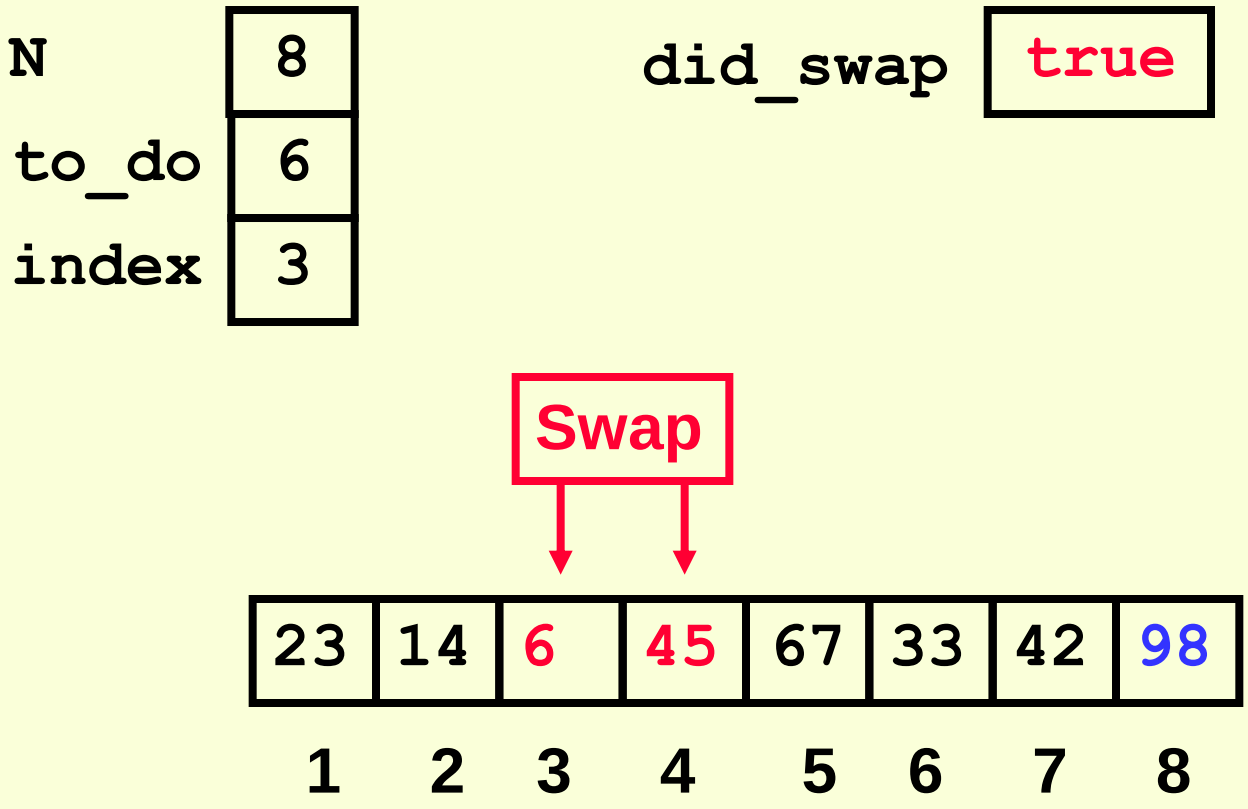
did_swap	true



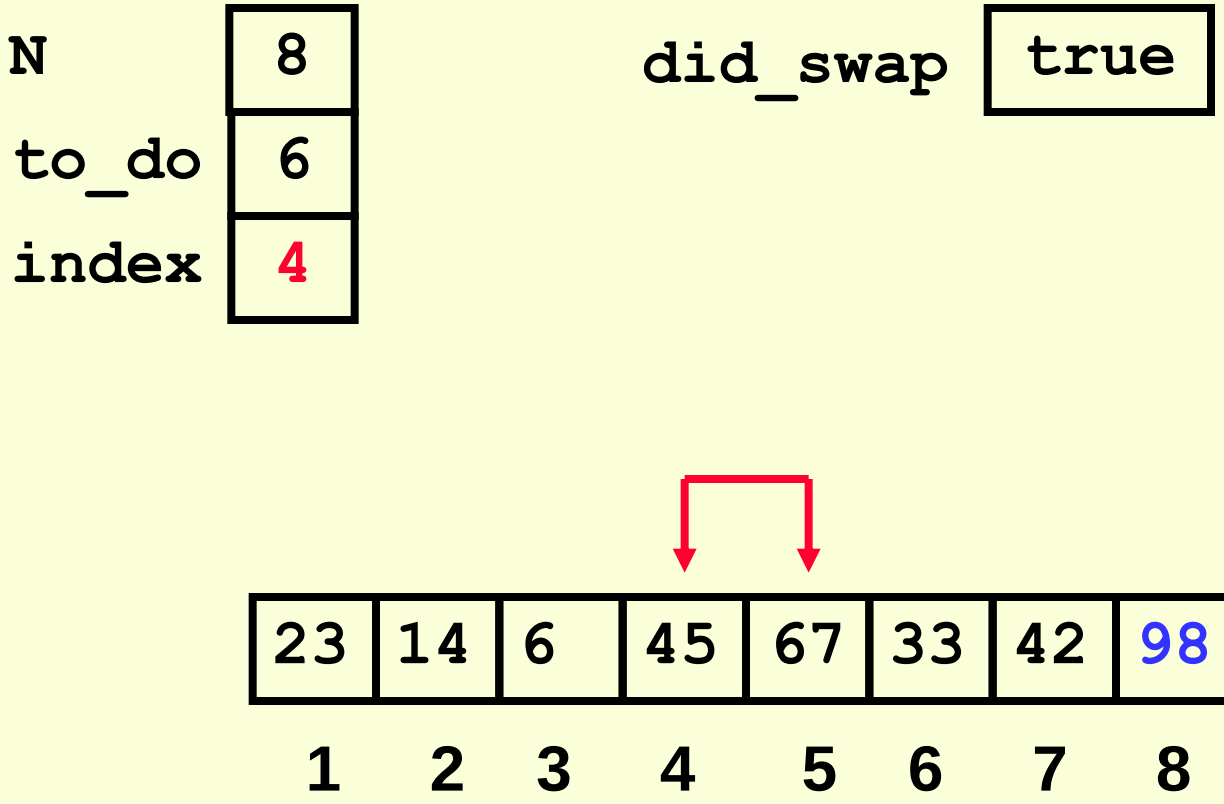
The Second “Bubble Up”



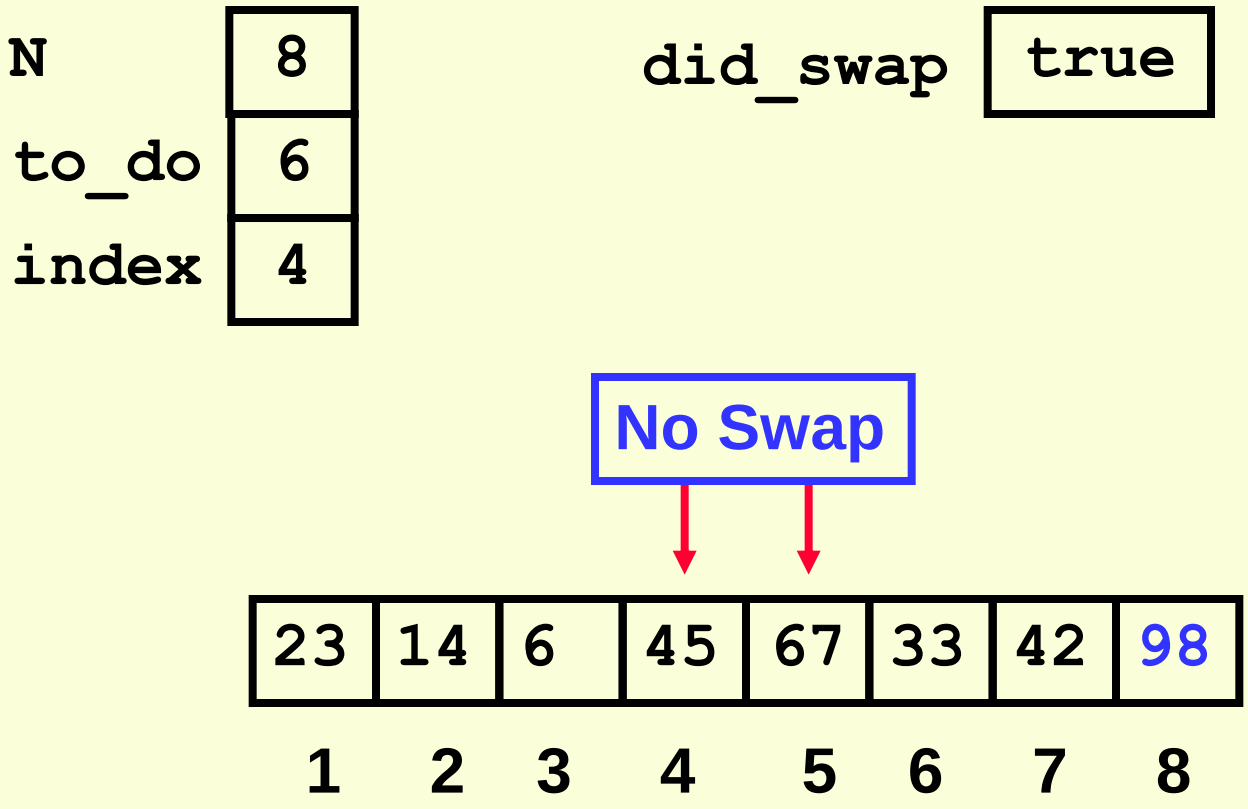
The Second “Bubble Up”



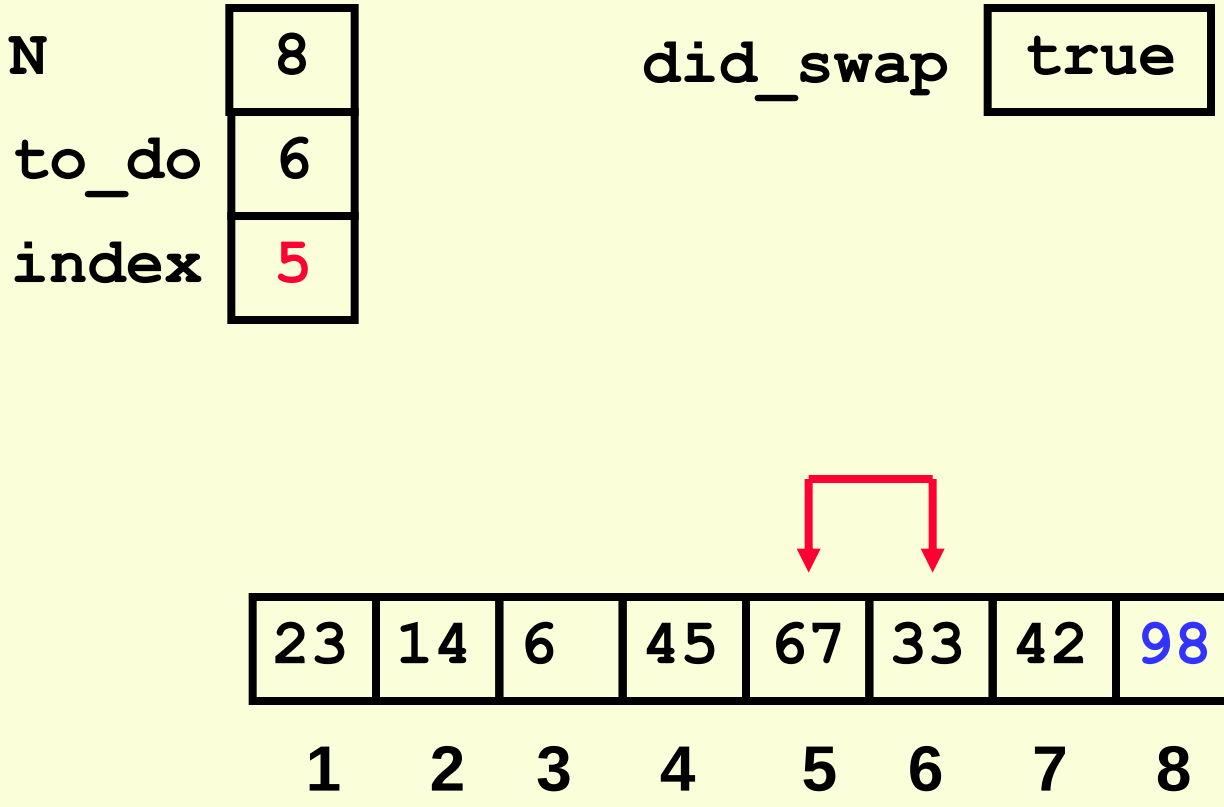
The Second “Bubble Up”



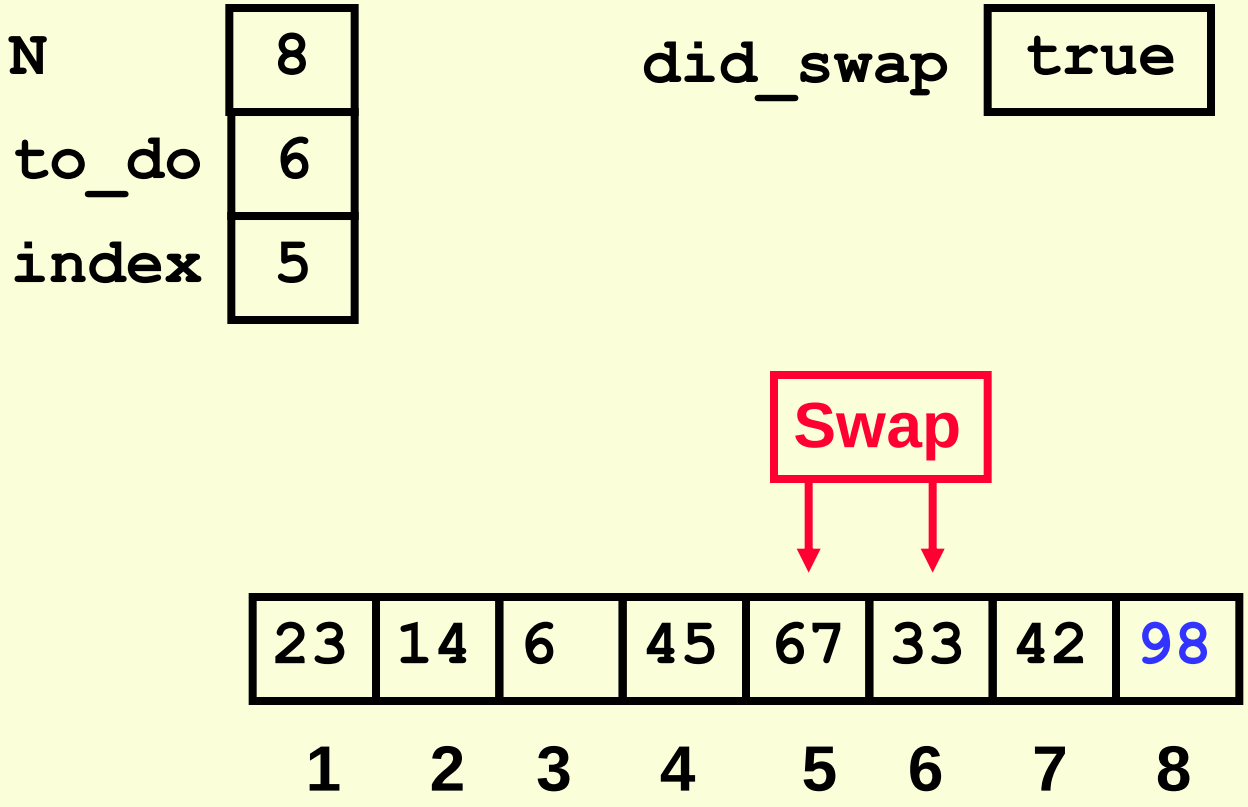
The Second “Bubble Up”



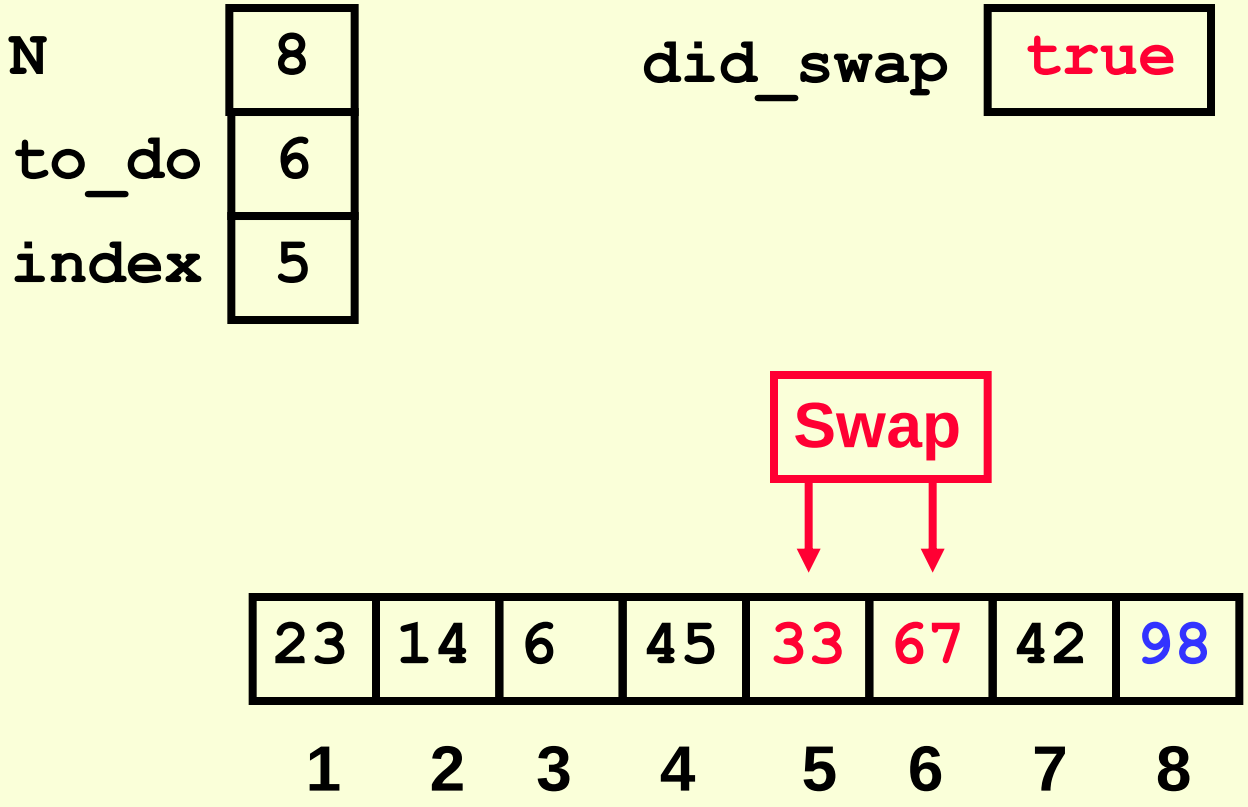
The Second “Bubble Up”



The Second “Bubble Up”



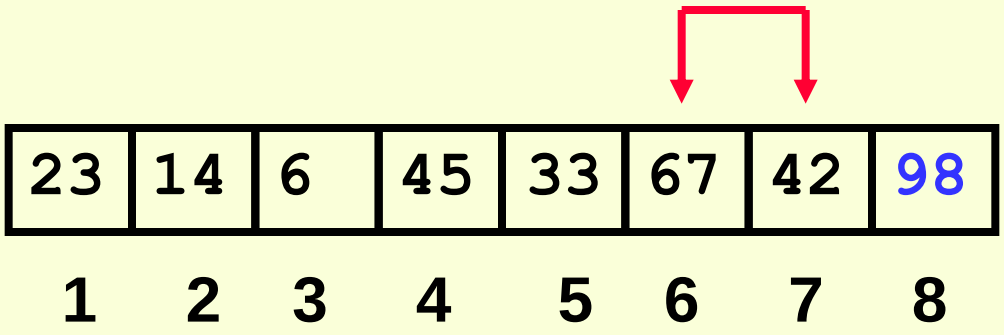
The Second “Bubble Up”



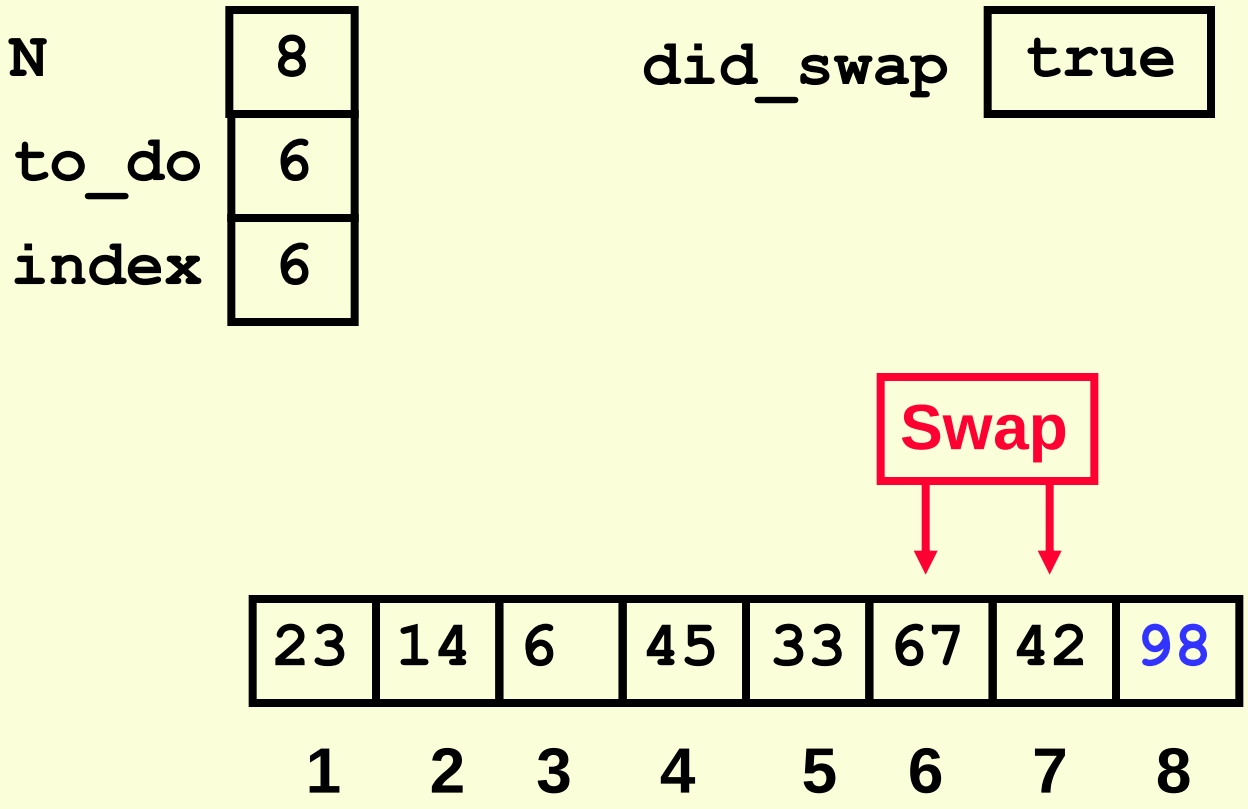
The Second “Bubble Up”

N	8
to_do	6
index	6

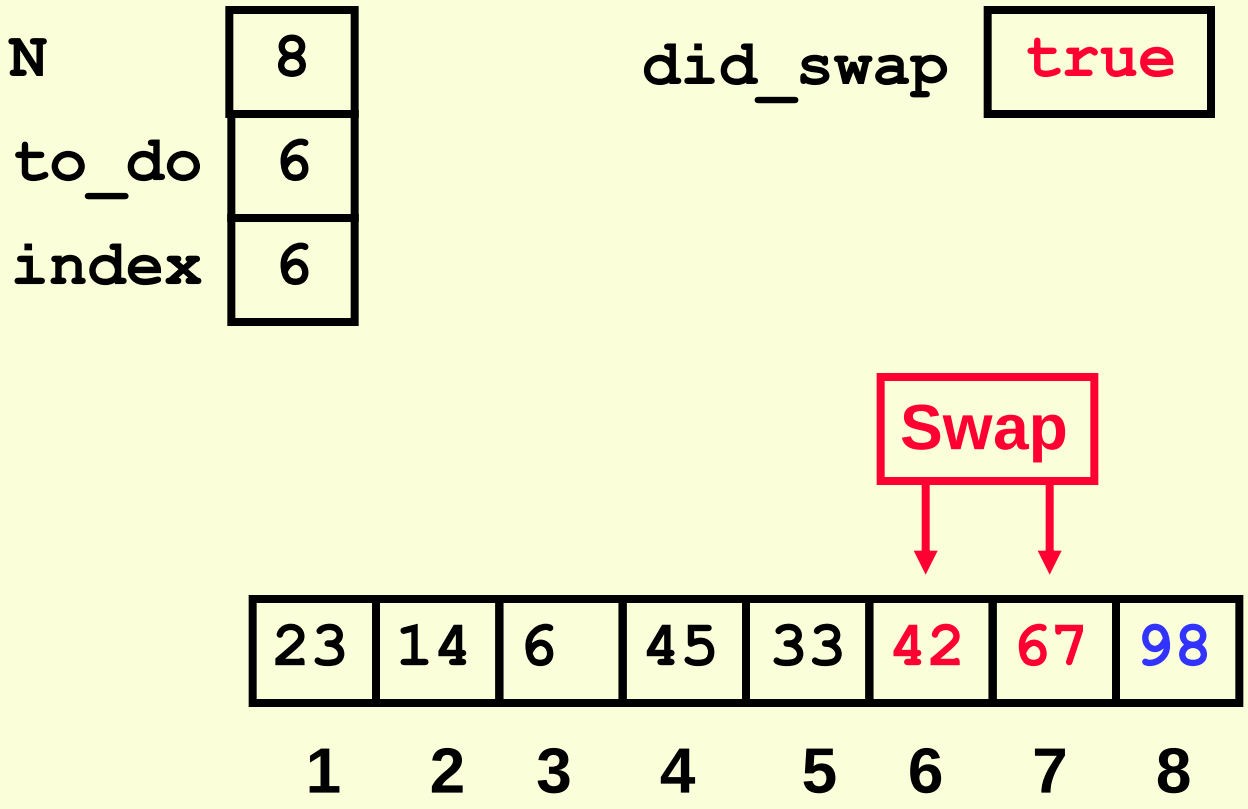
did_swap	true



The Second “Bubble Up”



The Second “Bubble Up”

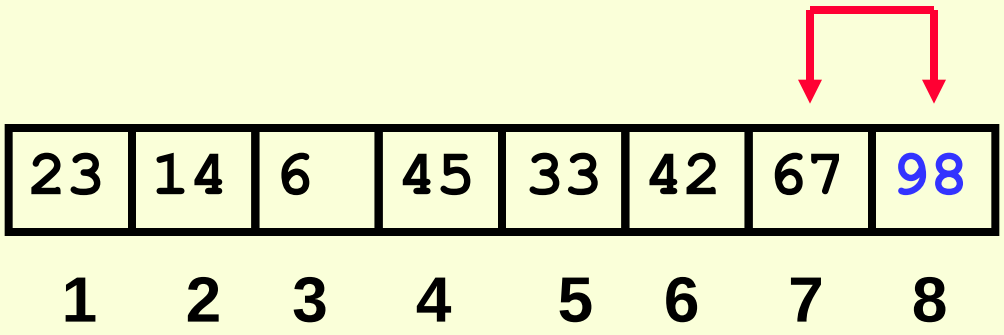


After Second Pass of Outer Loop

N	8
to_do	6
index	7

did_swap	true

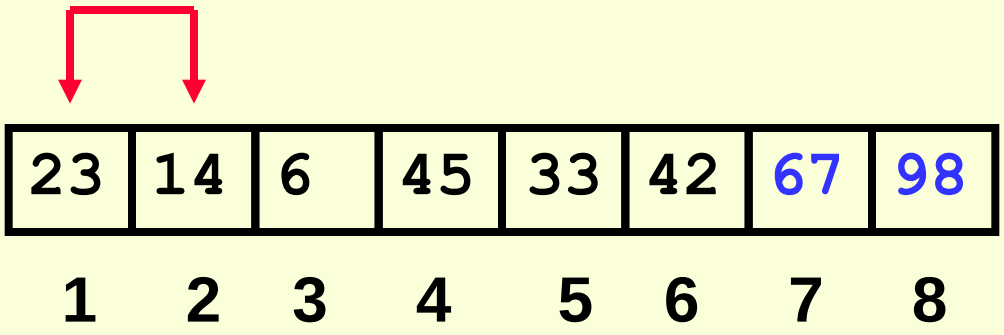
Finished second “Bubble Up”



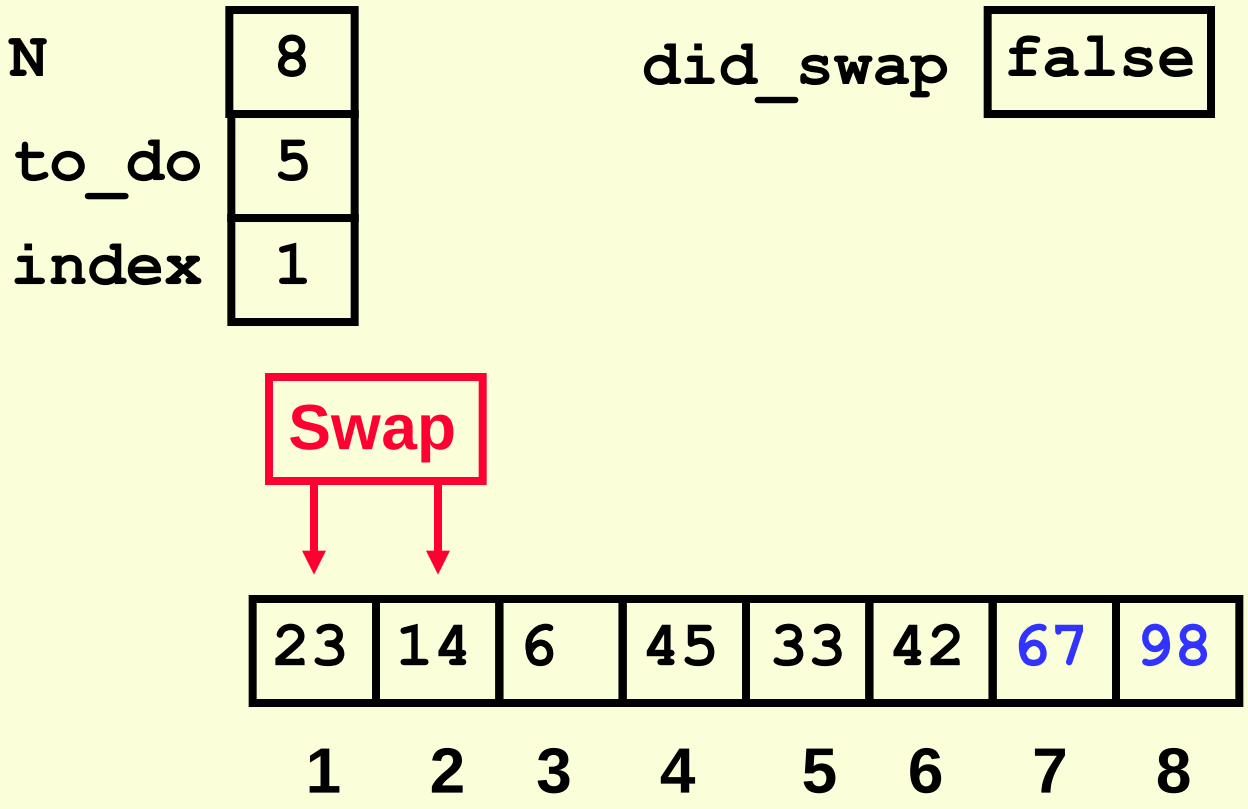
The Third “Bubble Up”

N	8
to_do	5
index	1

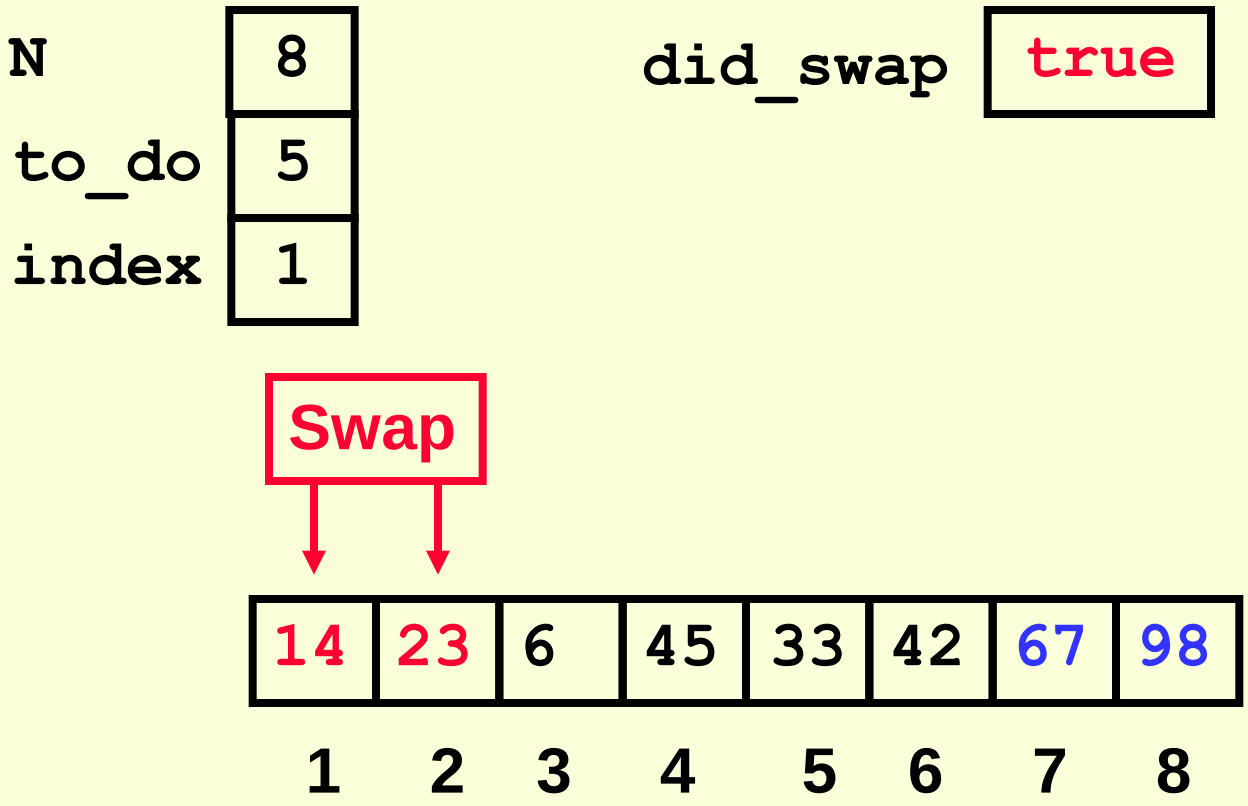
did_swap	false



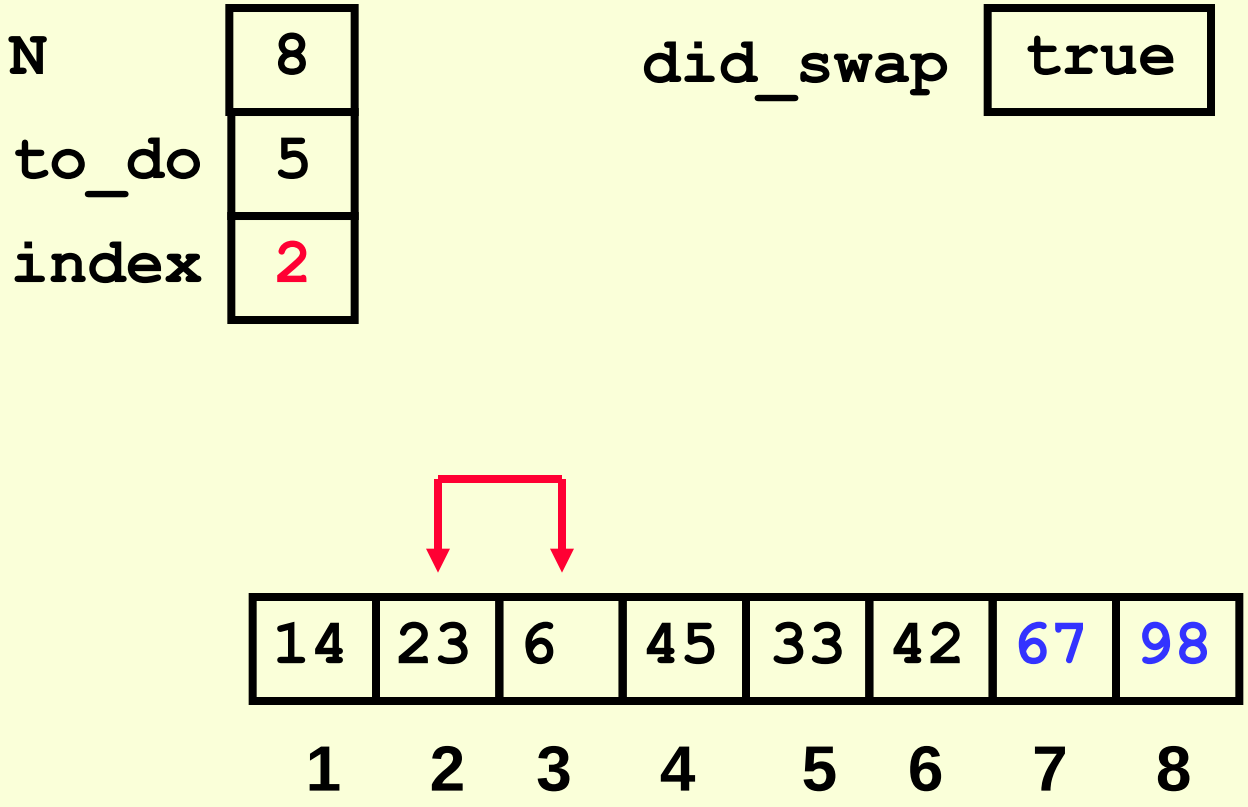
The Third “Bubble Up”



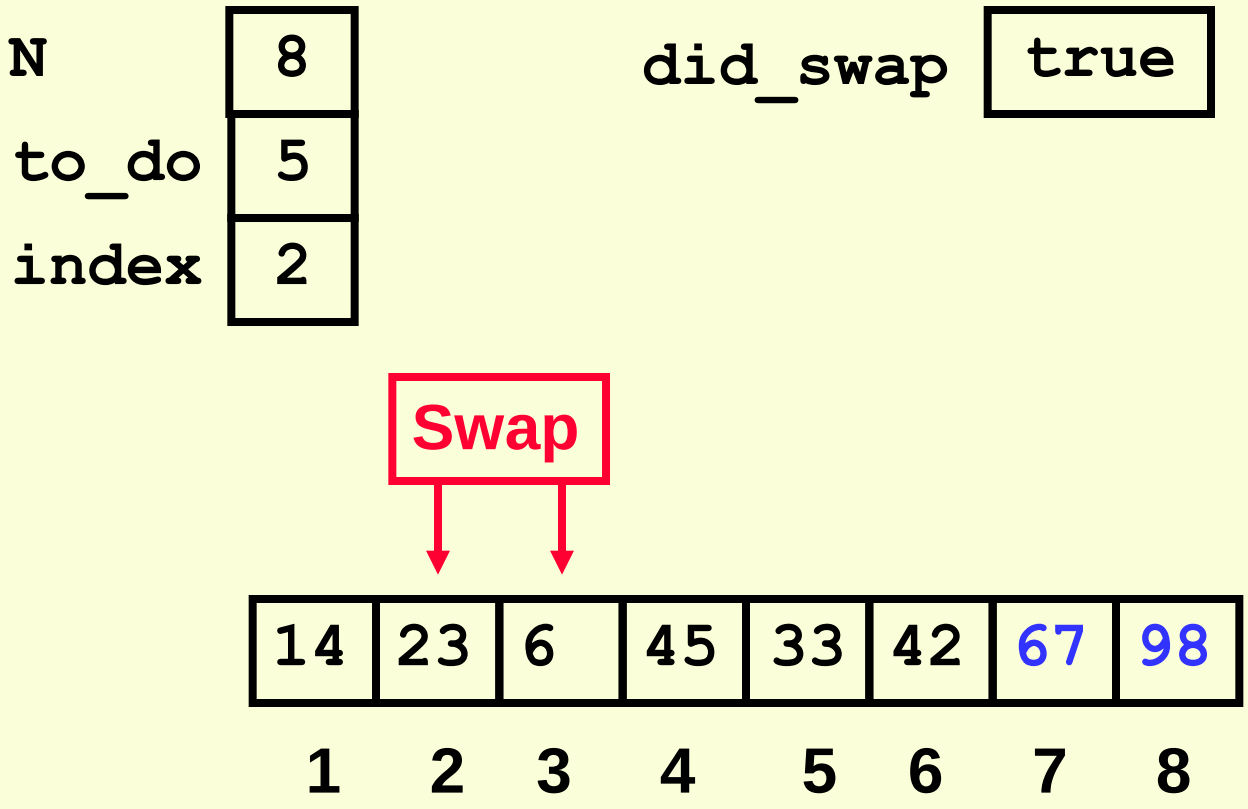
The Third “Bubble Up”



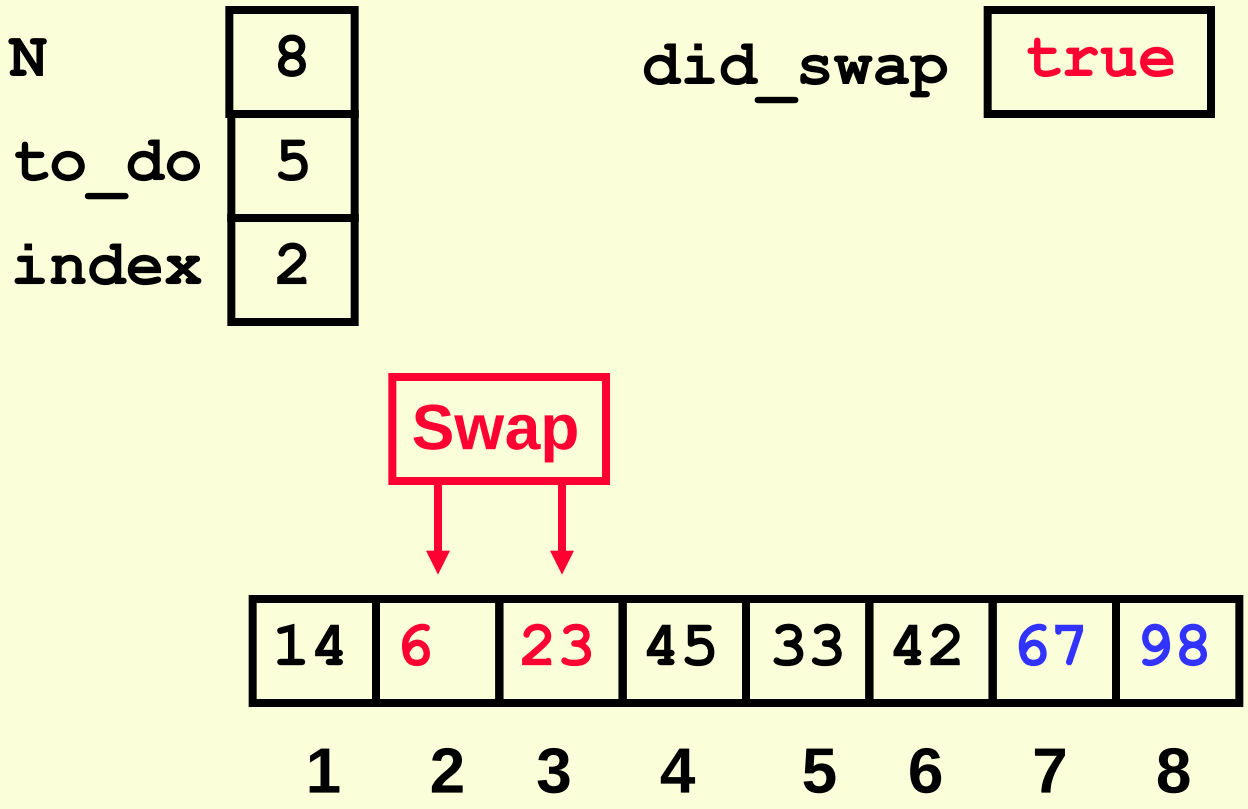
The Third “Bubble Up”



The Third “Bubble Up”



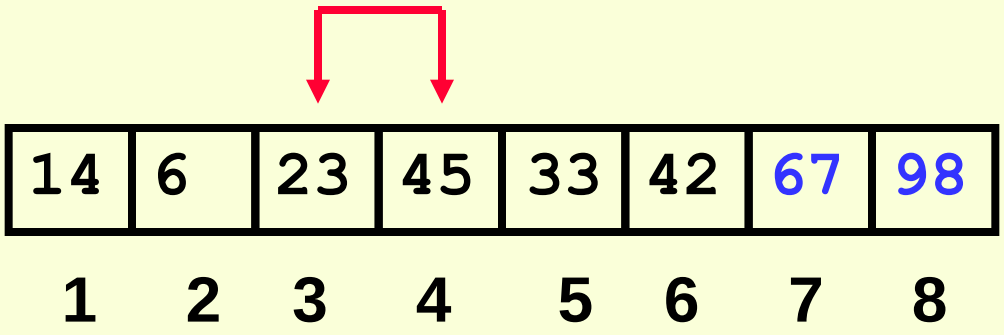
The Third “Bubble Up”



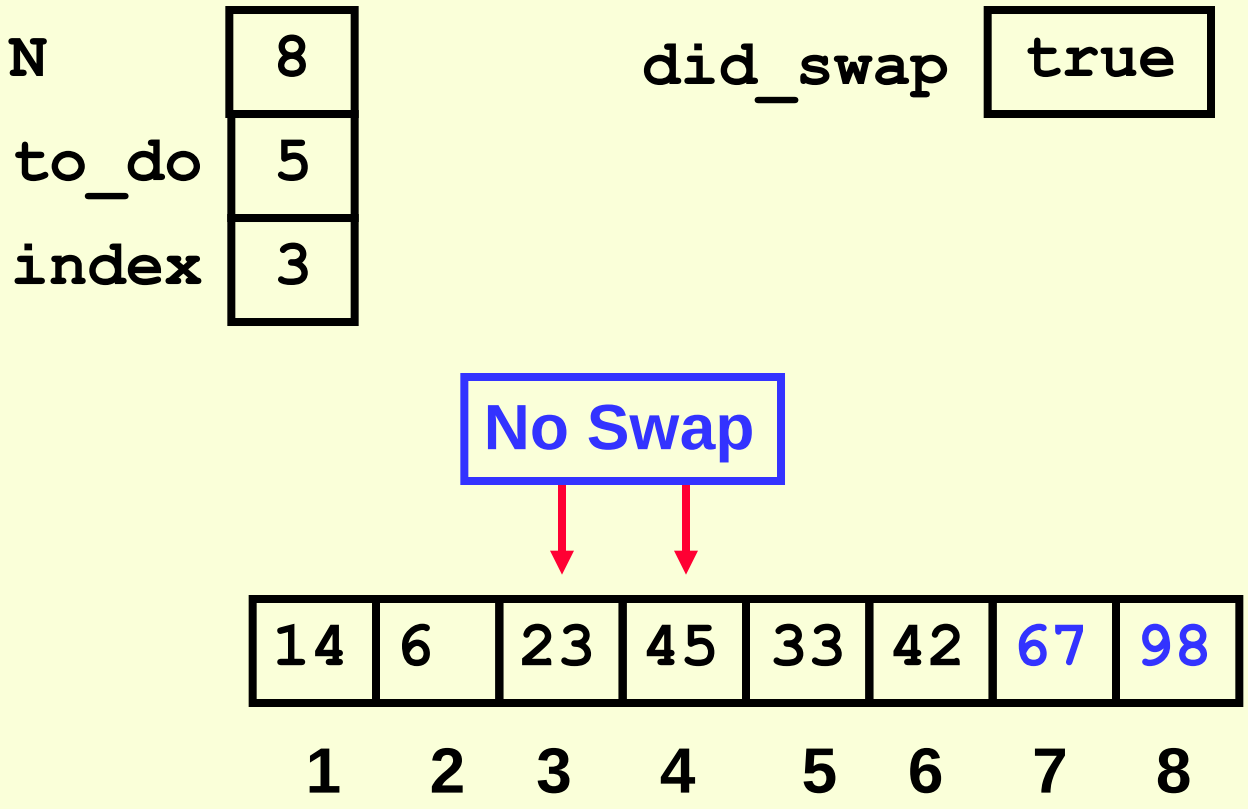
The Third “Bubble Up”

N	8
to_do	5
index	3

did_swap	true



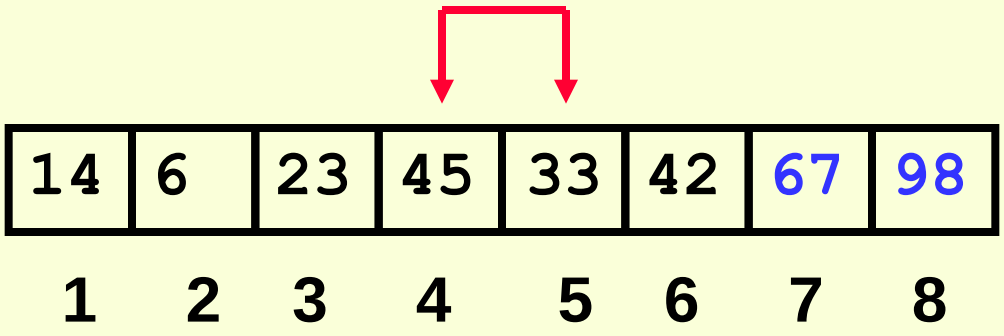
The Third “Bubble Up”



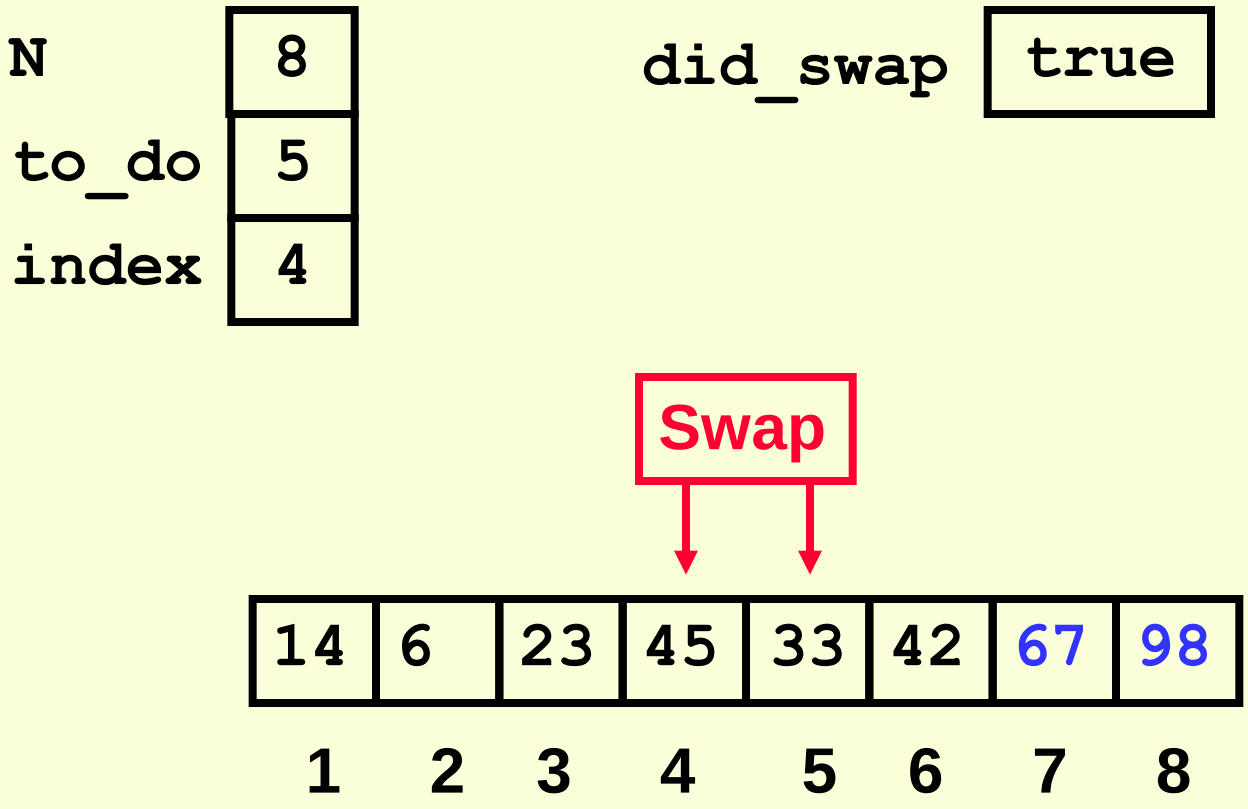
The Third “Bubble Up”

N	8
to_do	5
index	4

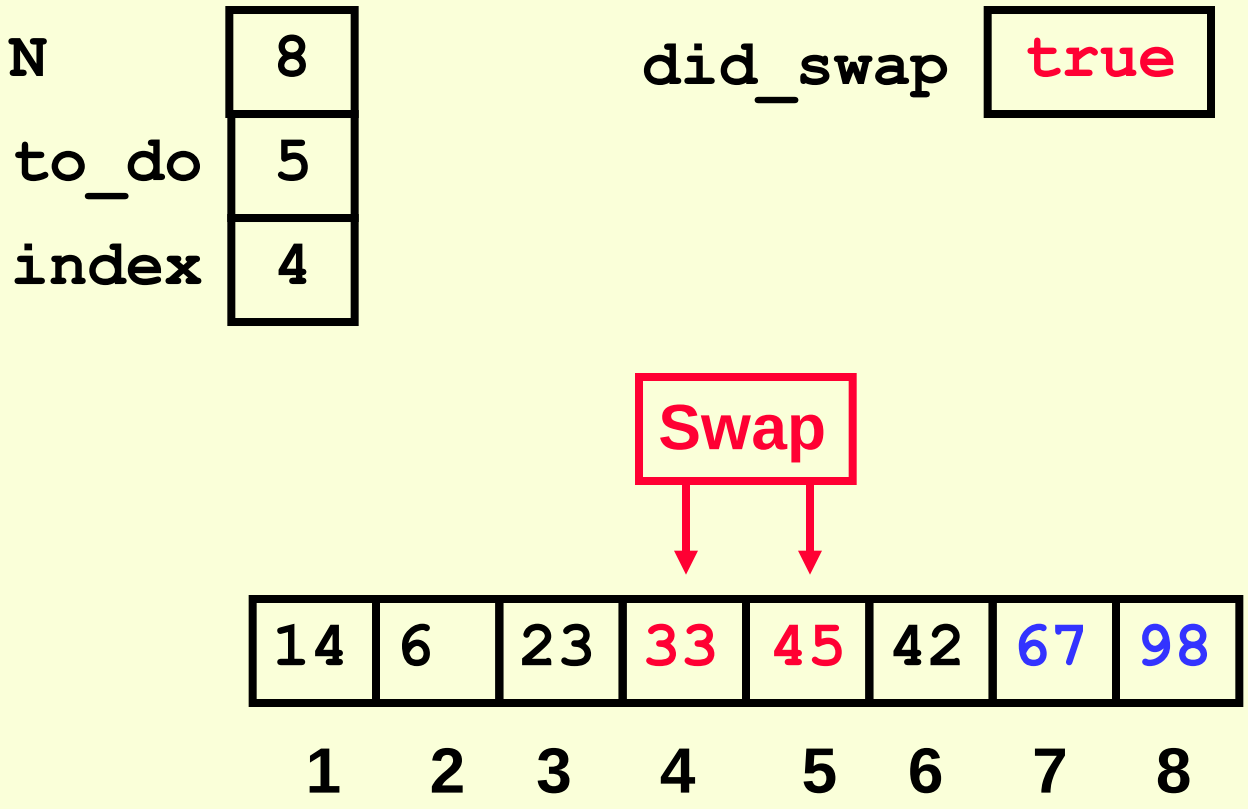
did_swap	true



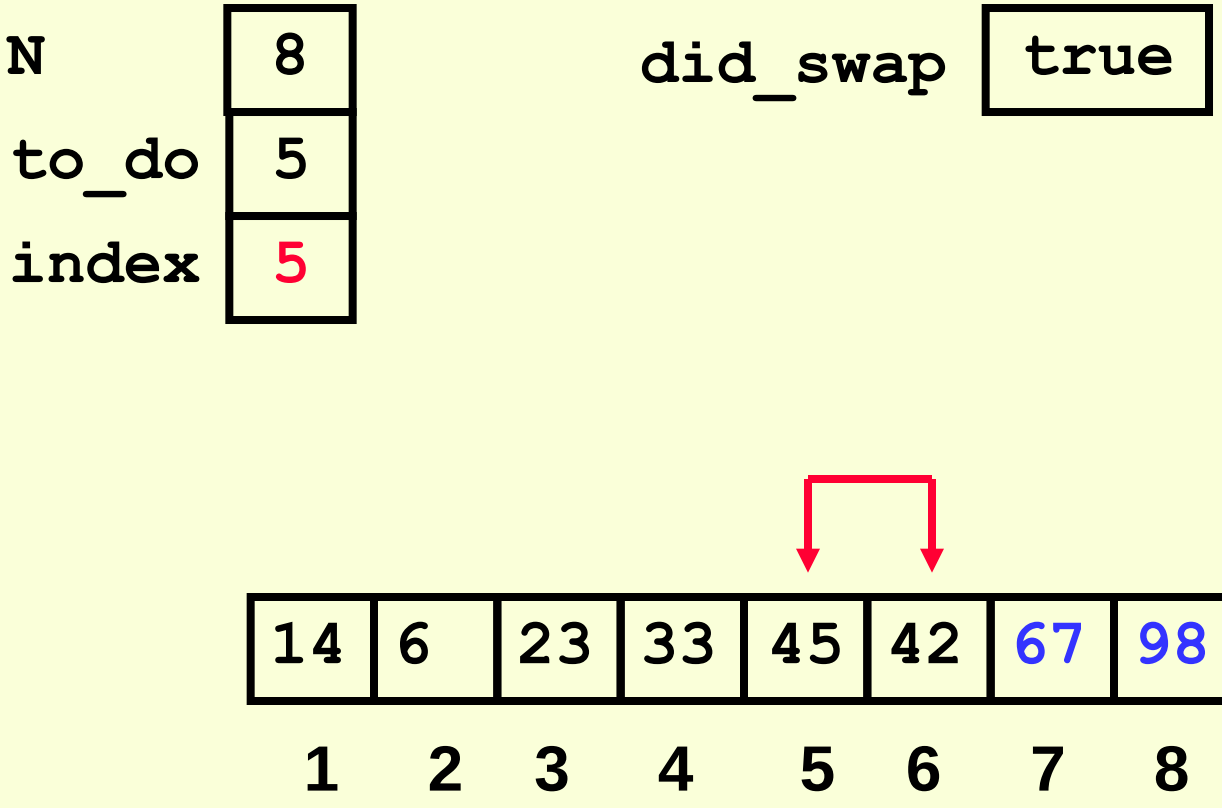
The Third “Bubble Up”



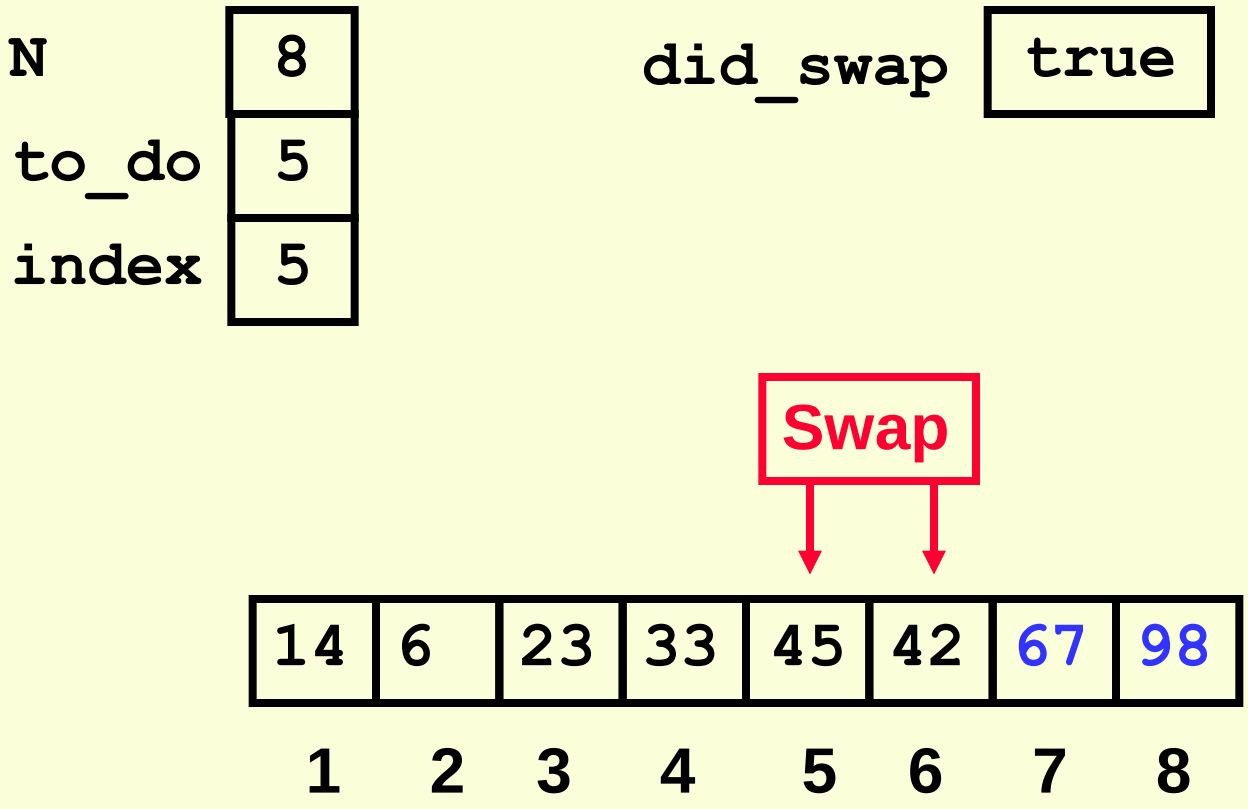
The Third “Bubble Up”



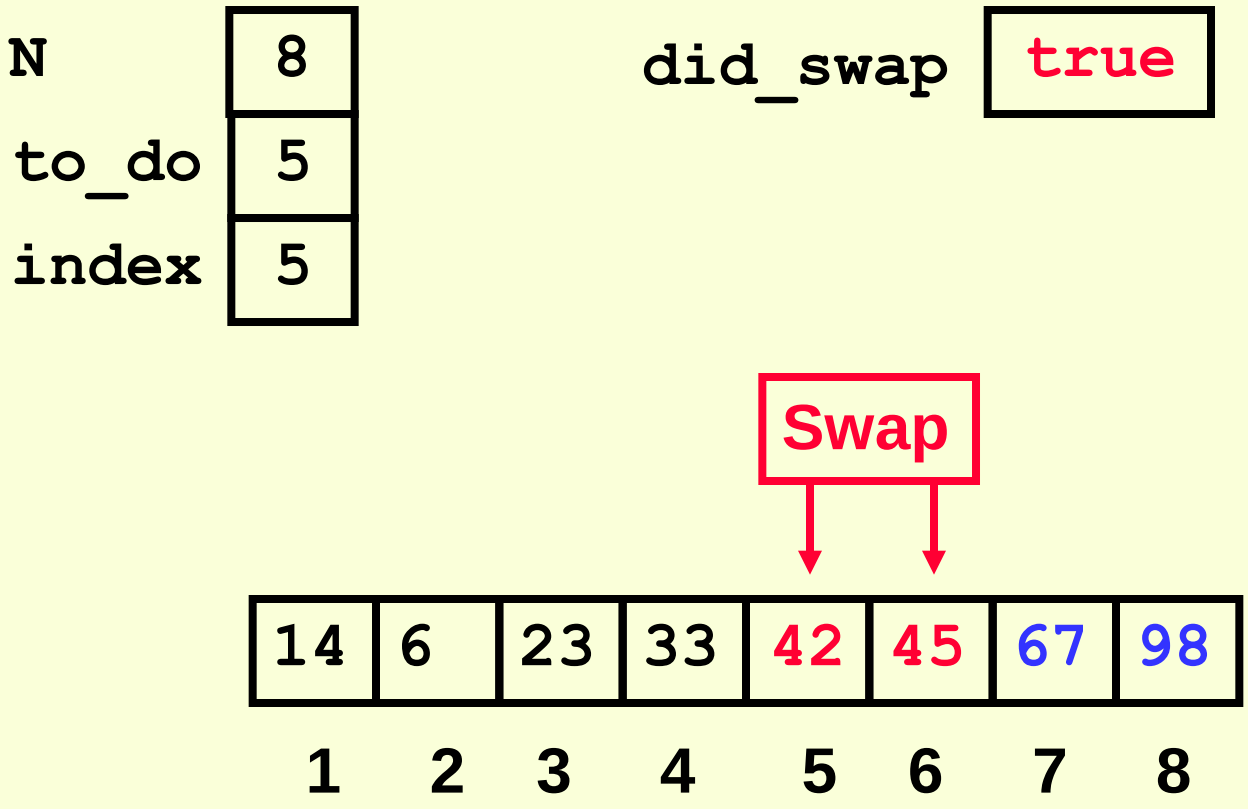
The Third “Bubble Up”



The Third “Bubble Up”



The Third “Bubble Up”

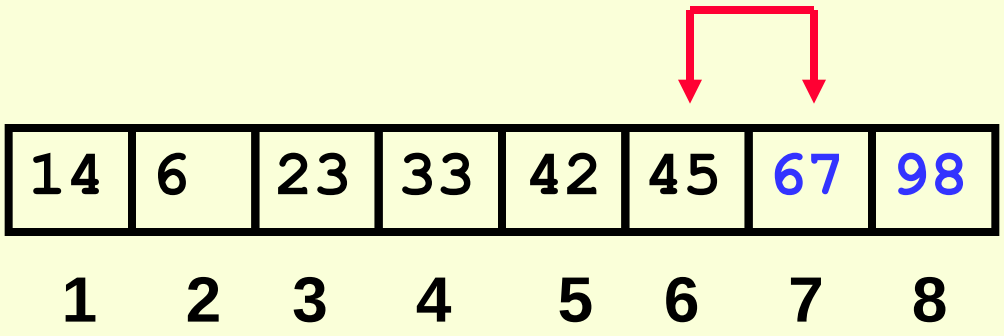


After Third Pass of Outer Loop

N	8
to_do	5
index	6

did_swap	true

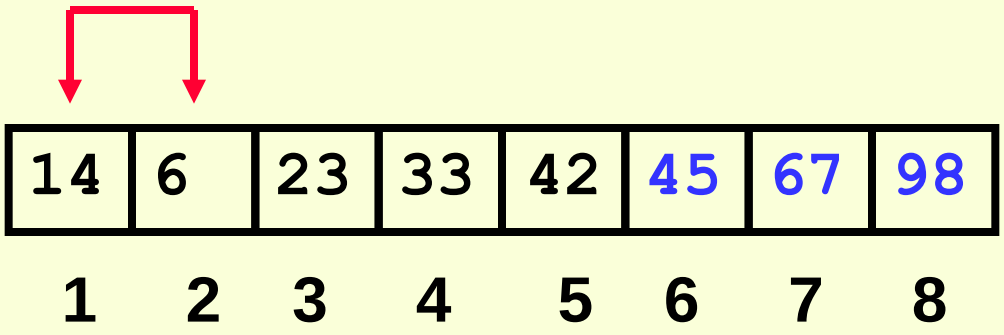
Finished third “Bubble Up”



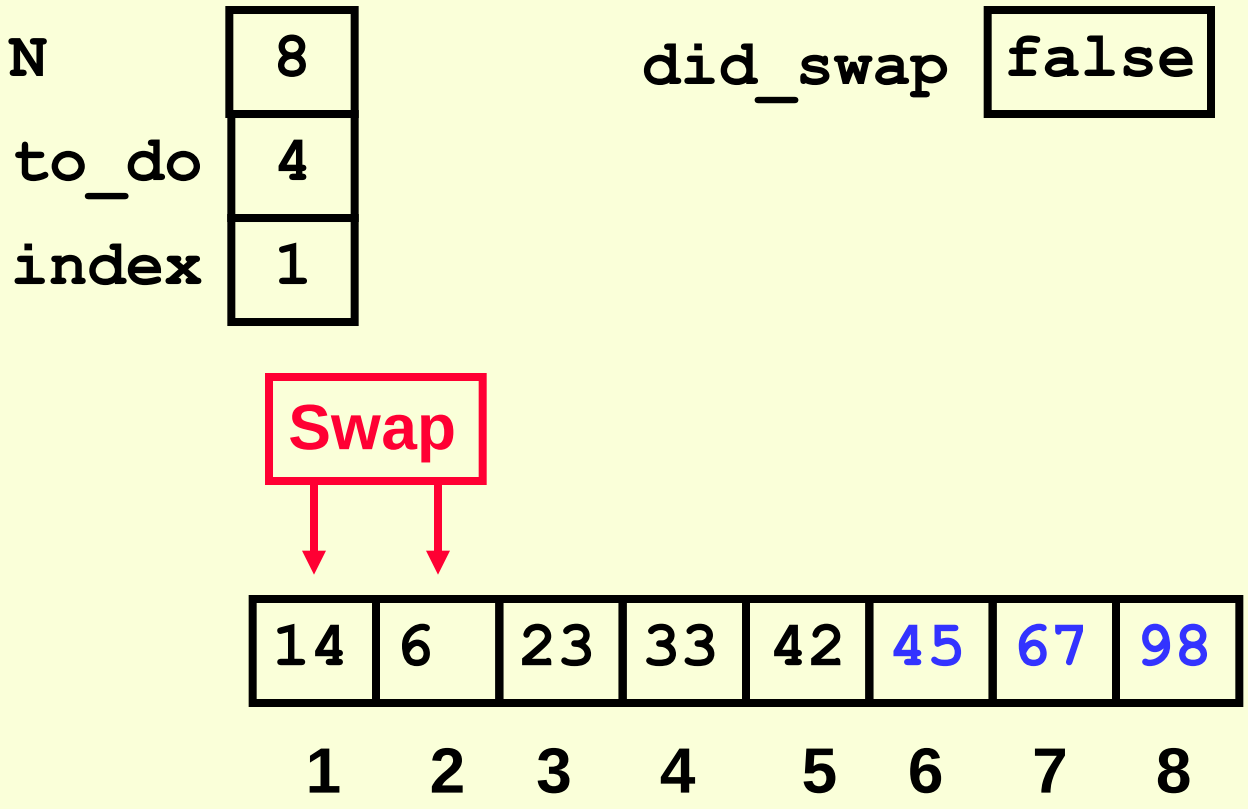
The Fourth “Bubble Up”

N	8
to_do	4
index	1

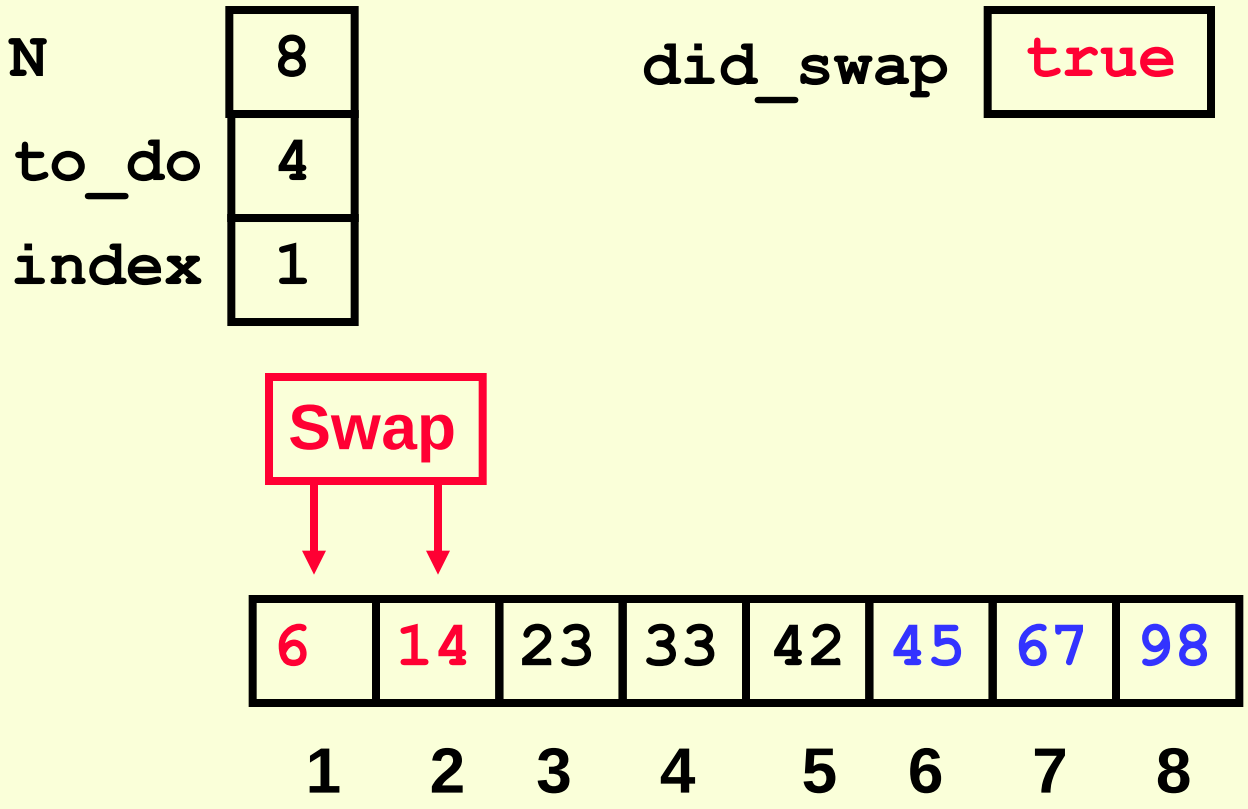
did_swap	false



The Fourth “Bubble Up”



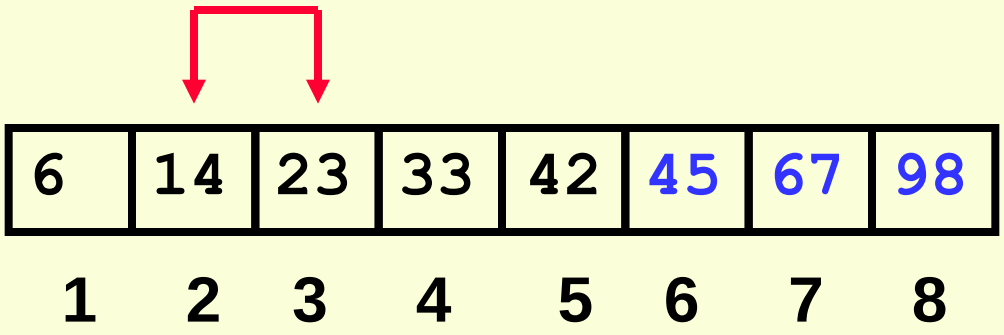
The Fourth “Bubble Up”



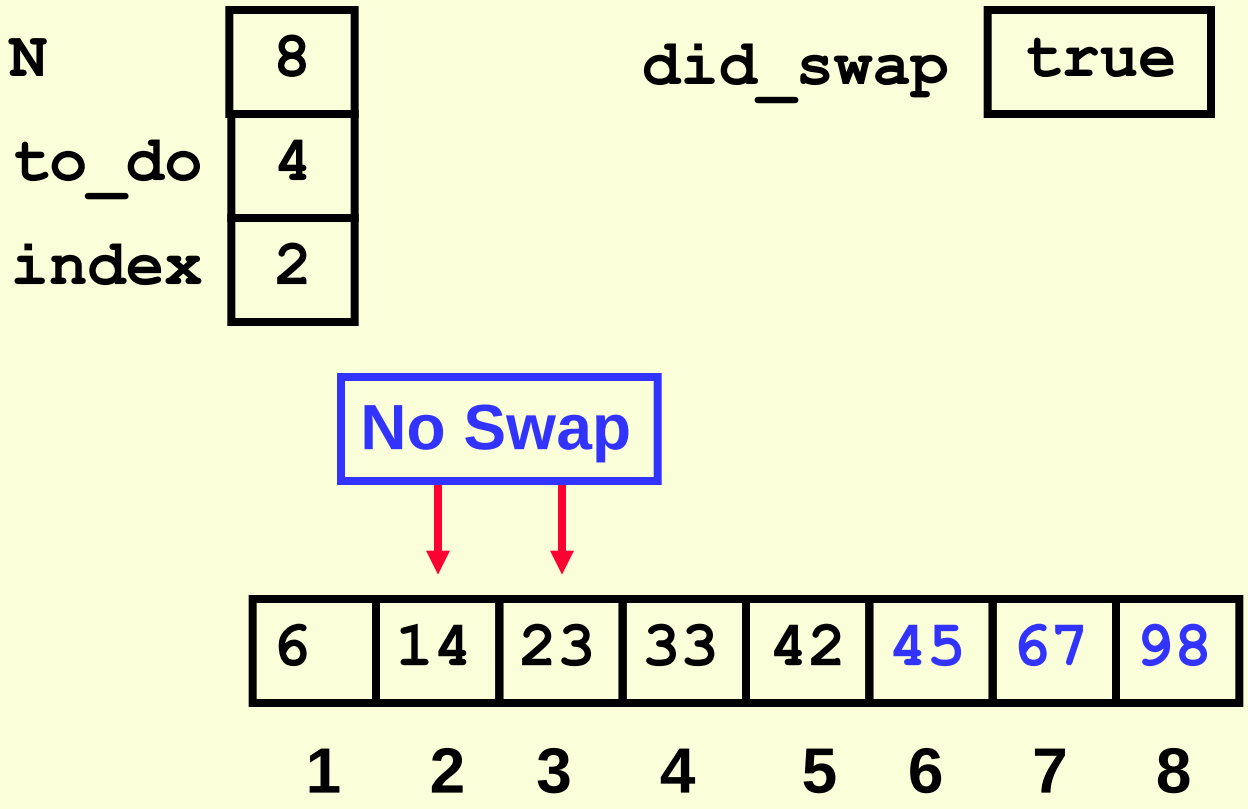
The Fourth “Bubble Up”

N	8
to_do	4
index	2

did_swap	true



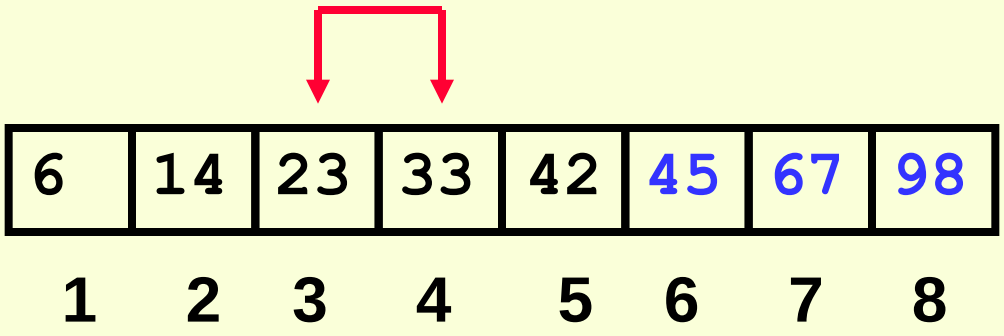
The Fourth “Bubble Up”



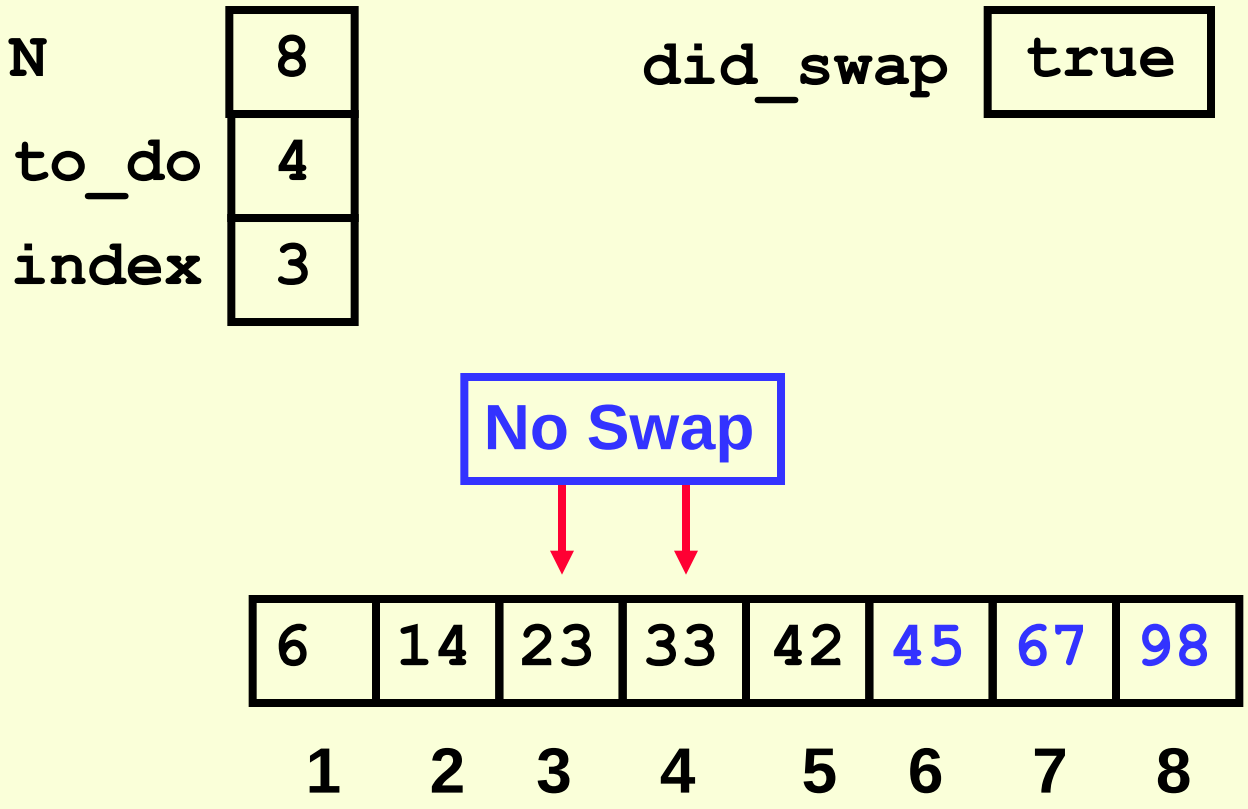
The Fourth “Bubble Up”

N	8
to_do	4
index	3

did_swap	true



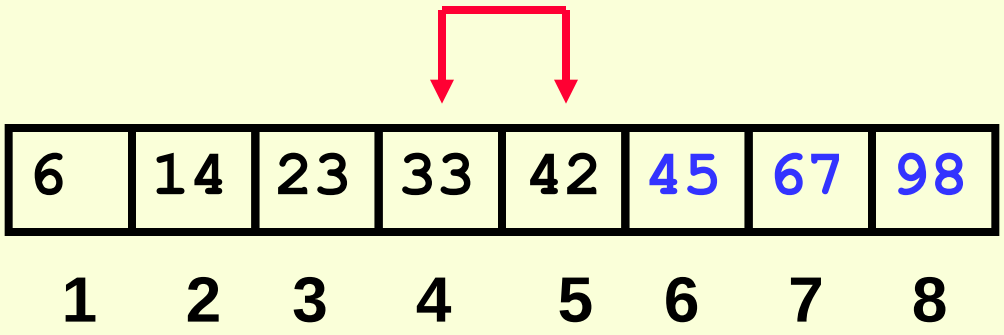
The Fourth “Bubble Up”



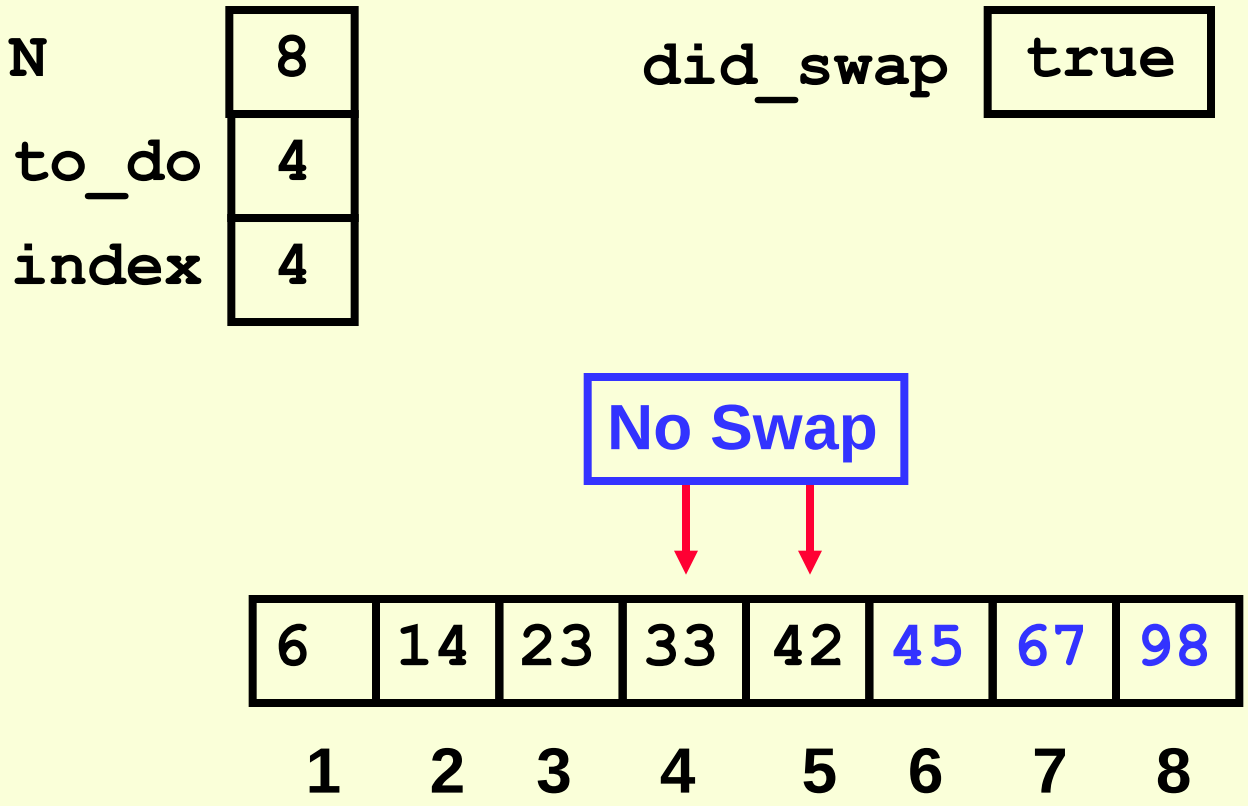
The Fourth “Bubble Up”

N	8
to_do	4
index	4

did_swap	true



The Fourth “Bubble Up”

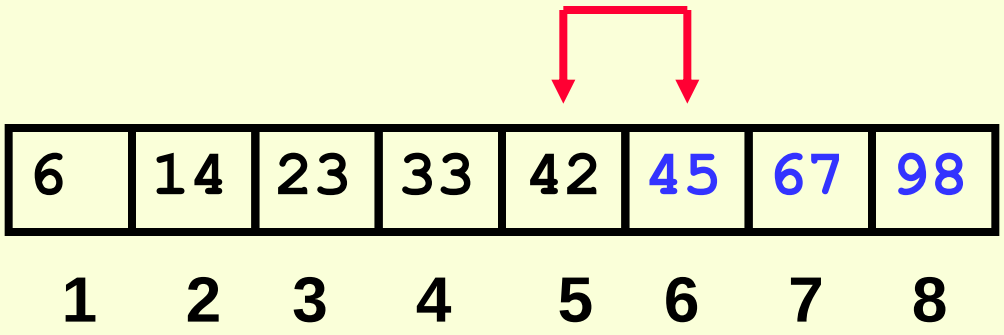


After Fourth Pass of Outer Loop

N	8
to_do	4
index	5

did_swap	true

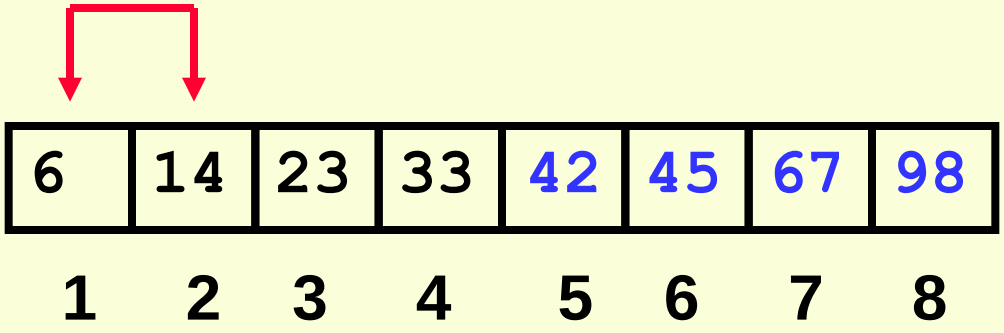
Finished fourth “Bubble Up”



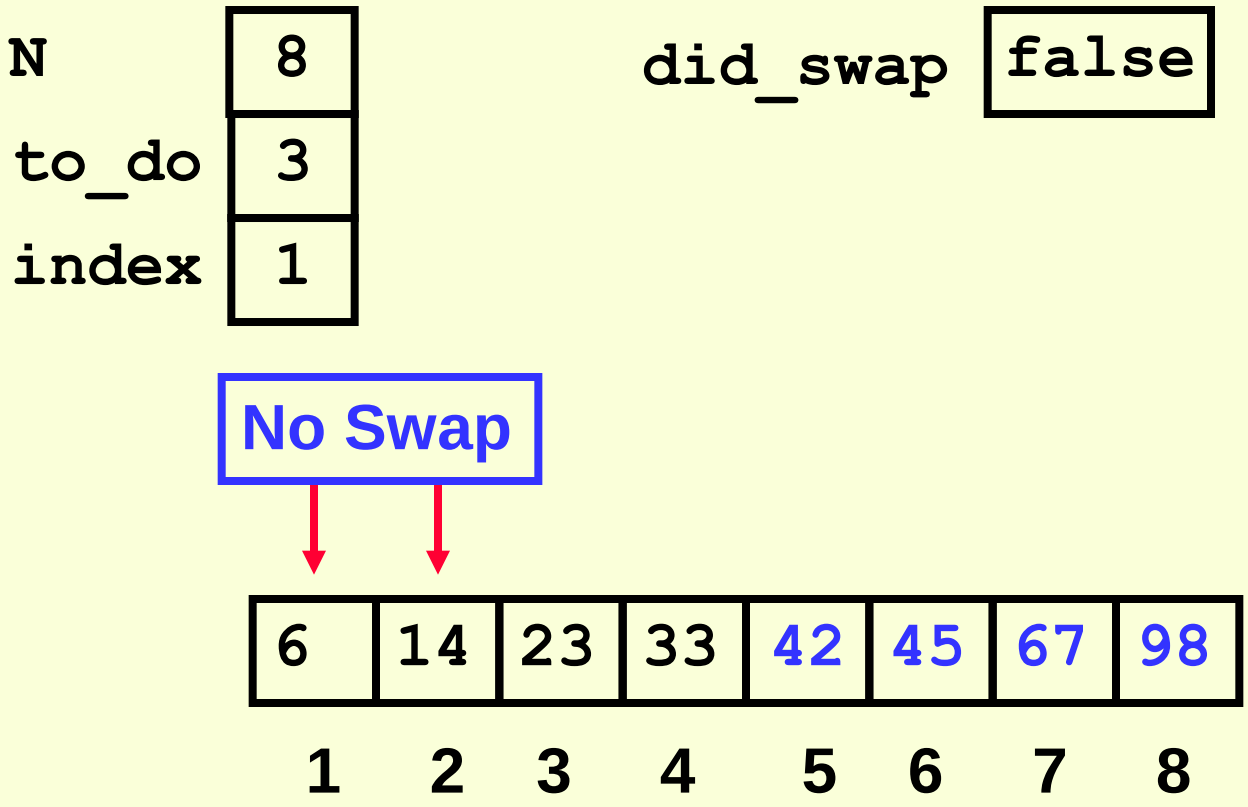
The Fifth “Bubble Up”

N	8
to_do	3
index	1

did_swap	false



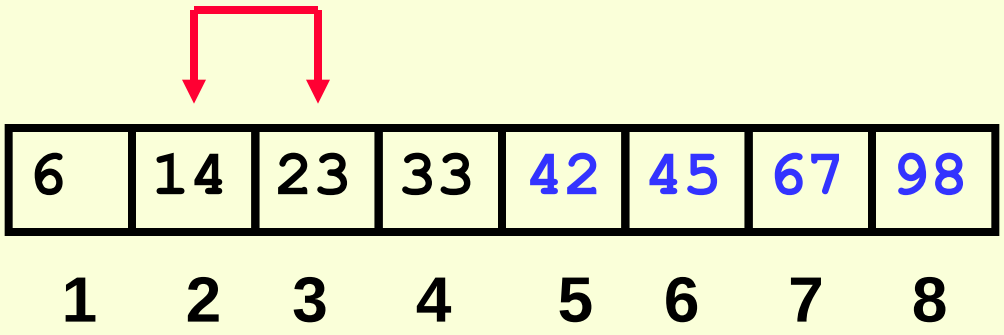
The Fifth “Bubble Up”



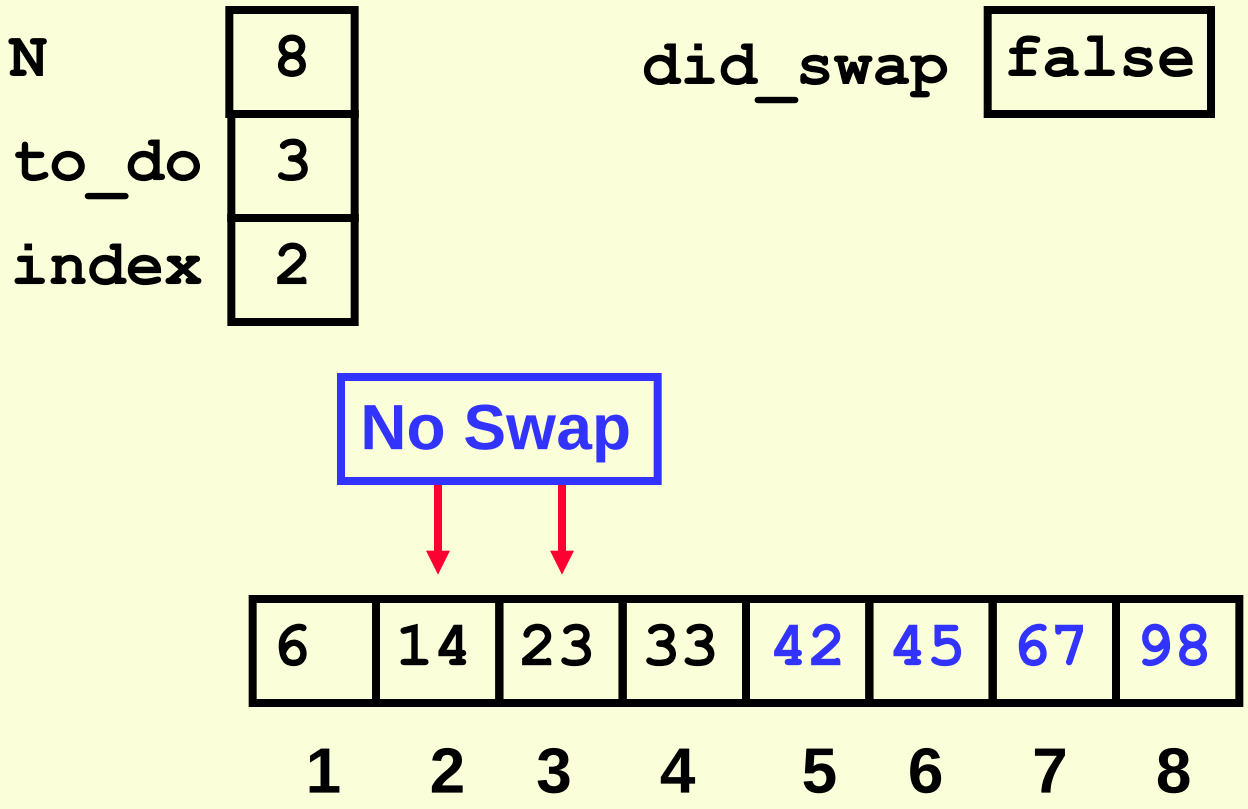
The Fifth “Bubble Up”

N	8
to_do	3
index	2

did_swap	false



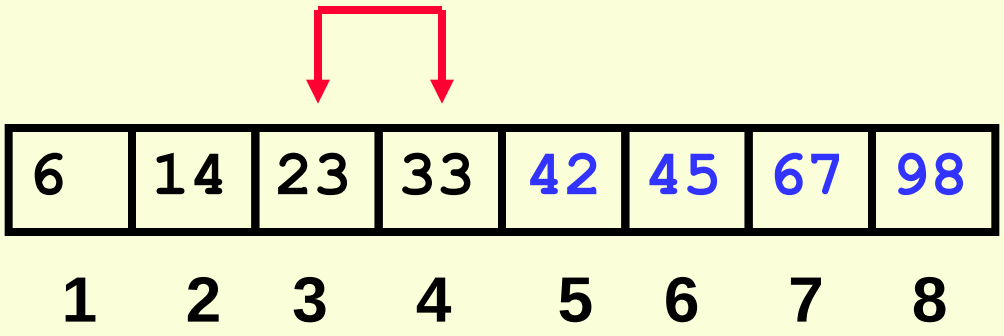
The Fifth “Bubble Up”



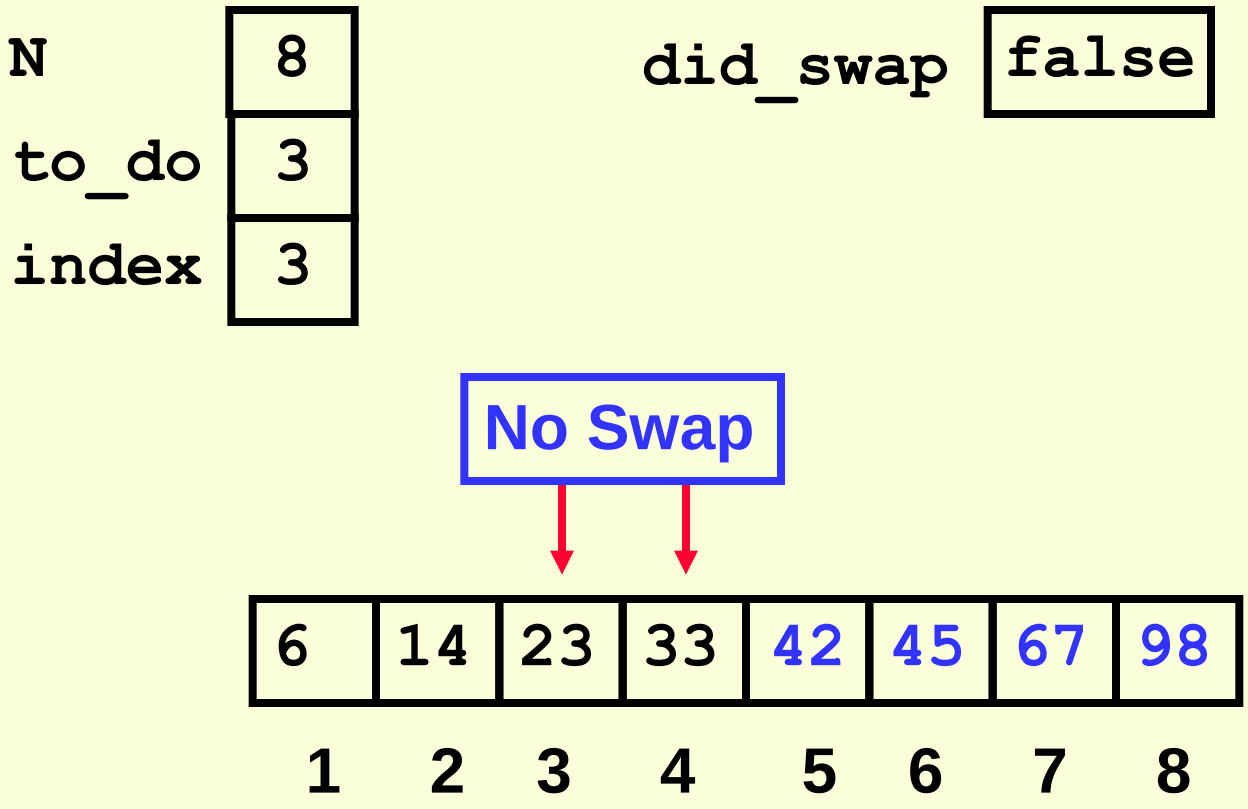
The Fifth “Bubble Up”

N	8
to_do	3
index	3

did_swap	false



The Fifth “Bubble Up”

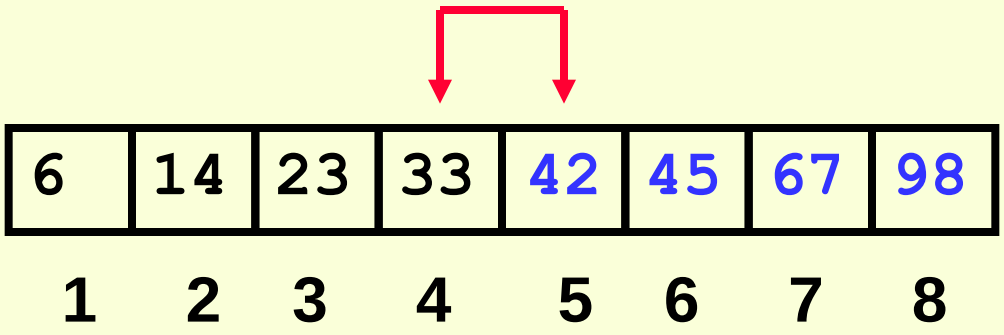


After Fifth Pass of Outer Loop

N	8
to_do	3
index	4

did_swap	false

Finished fifth “Bubble Up”



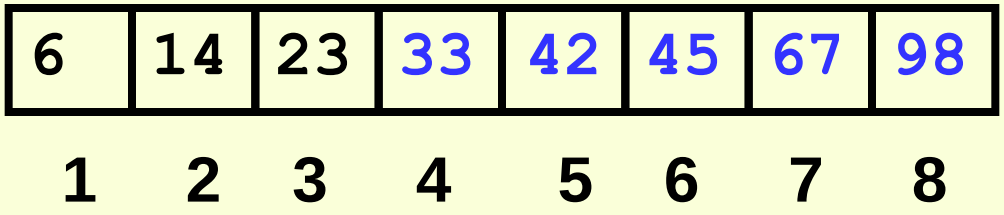
Finished “Early”

N	8
to_do	3
index	4

did_swap	false

We didn’t do any swapping, so all of the other elements must be correctly placed.

We can “skip” the last two passes of the outer loop.



Summary

- **"Bubble Up" algorithm will move largest value to its correct location (to the right)**
- **Repeat "Bubble Up" until all elements are correctly placed:**
  - **Maximum of N-1 times**
  - **Can finish early if no swapping occurs**
- **We reduce the number of elements we compare each time one is correctly placed**

LB

Truth in CS Act

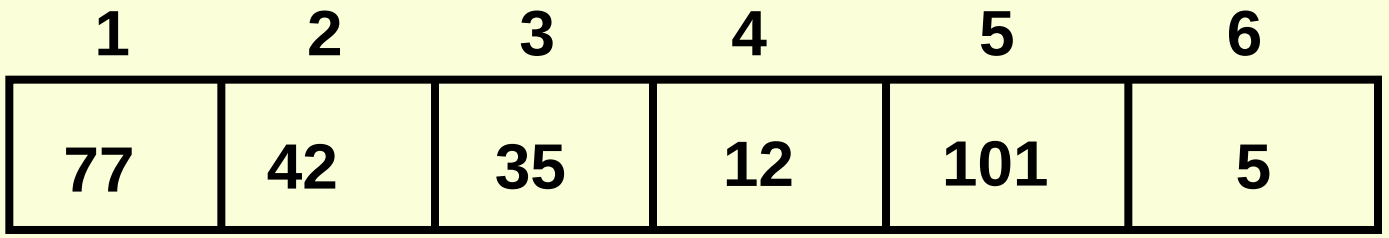
- NOBODY EVER USES BUBBLE SORT
- NOBODY
- NOT EVER
- BECAUSE IT IS EXTREMELY INEFFICIENT

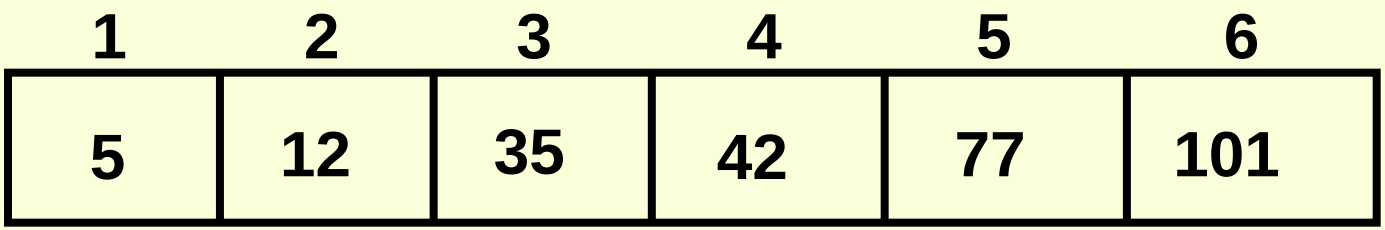
Questions?

Mergesort

Sorting

- **Sorting takes an unordered collection and makes it an ordered one.**





Divide and Conquer

- **Divide and Conquer cuts the problem in half each time, but uses the result of both halves:**
  - **cut the problem in half until the problem is trivial**
  - **solve for both halves**
  - **combine the solutions**

Mergesort

- A divide-and-conquer algorithm:
- Divide the unsorted array into 2 halves until the sub-arrays only contain one element
- Merge the sub-problem solutions together:
  - Compare the sub-array's first elements
  - Remove the smallest element and put it into the result array
  - Continue the process until all elements have been put into the result array

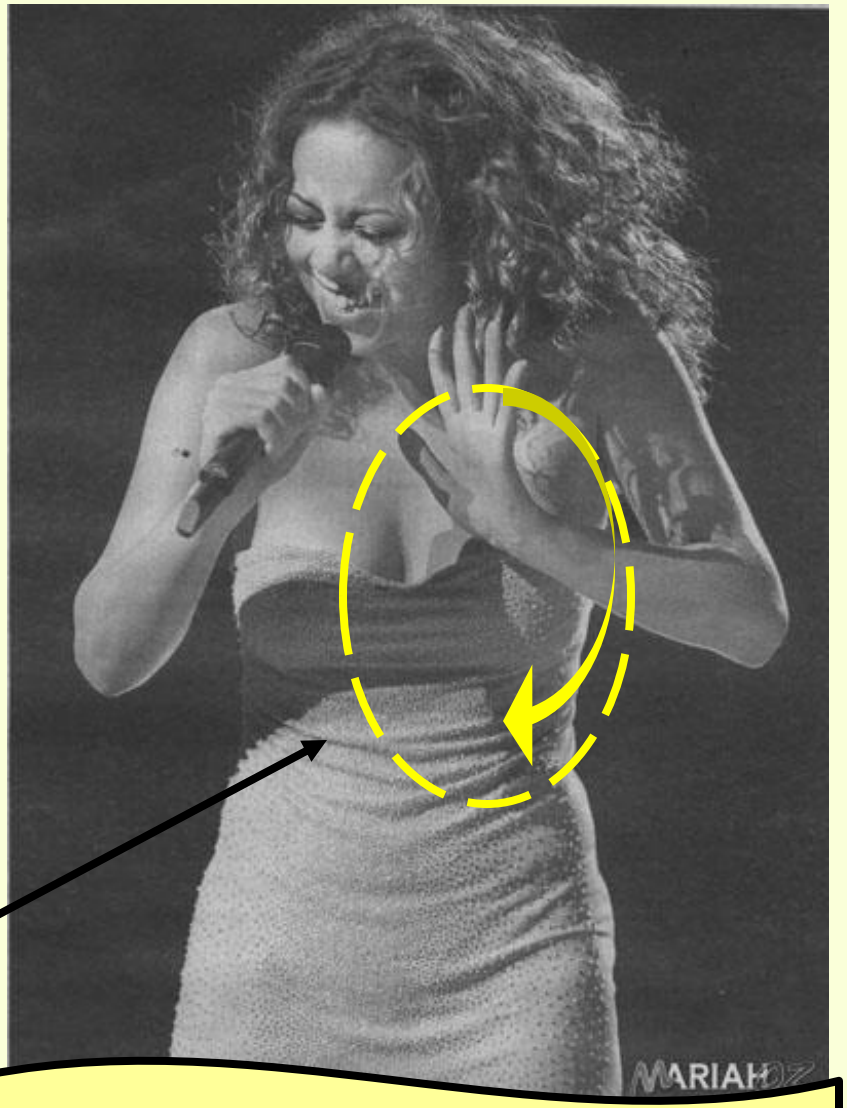
37	23	6	89	15	12	2	19

How to Remember Merge Sort?

That's easy. Just remember Mariah Carey.

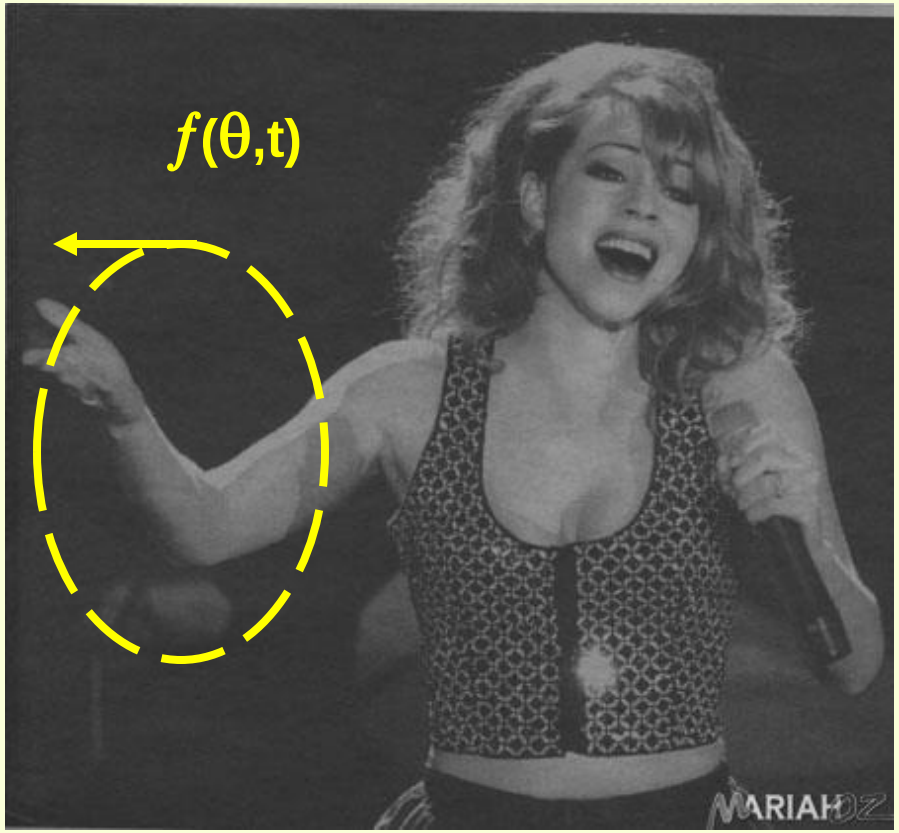
As a singing star, Ms. Carey has perfected the "wax-on" wave motion--a clockwise sweep of her hand used to emphasize lyrics.

The Maria "Wax-on" Angle:
$f(\theta,t)$

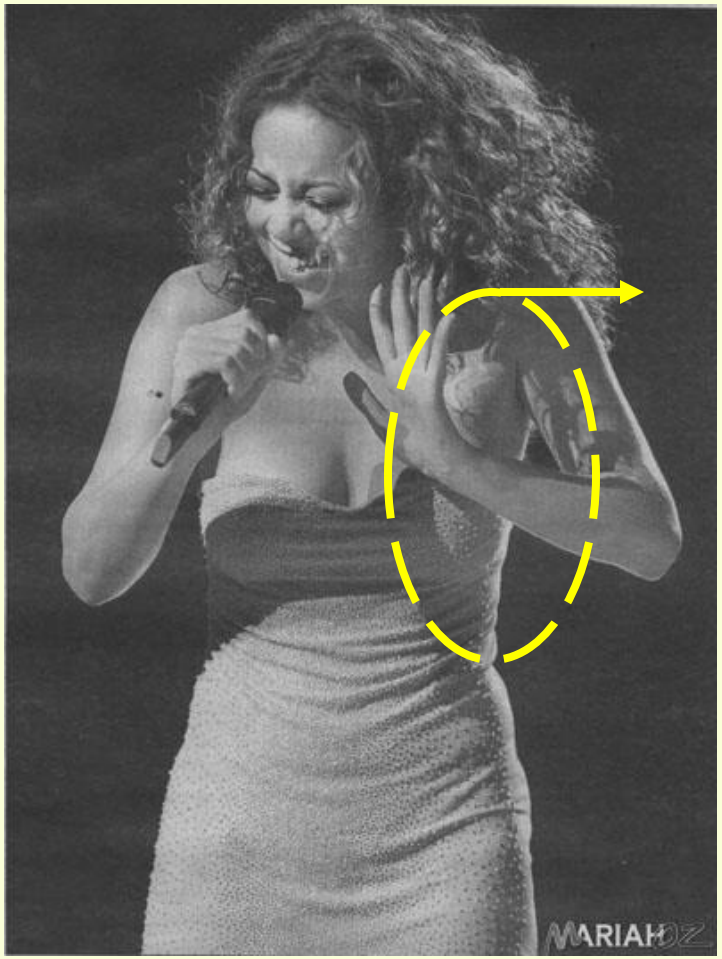


The Siren of Subquadratic Sorts

How To Remember Merge Sort?



Just as Mariah recursively moves her hands into smaller circles, so too does merge sort recursively split an array into smaller segments.



We need two such recursions, one for each half of the split array.

Algorithm

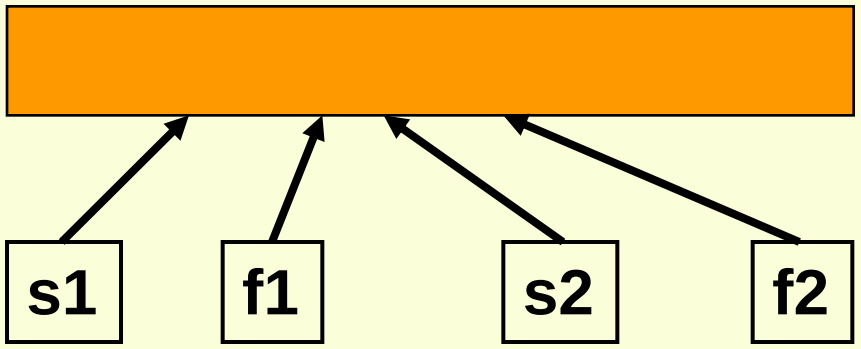
```
Mergesort(Passed an array)
  if array size > 1
    Divide array in half
    Call Mergesort on first half.
    Call Mergesort on second half.
    Merge two halves.

Merge(Passed two arrays)
  Compare leading element in each array
  Select lower and place in new array.
    (If one input array is empty then place
     remainder of other array in output array)
```

LB

More TRUTH in CS

- We don’t really pass in two arrays!
- We pass in one array with indicator variables which tell us where one set of data starts and finishes and where the other set of data starts and finishes.
- Honest.

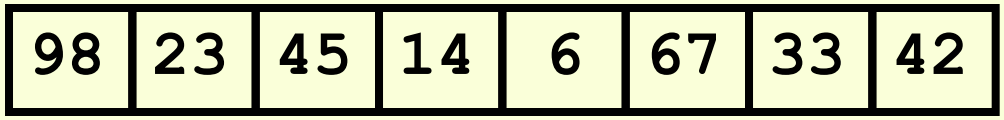


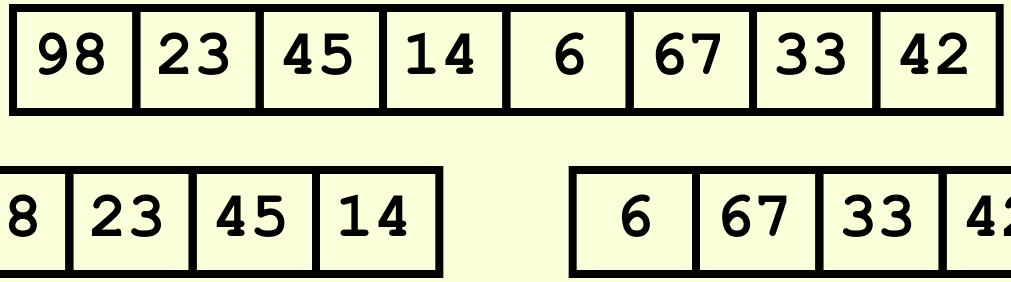
LB

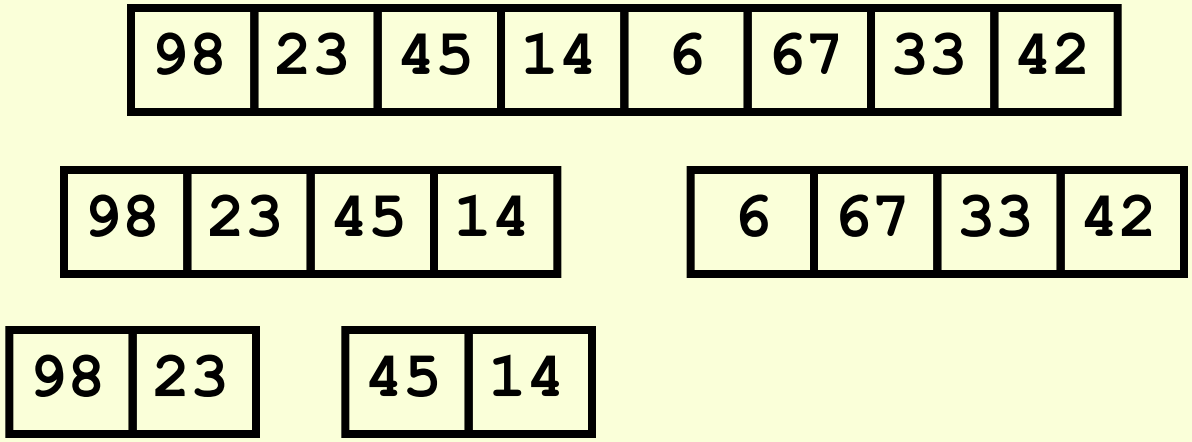
Algorithm

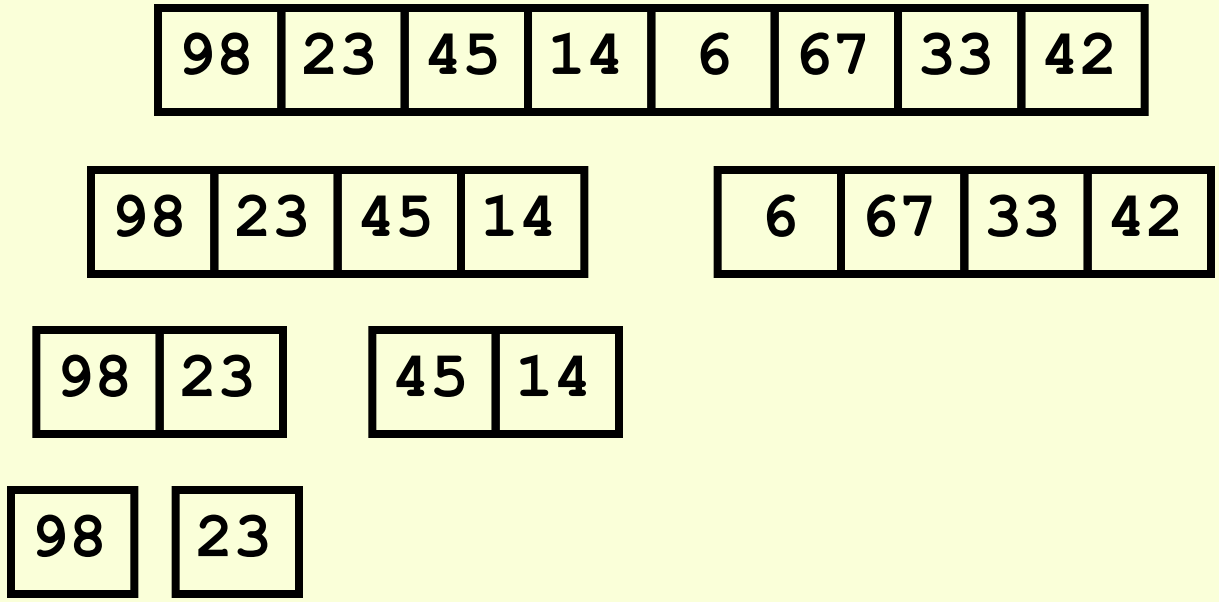
```
Mergesort(Passed an array)
   if array size > 1
      Divide array in half
      Call Mergesort on first half.
      Call Mergesort on second half.
      Merge two halves.

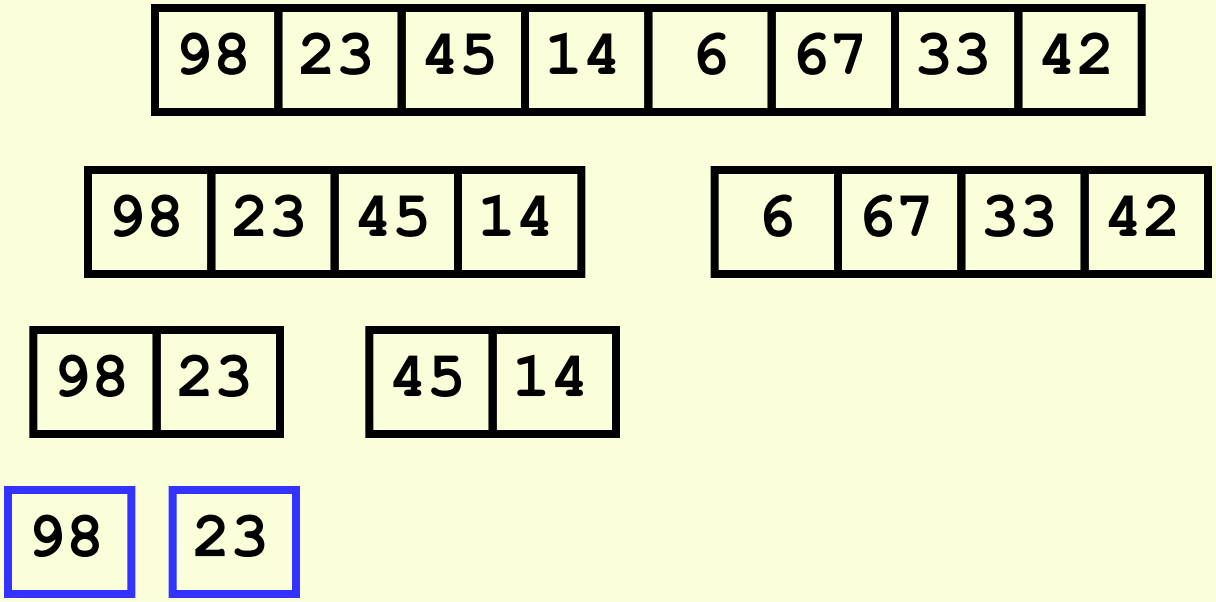
Merge(Passed two arrays)
   Compare leading element in each array
   Select lower and place in new array.
      (If one input array is empty then place
       remainder of other array in output array)
```

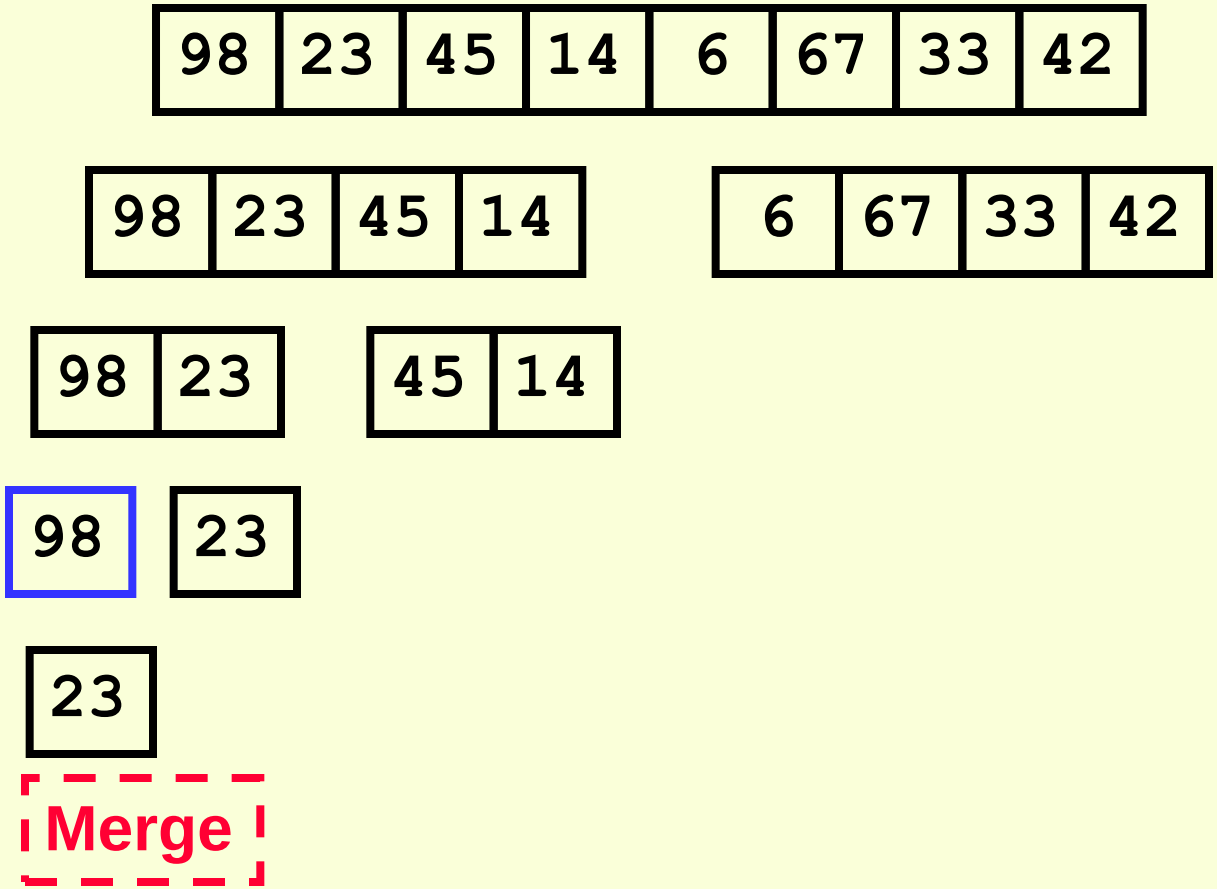

98
23
45
14
6
67
33
42

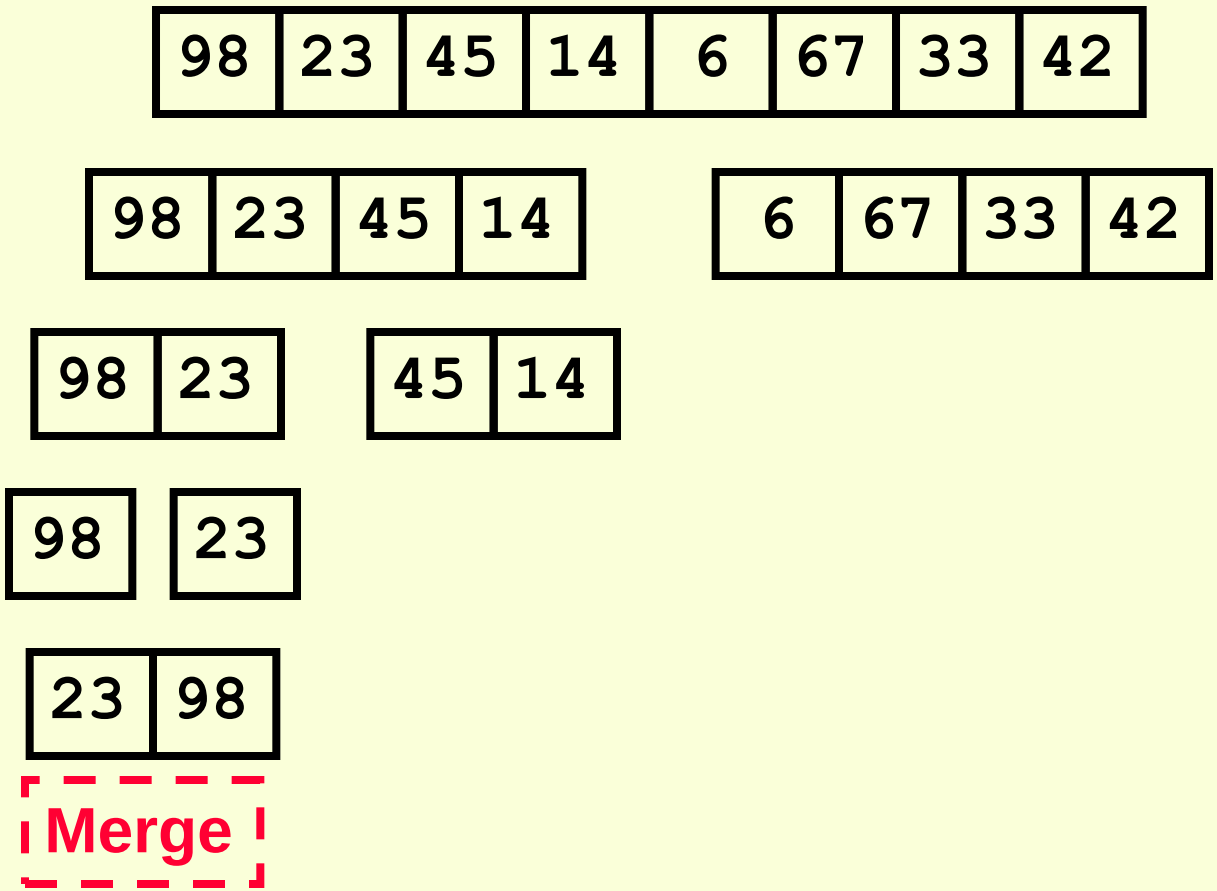

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42

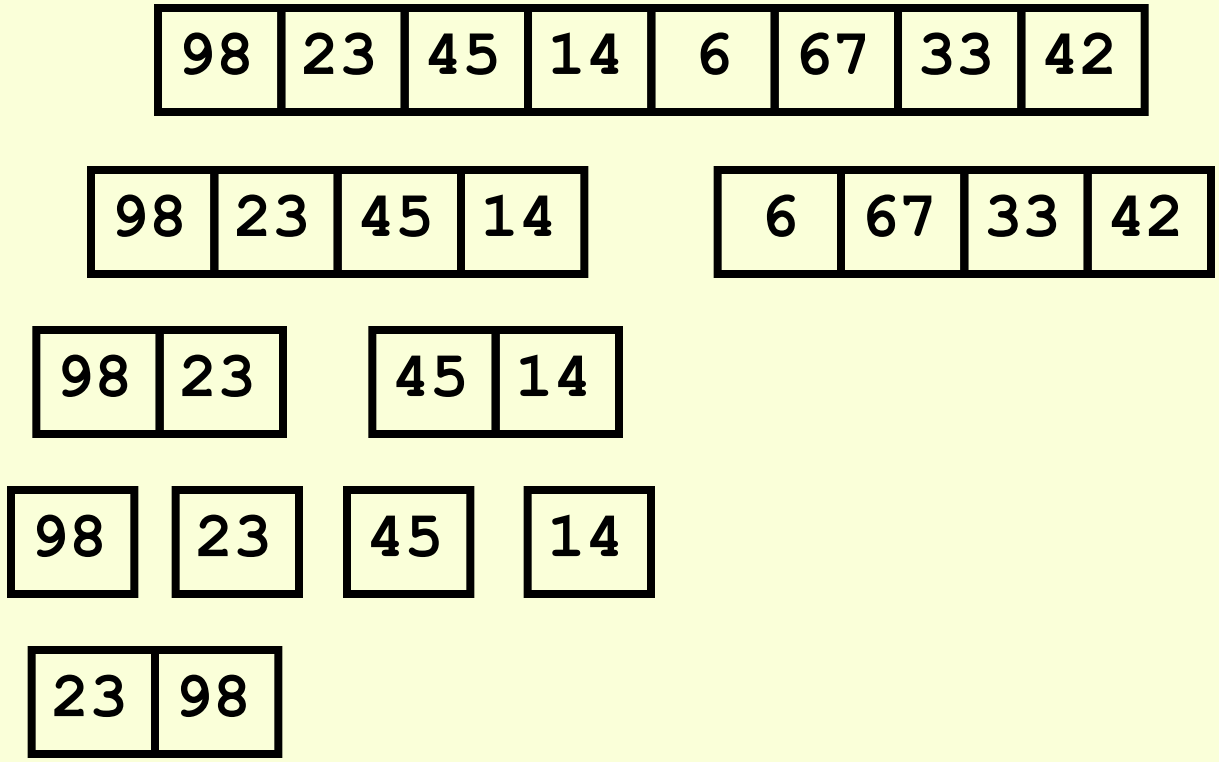

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14

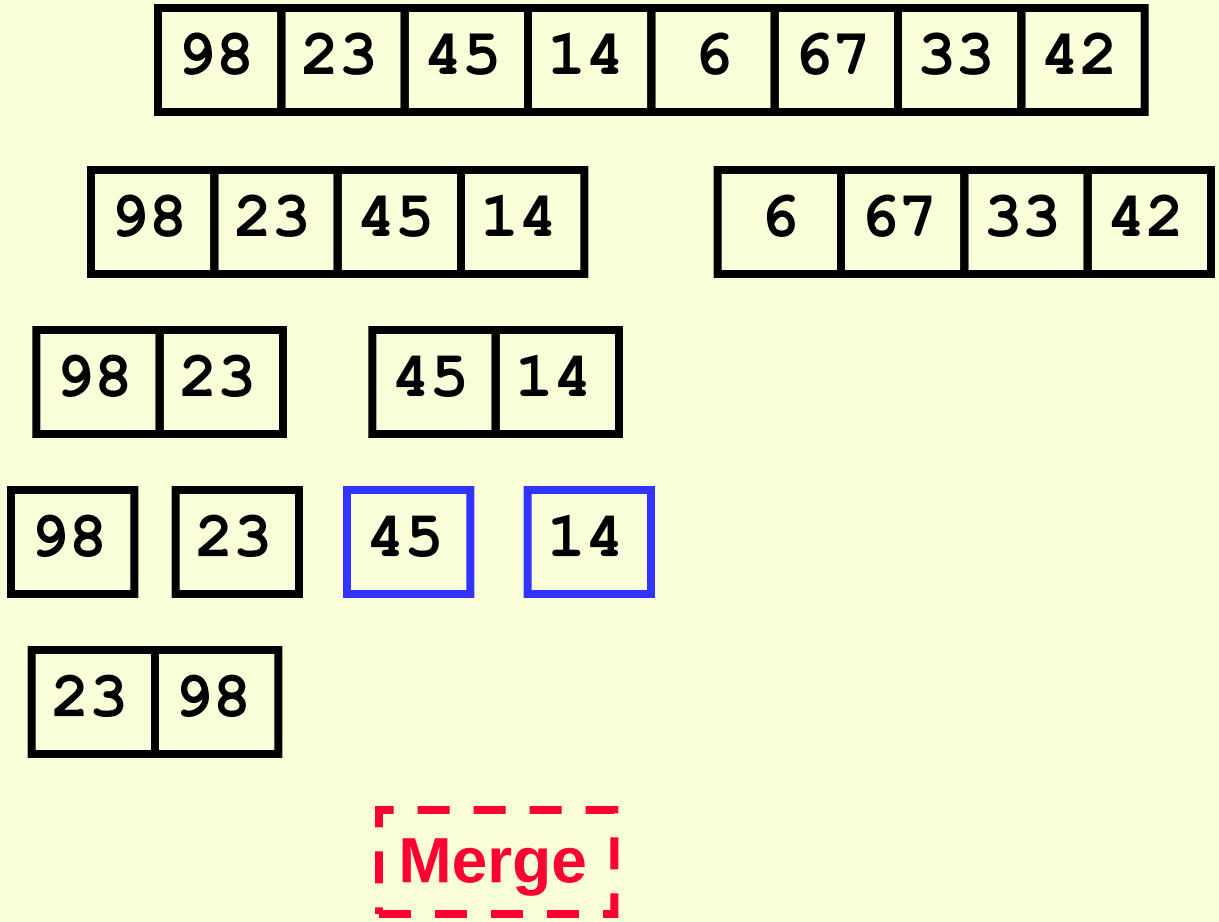

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23

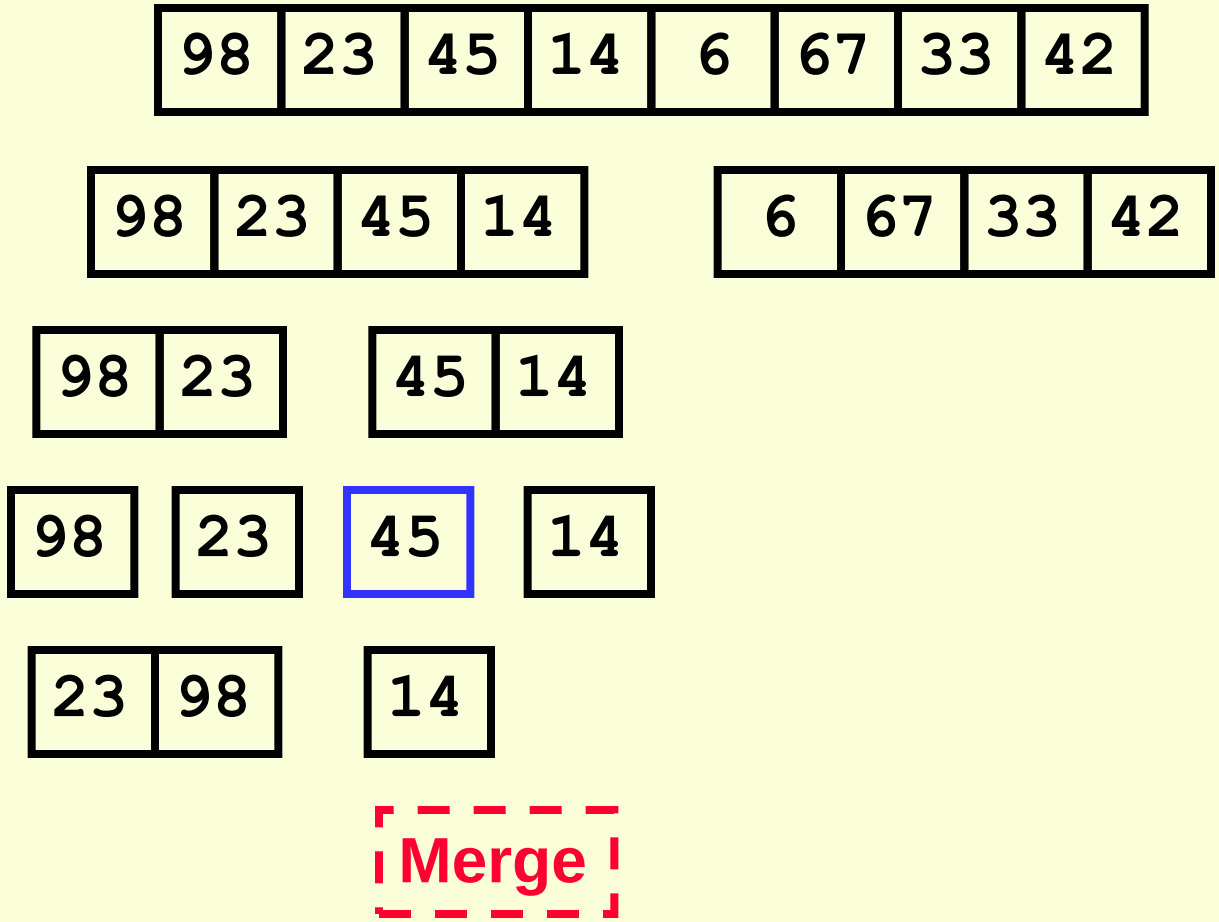

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
Merge

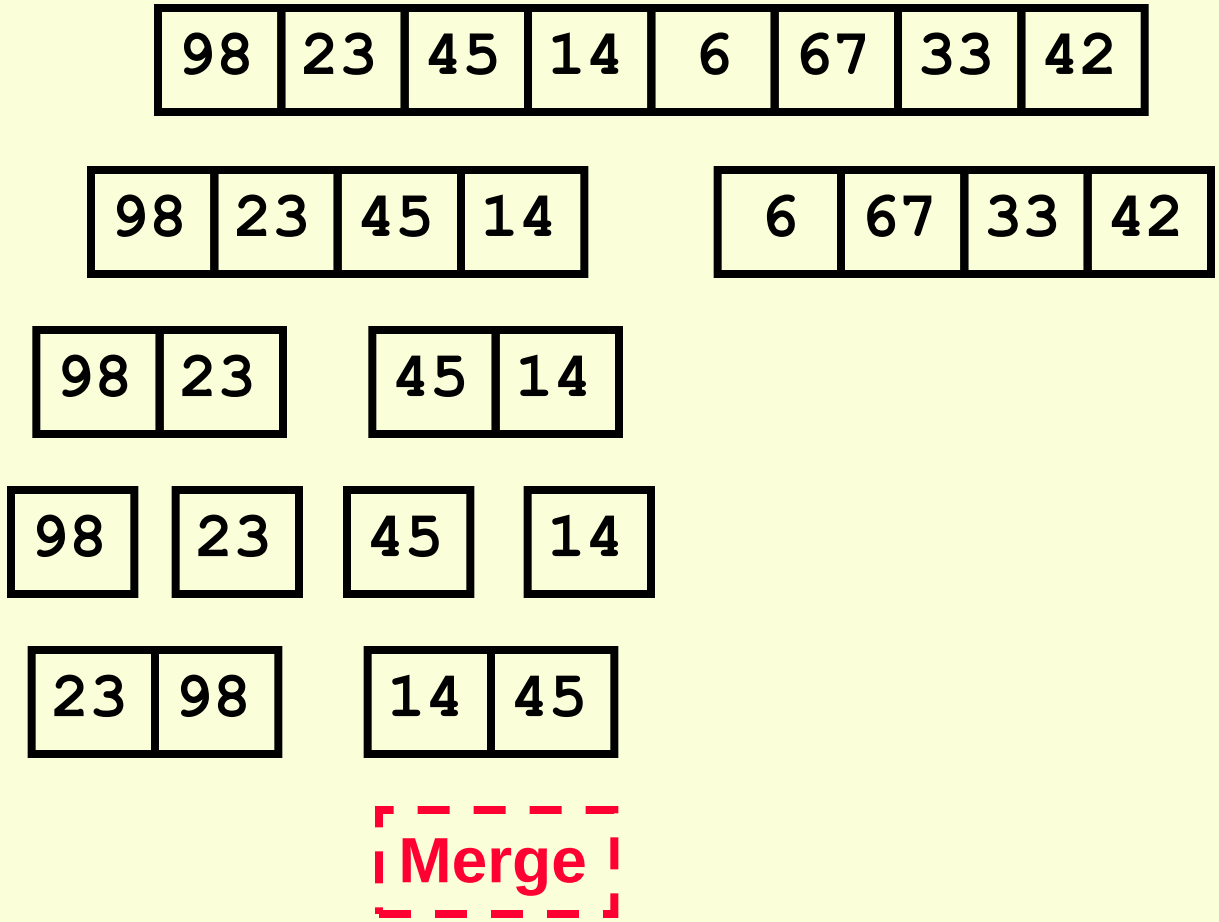

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
23
Merge

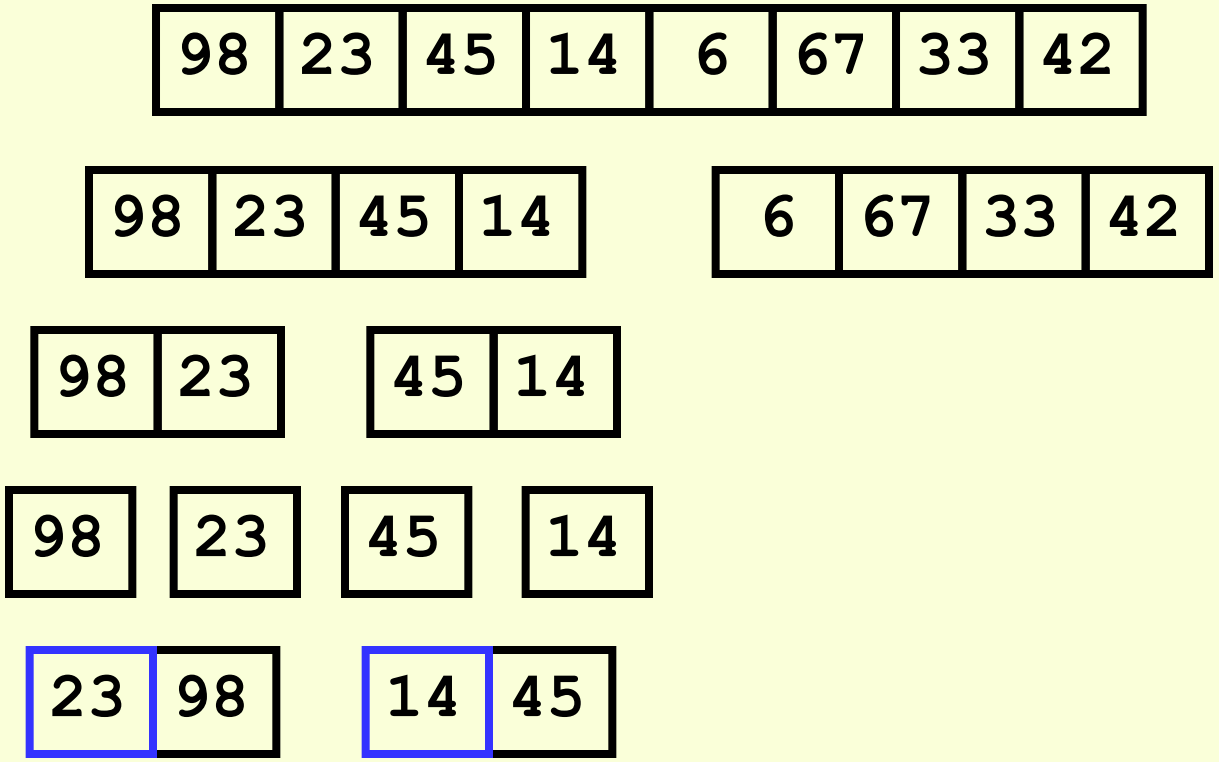

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
23 98
Merge

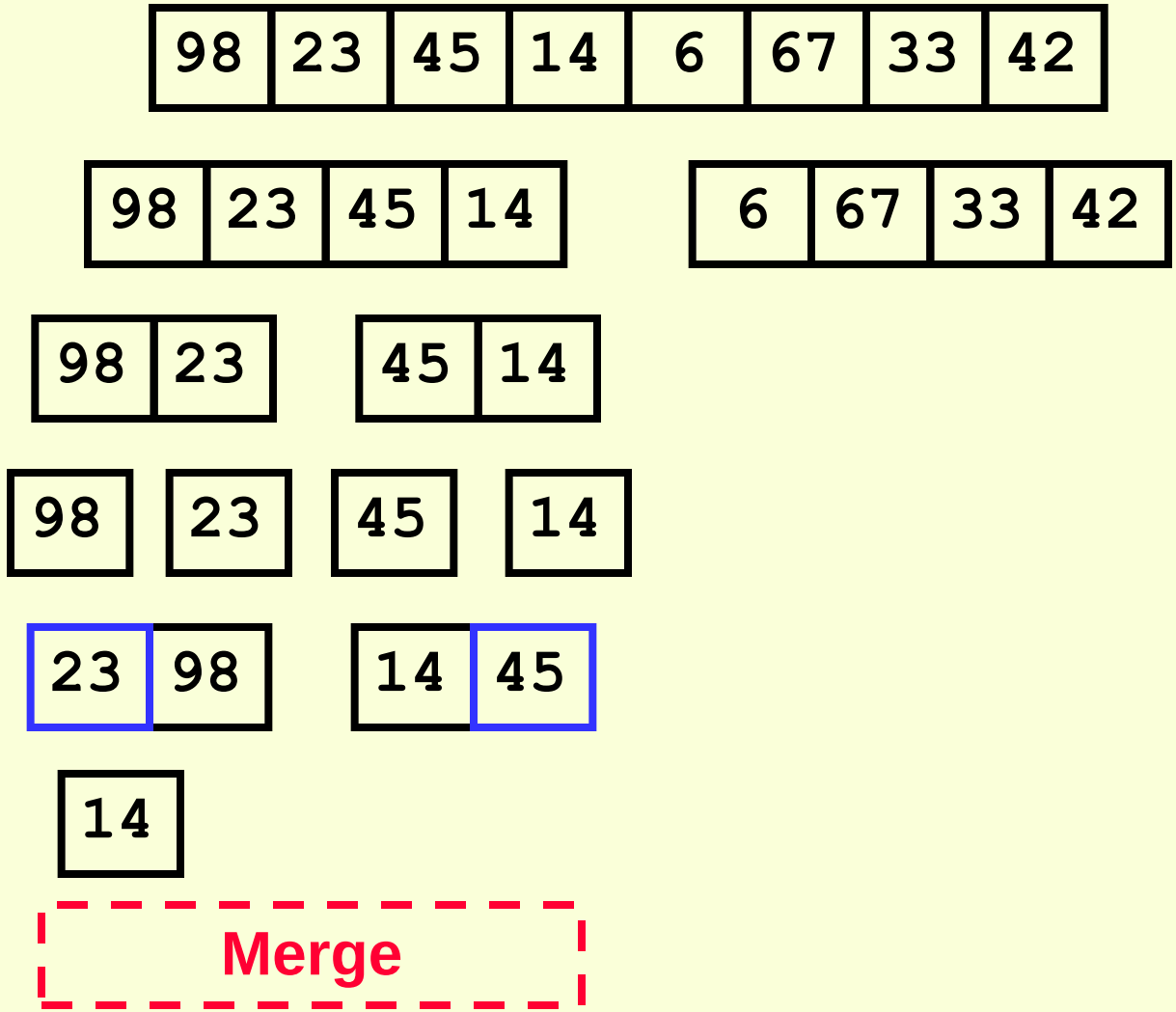

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98

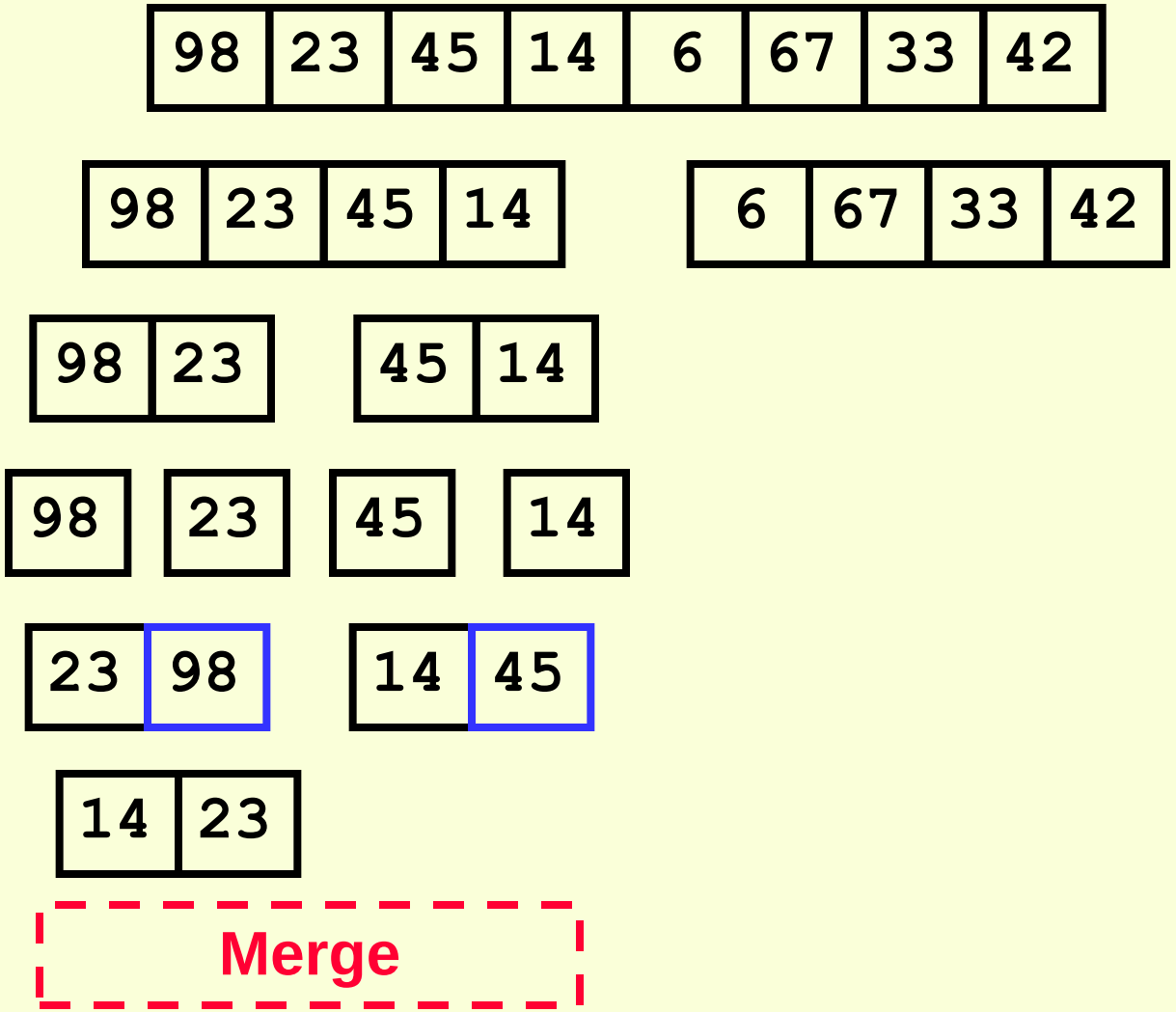

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
Merge

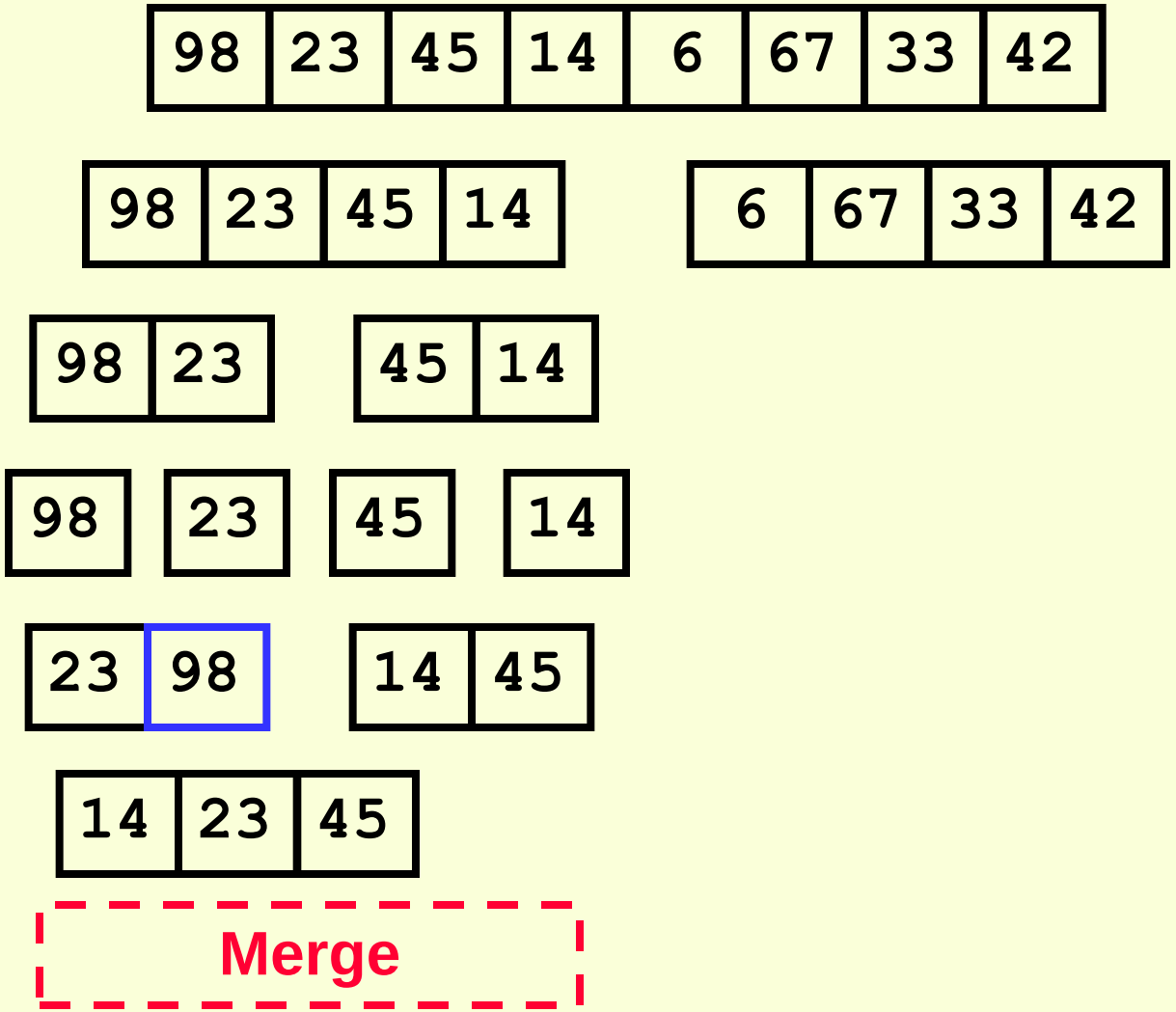

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14
Merge

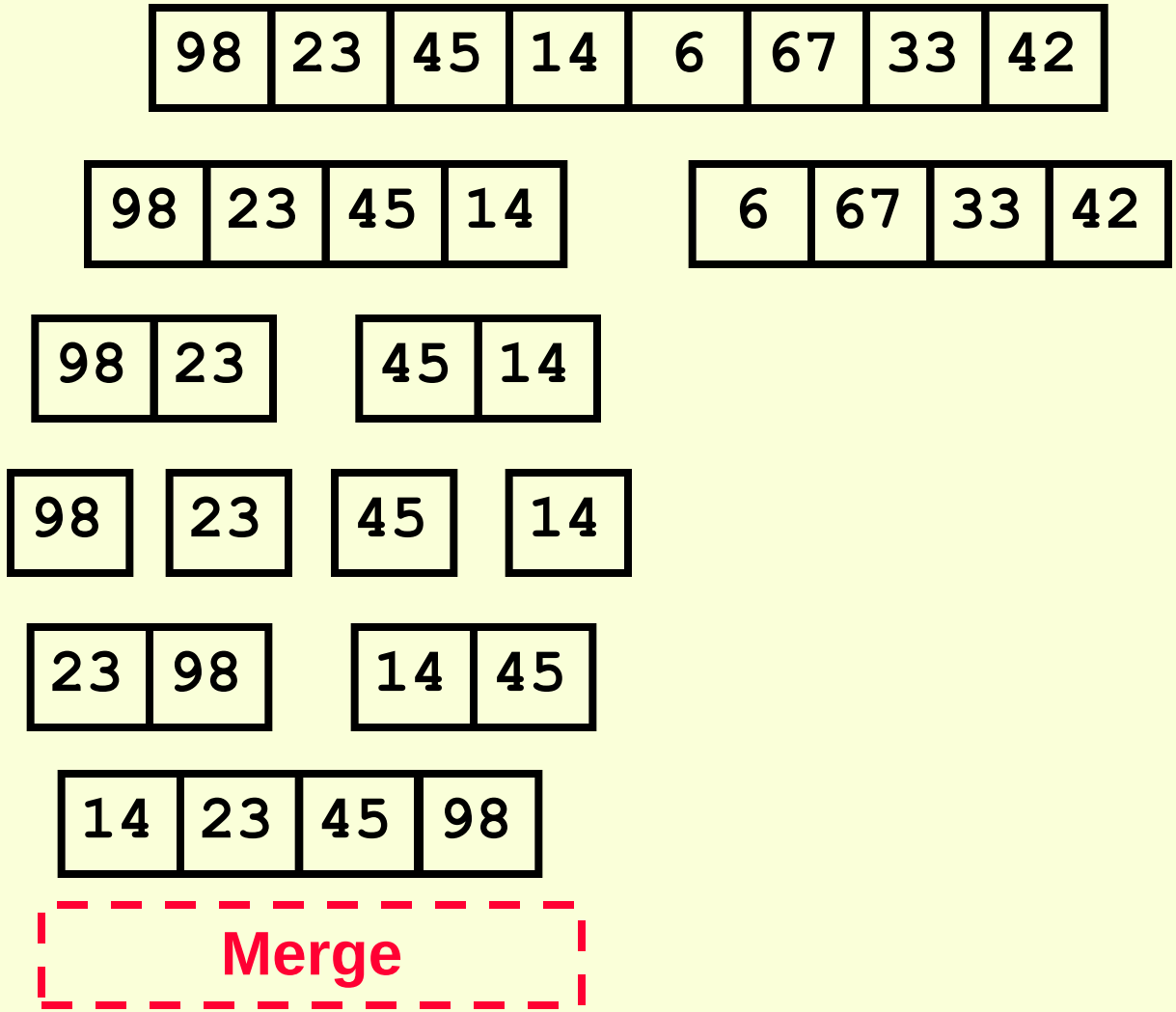

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
Merge

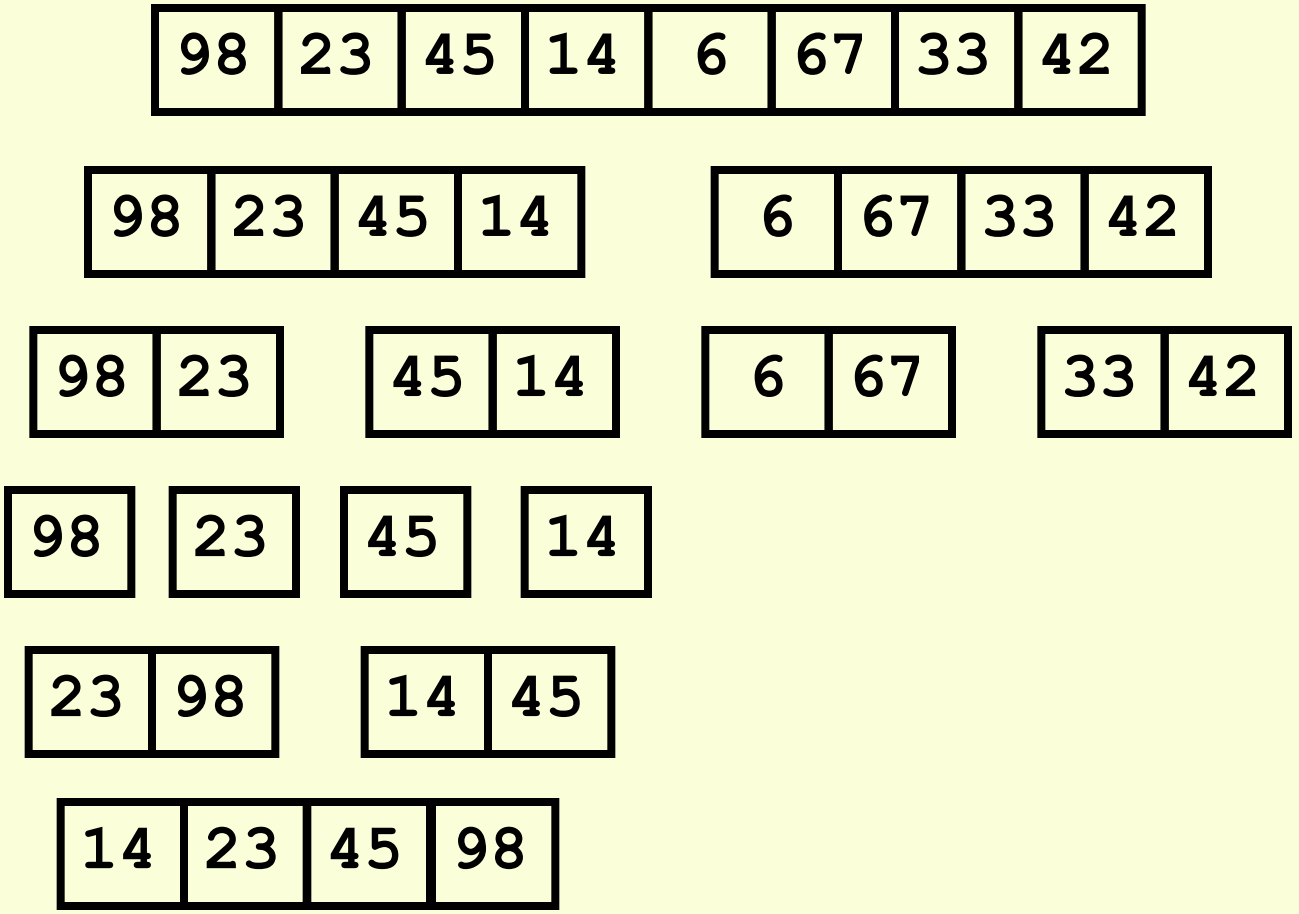

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
Merge

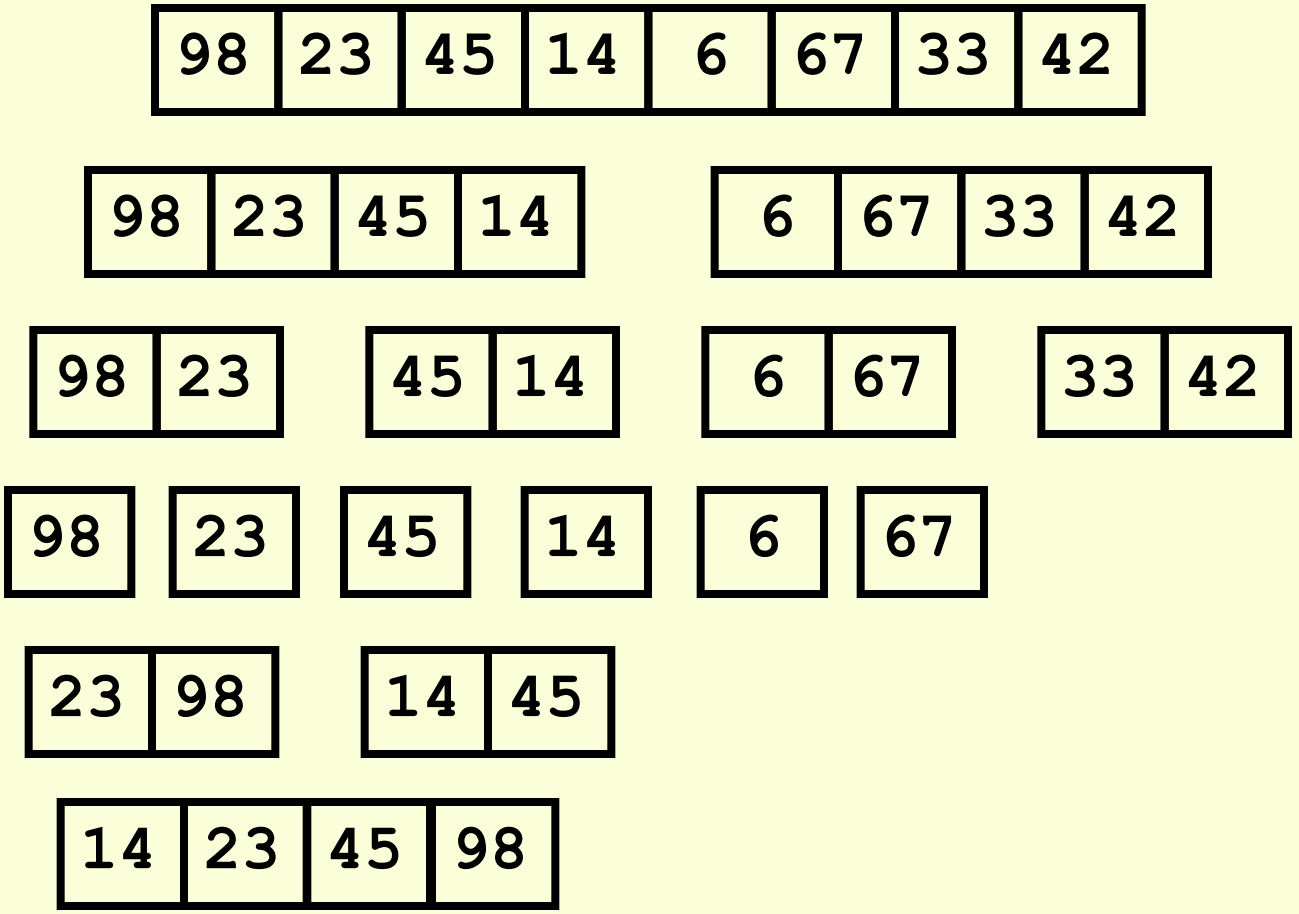

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14
Merge

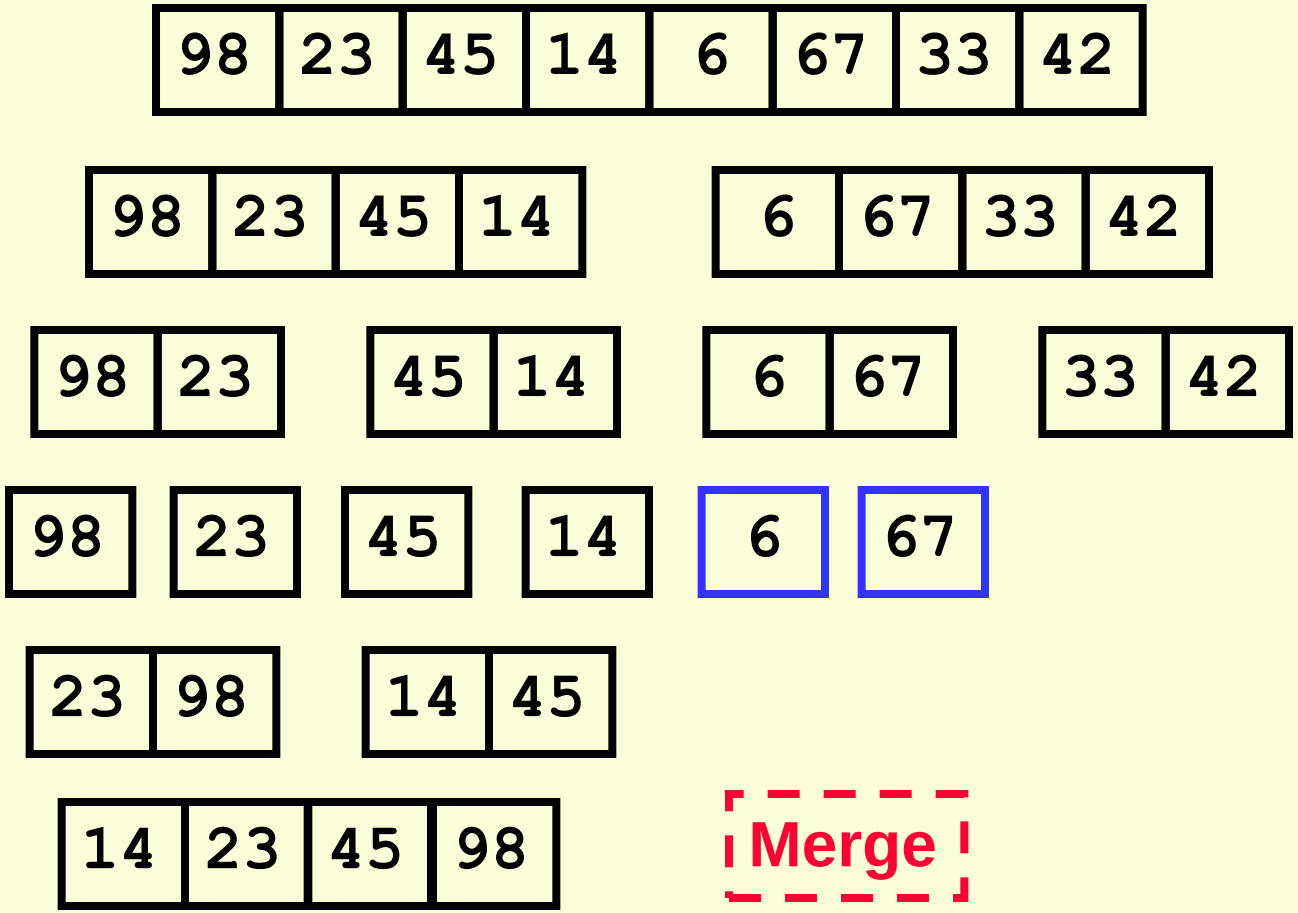

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14 23
Merge

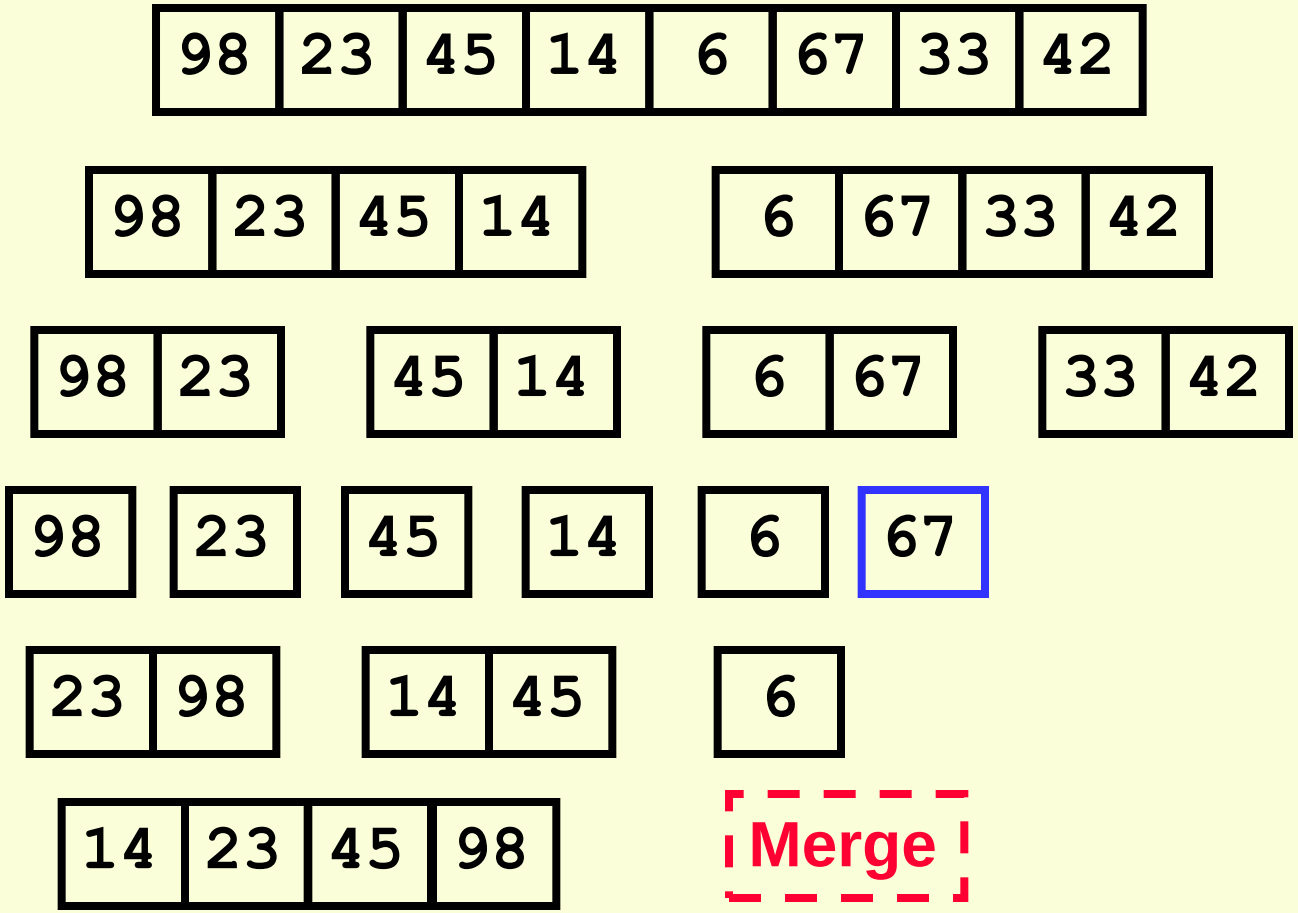

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14 23 45
Merge

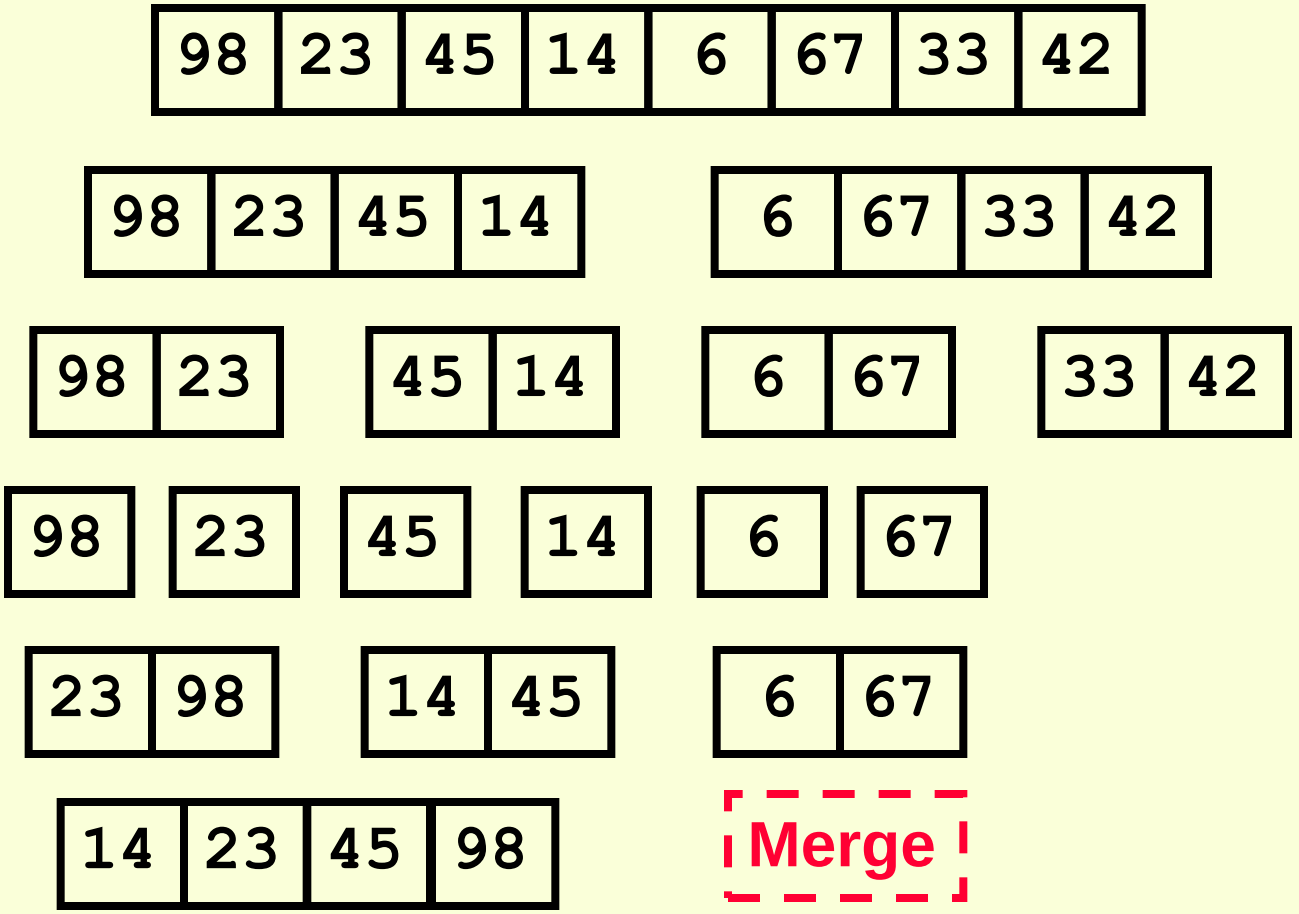

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14 23 45 98
Merge

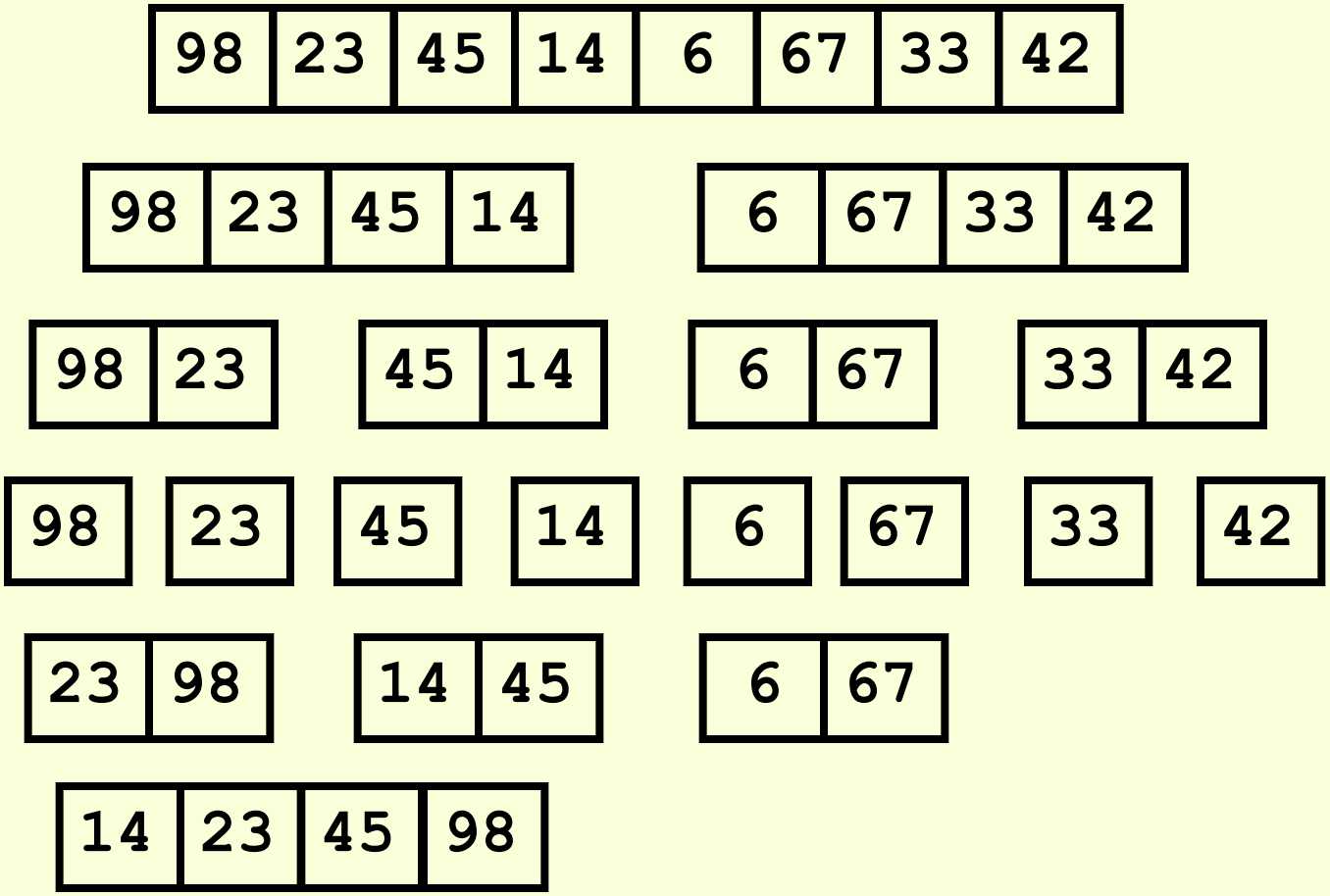

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
23 98
14 45
14 23 45 98

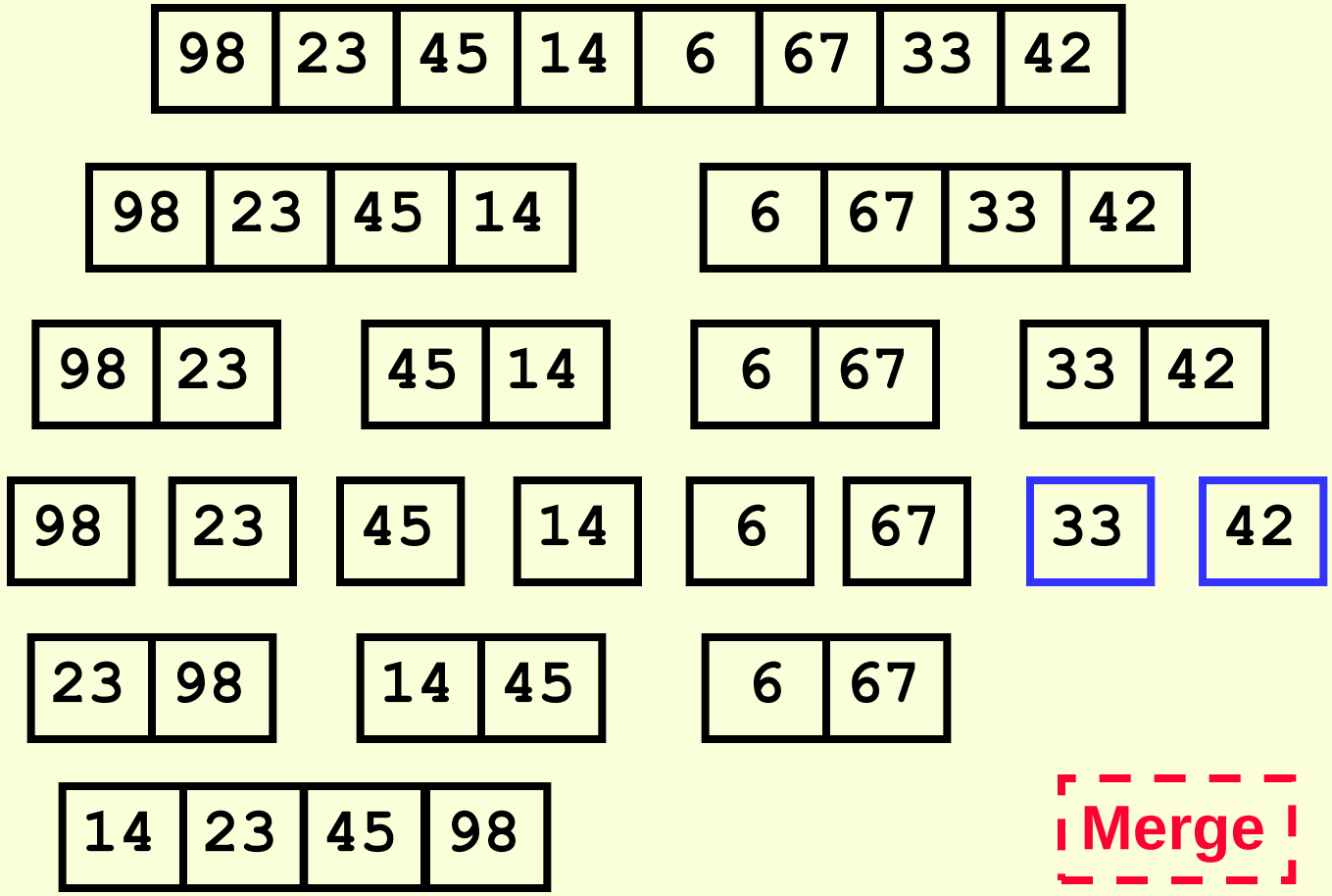

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
14 23 45 98

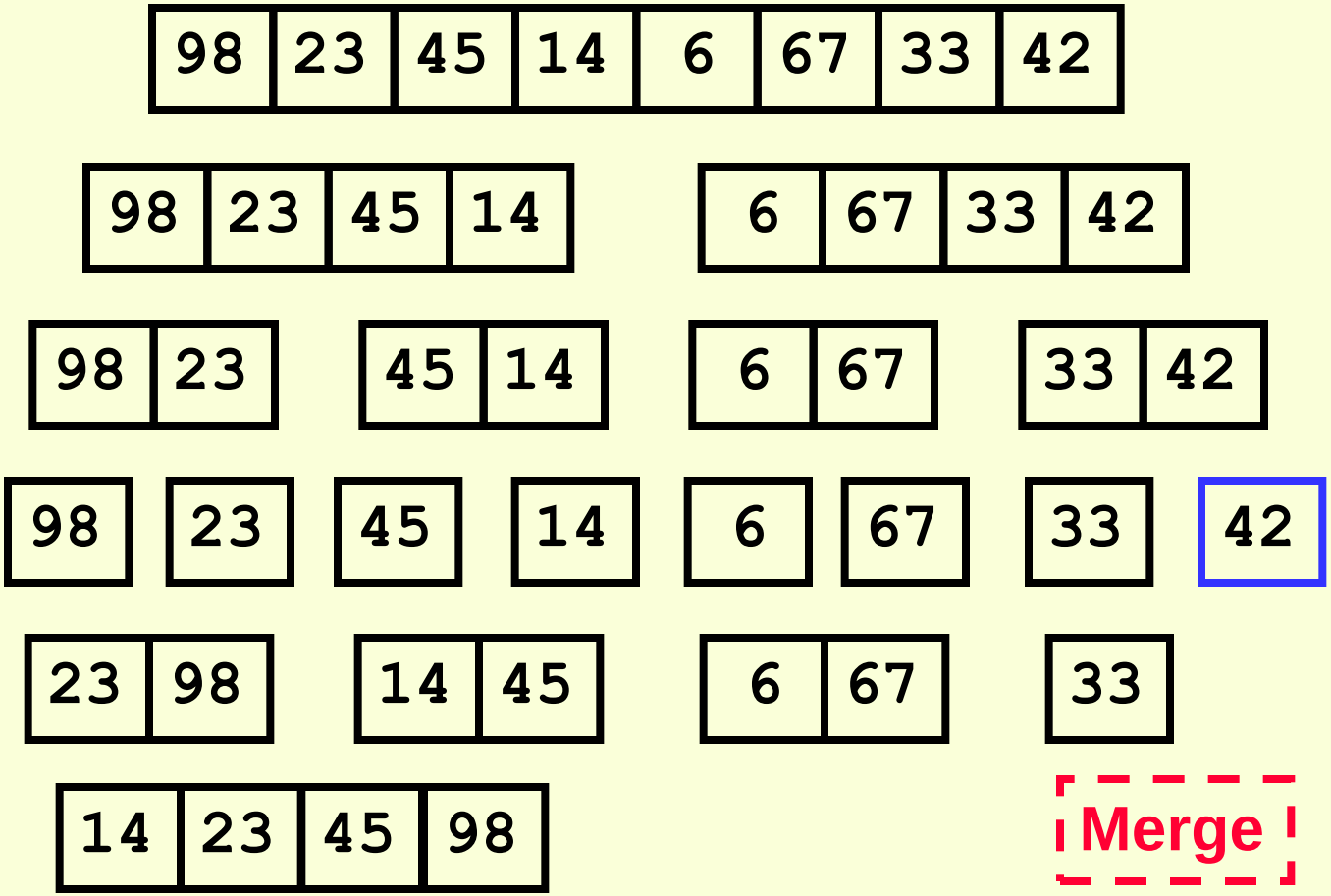

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
14 23 45 98
Merge

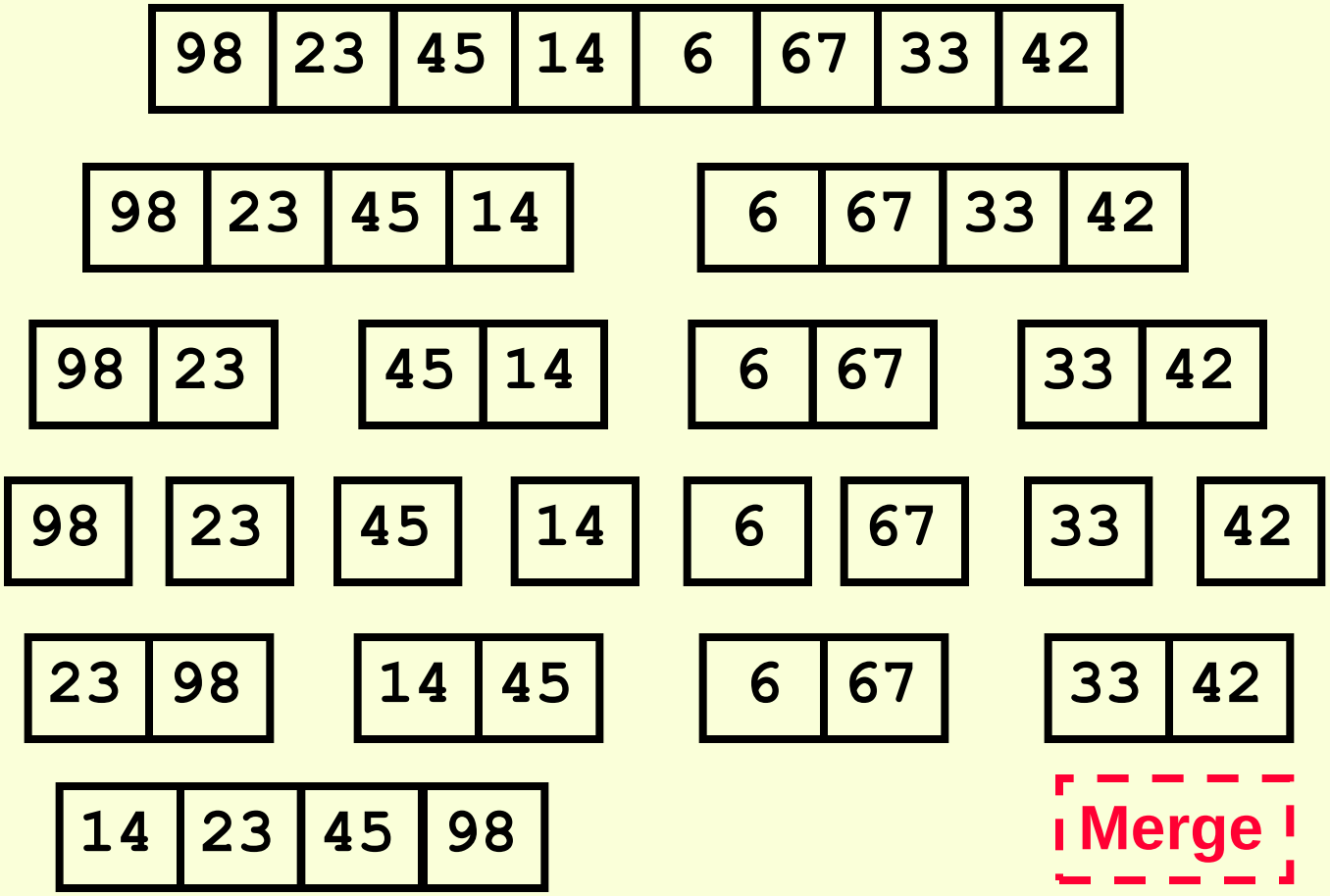

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
6
14 23 45 98
Merge

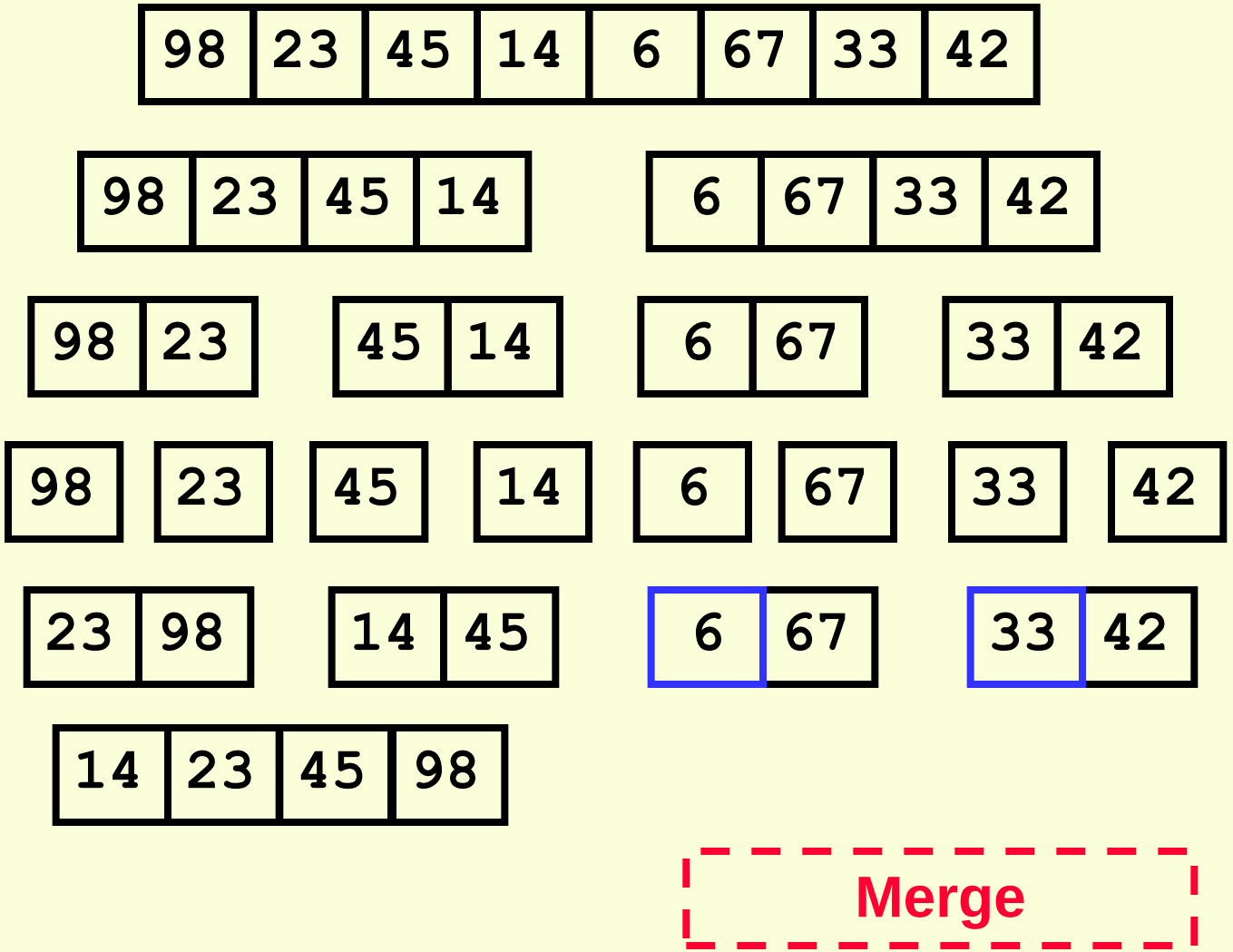

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
6 67
14 23 45 98
Merge

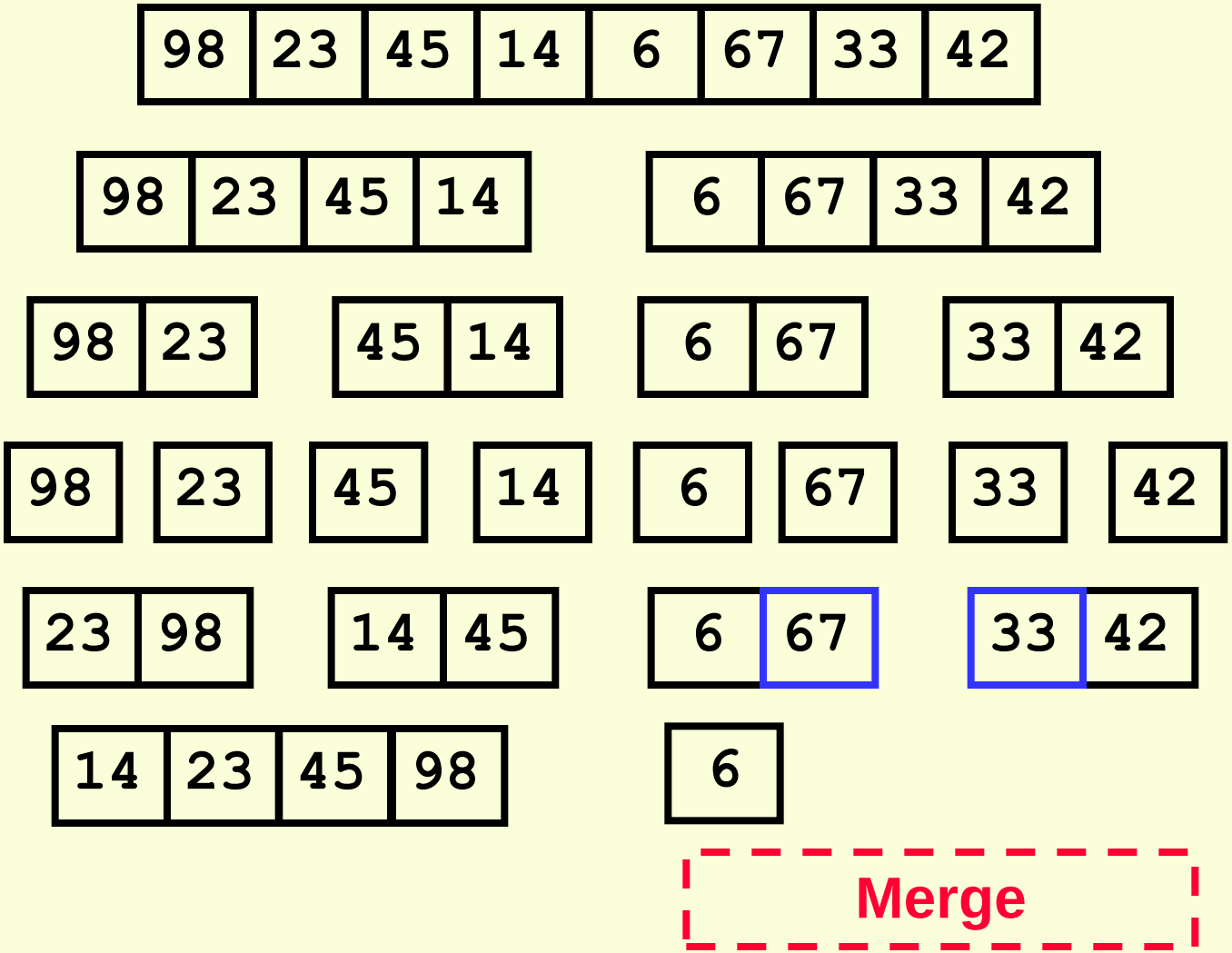

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
14 23 45 98

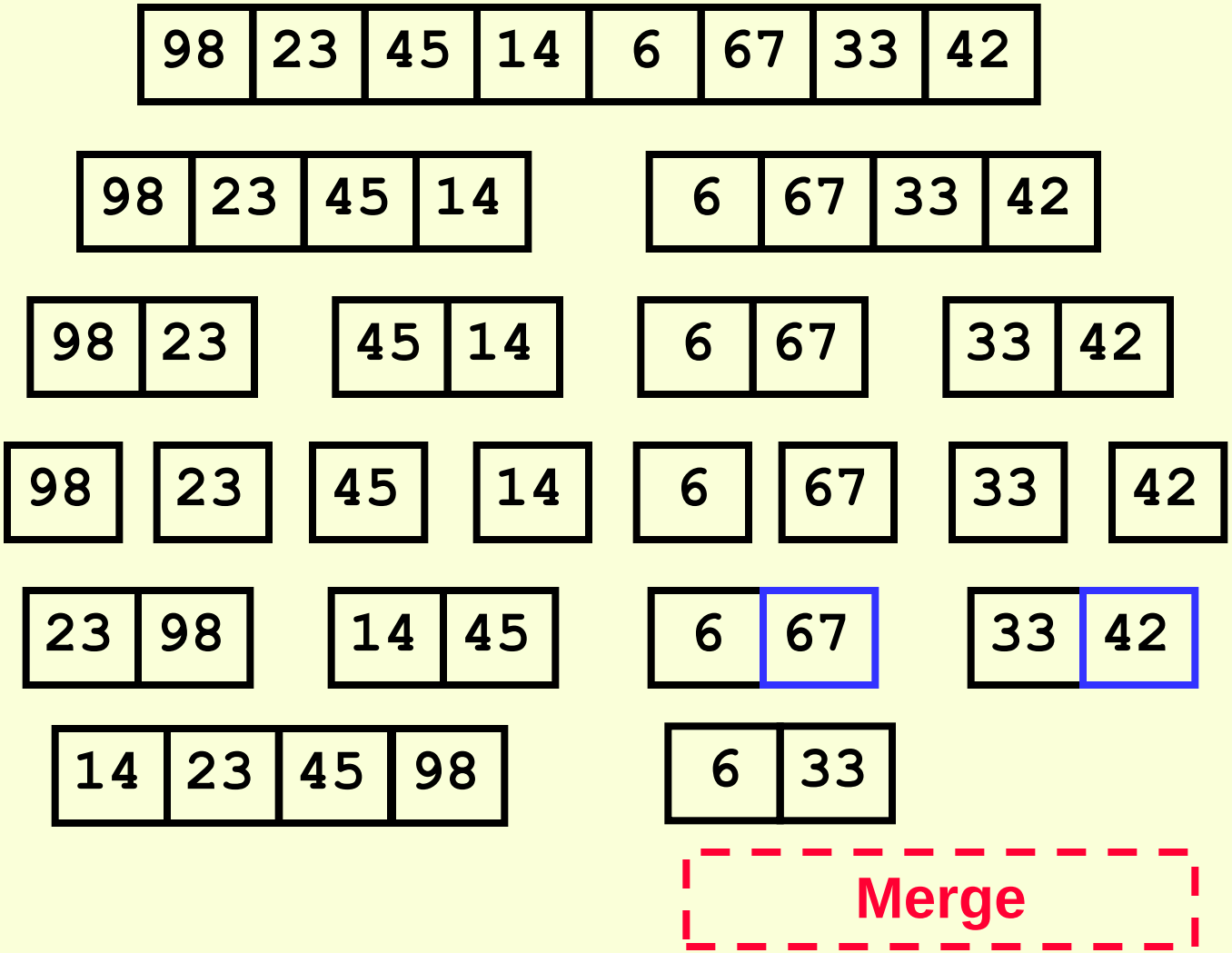

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
14 23 45 98
Merge

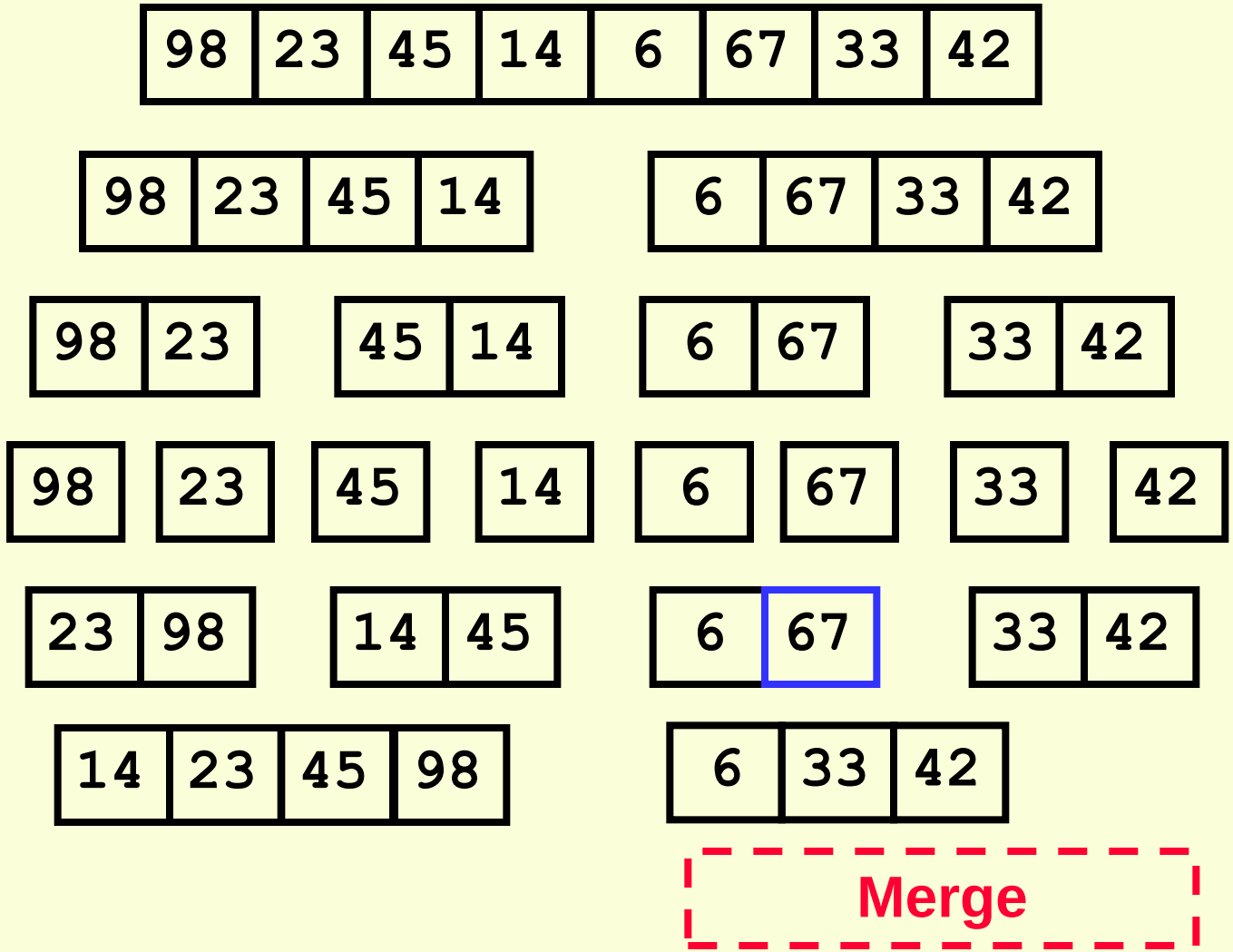

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33
14 23 45 98
Merge

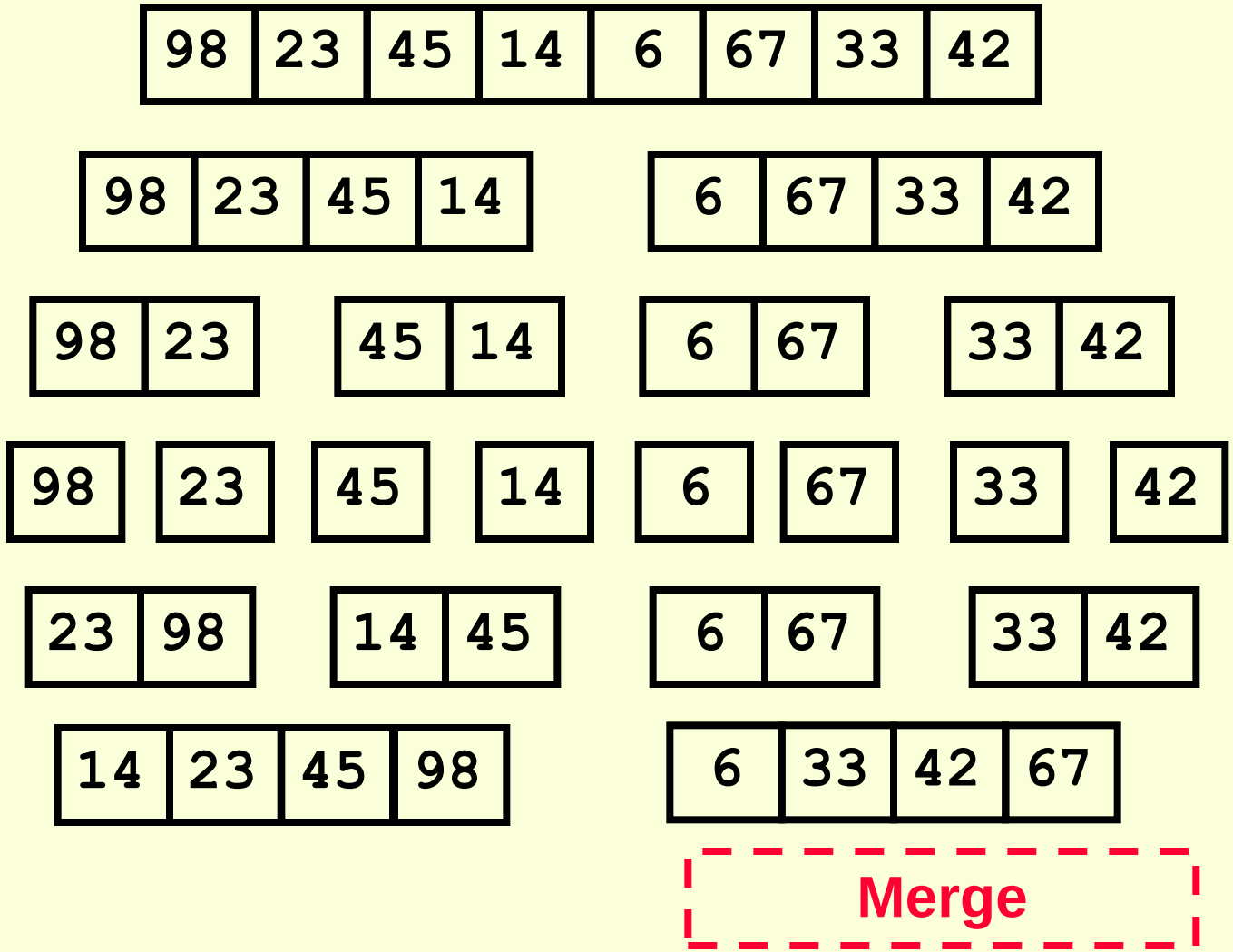

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
Merge

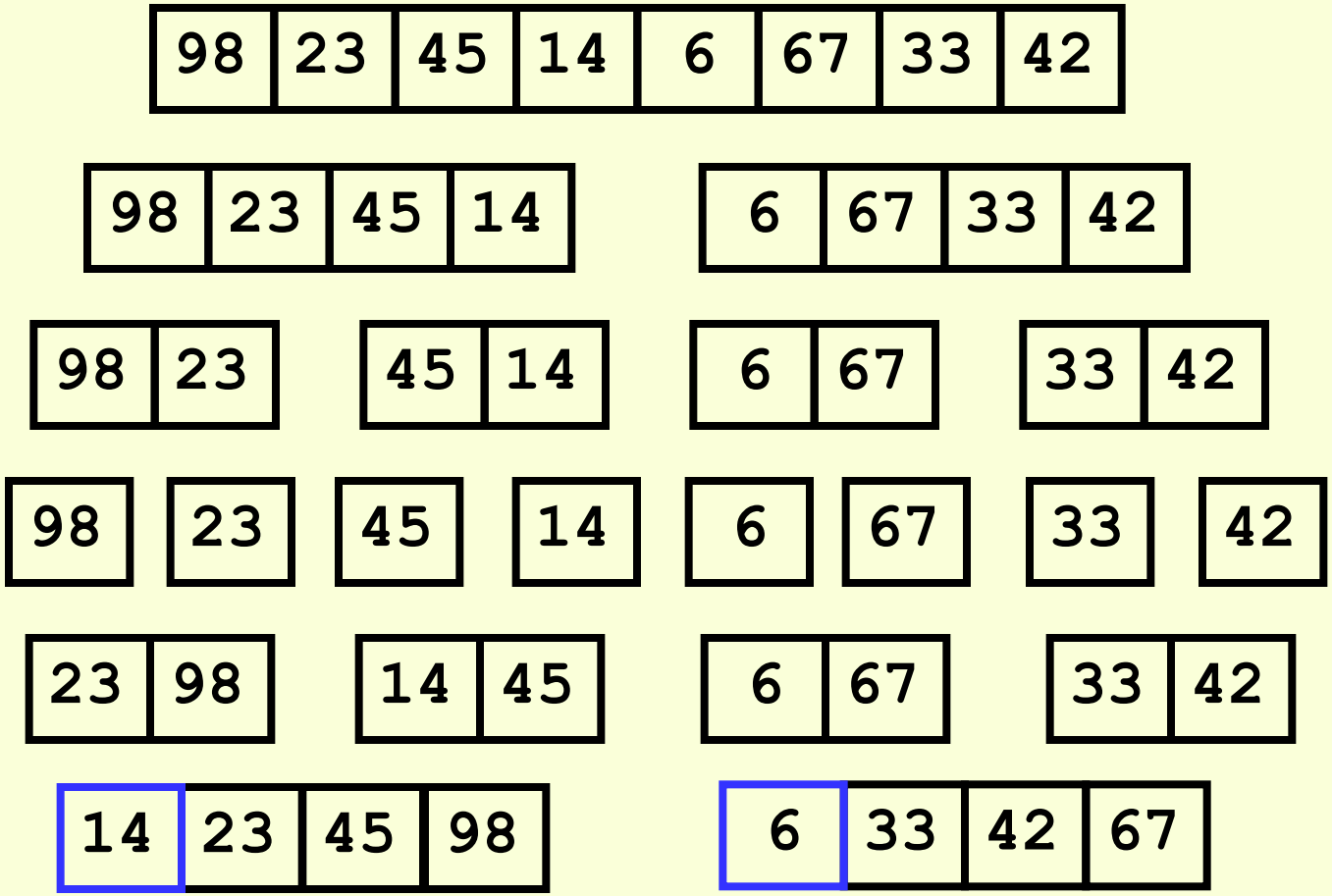

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
Merge

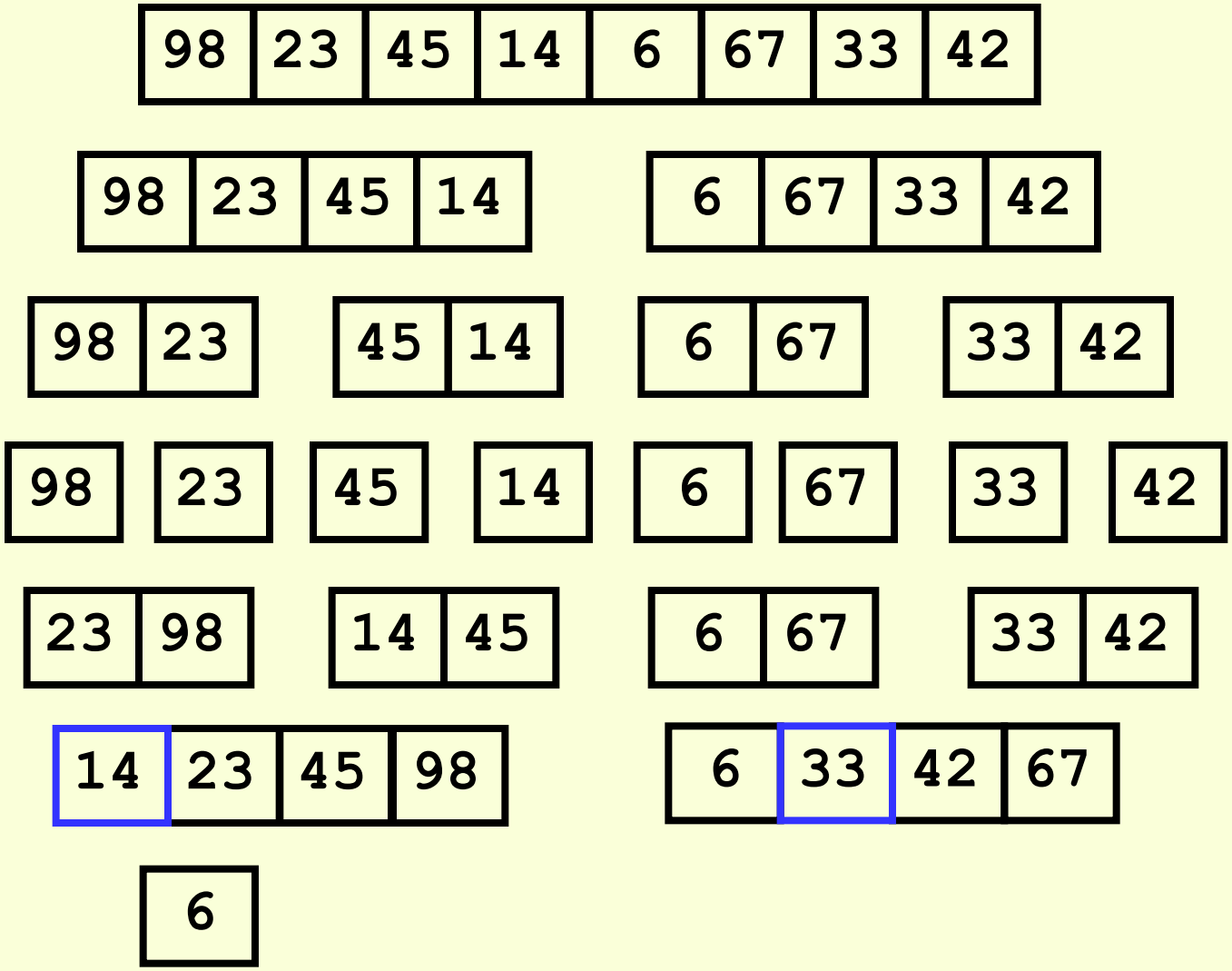

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6
Merge

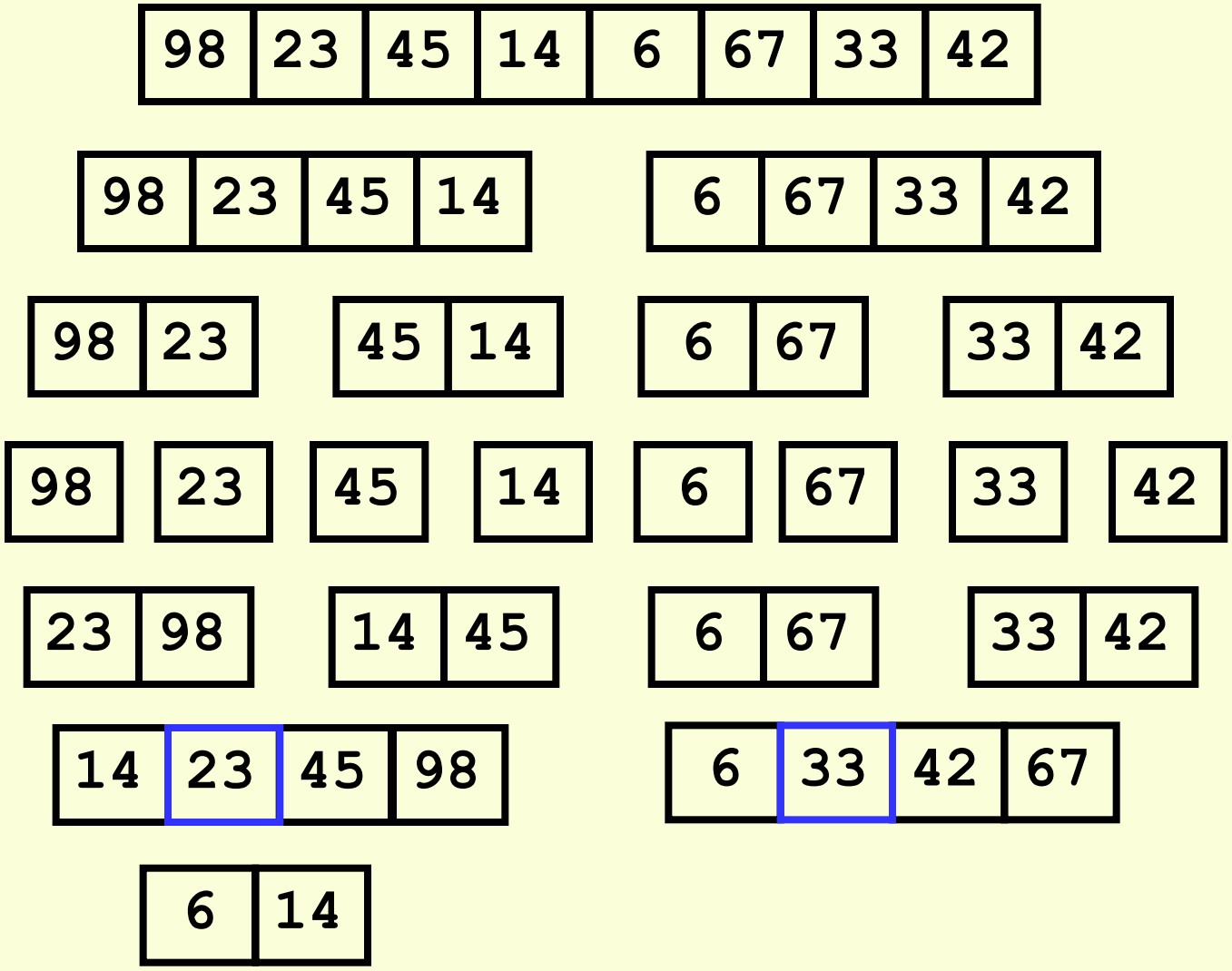

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33
Merge

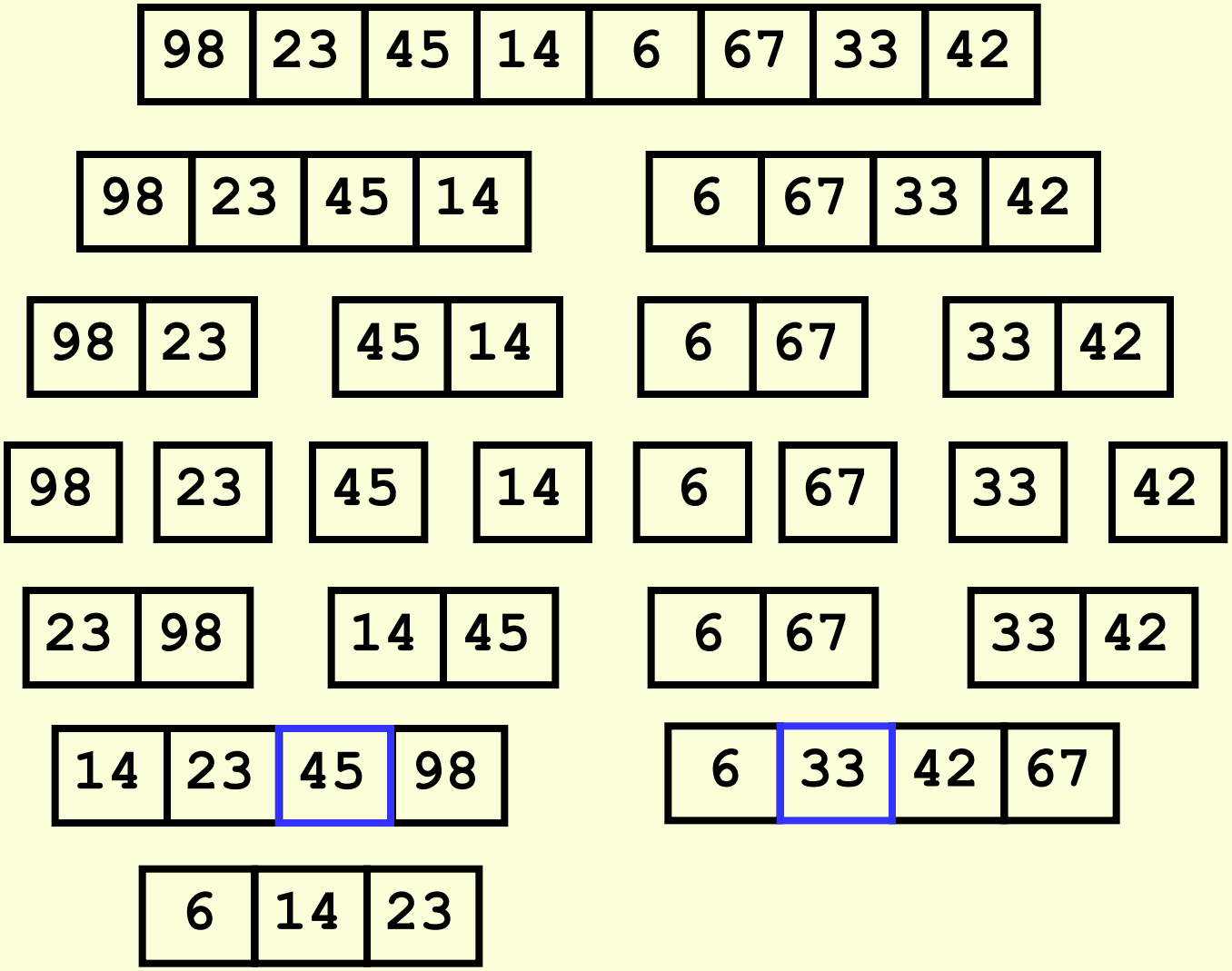

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42
Merge

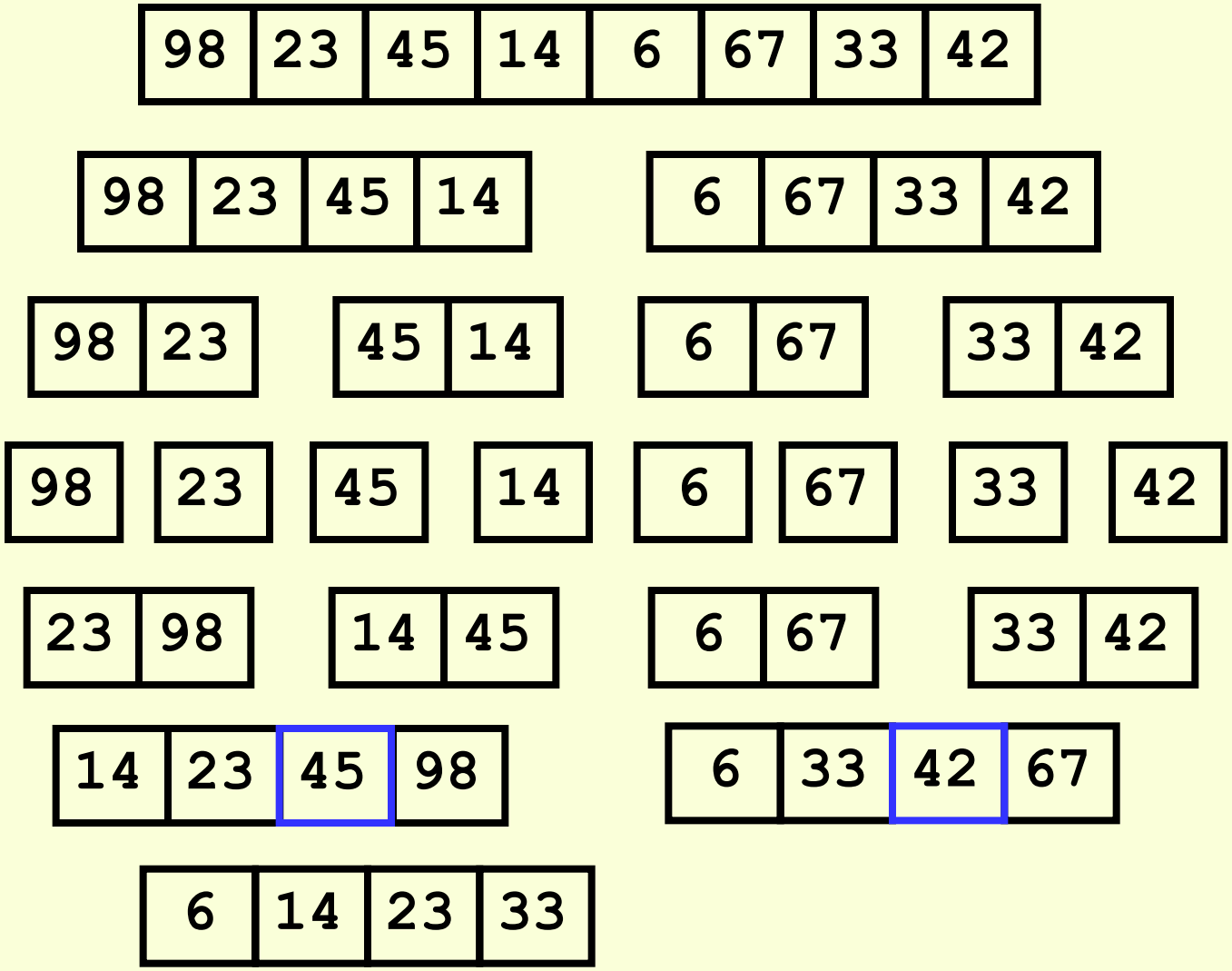

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
Merge

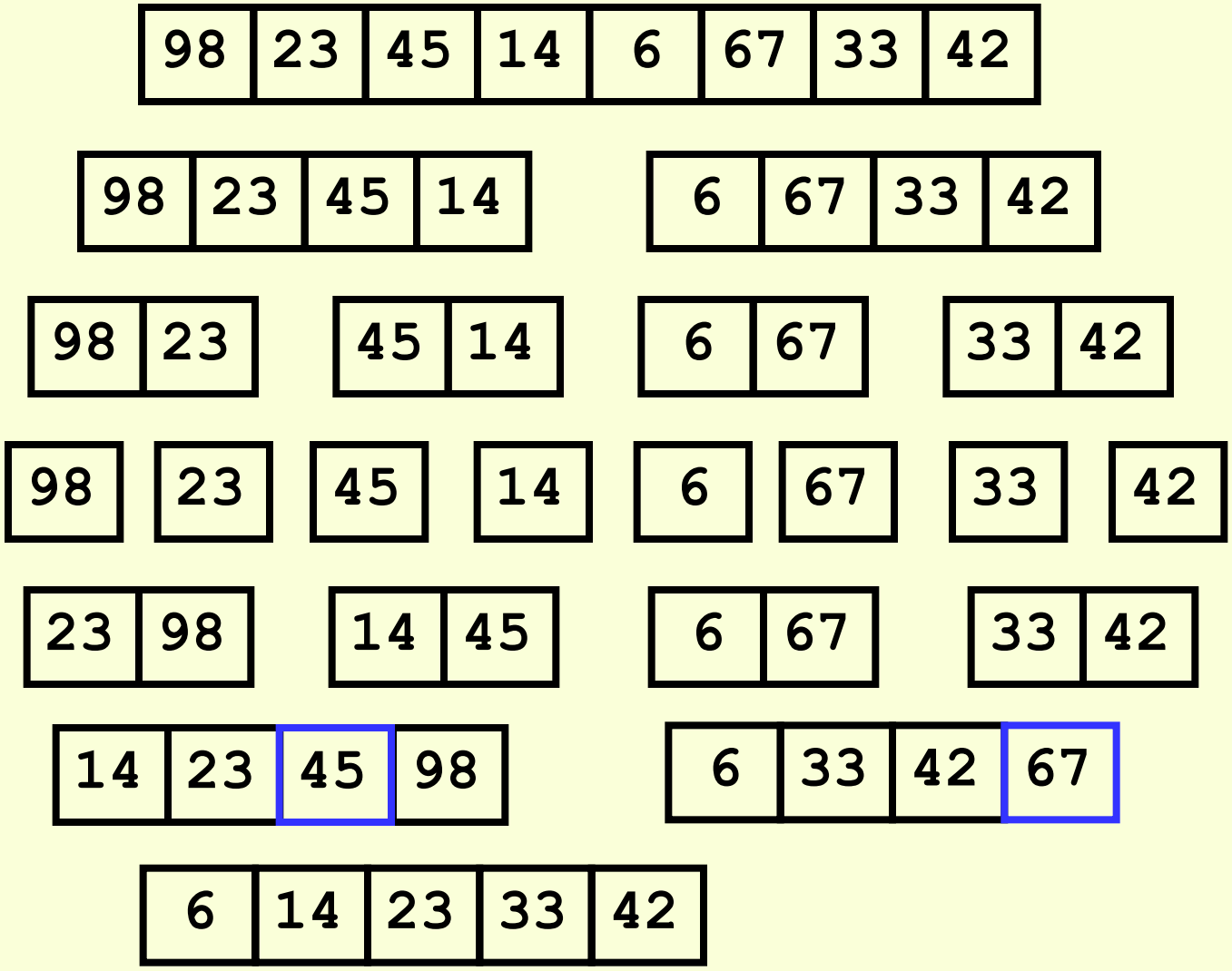

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
Merge

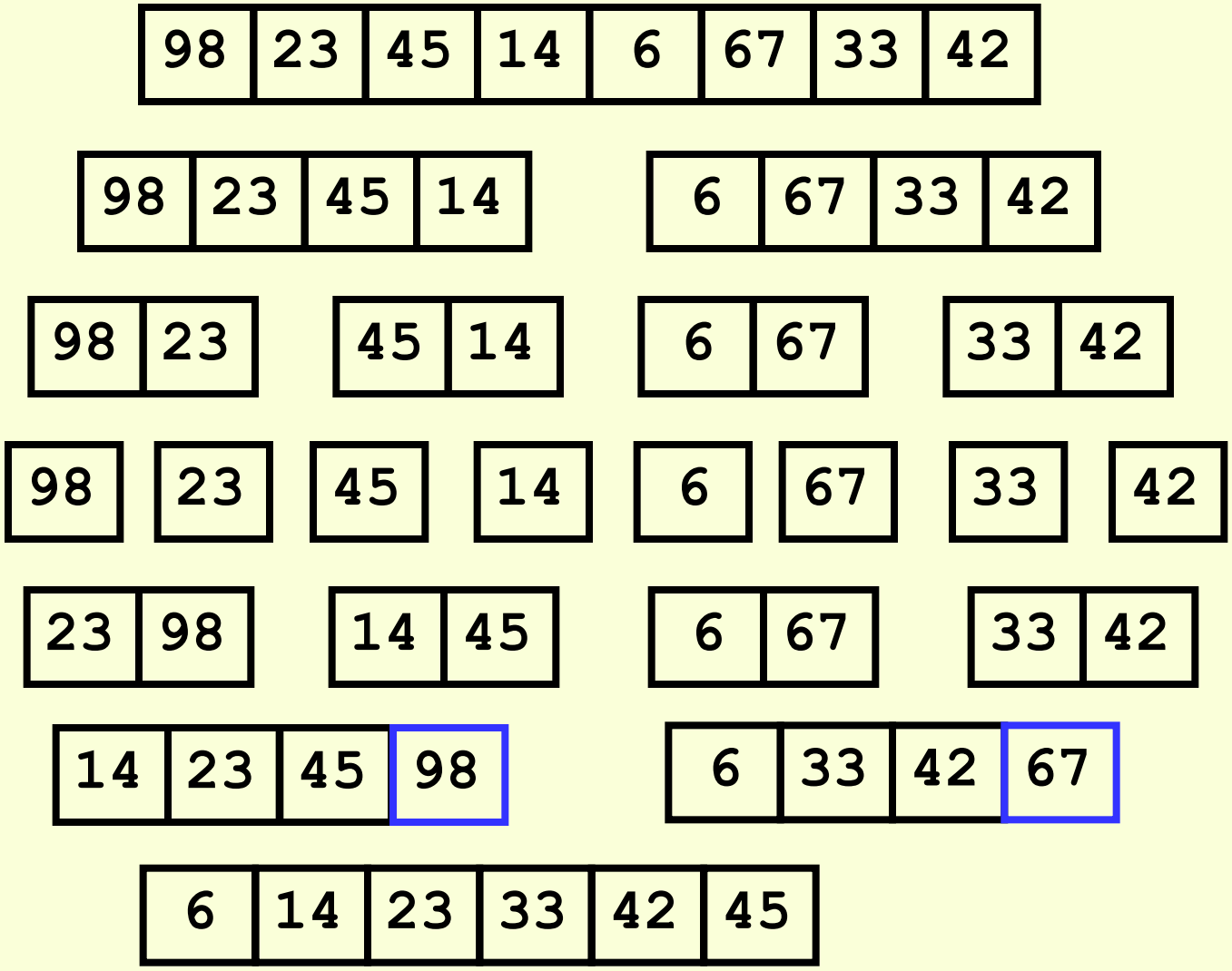

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6
Merge

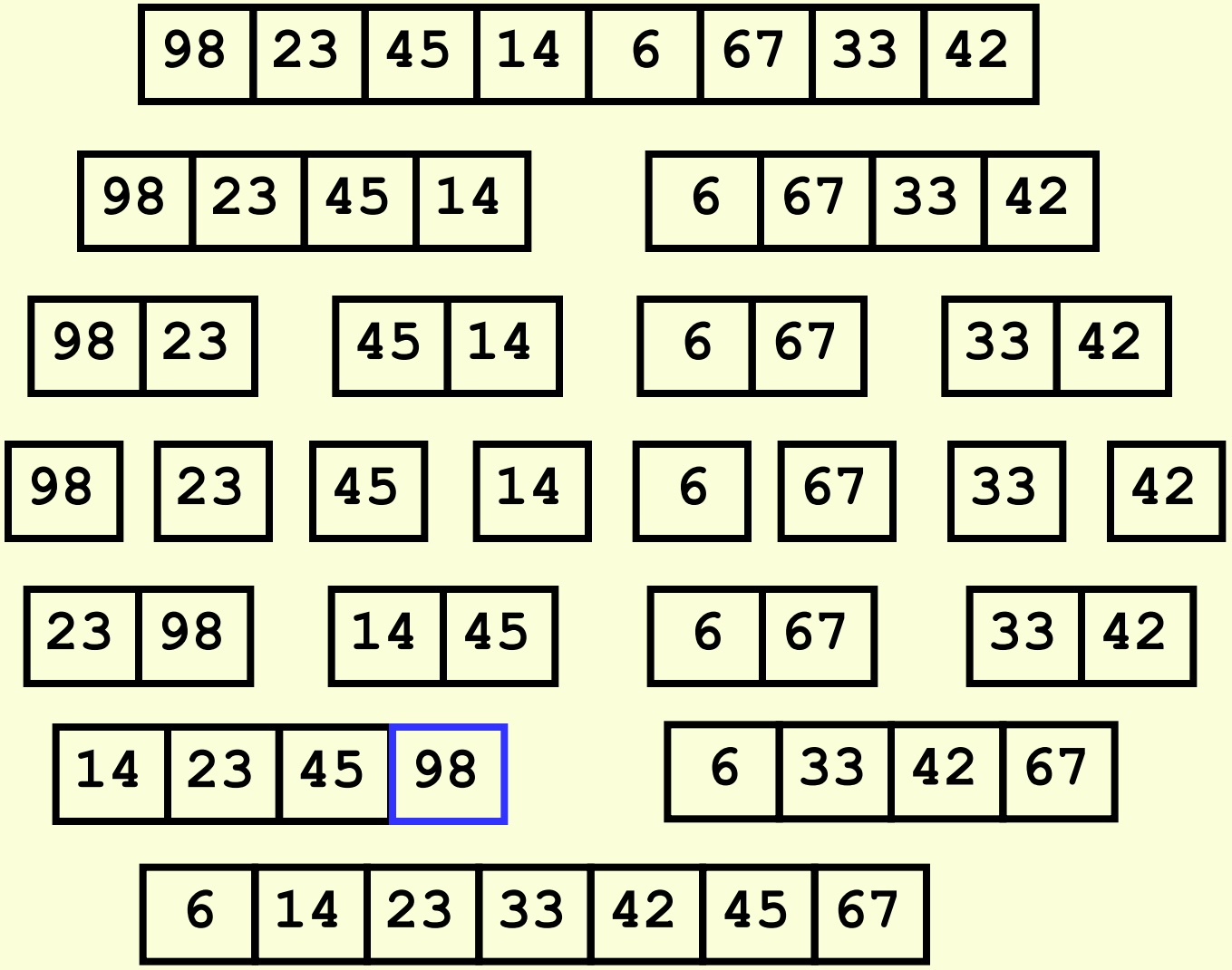

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14
Merge

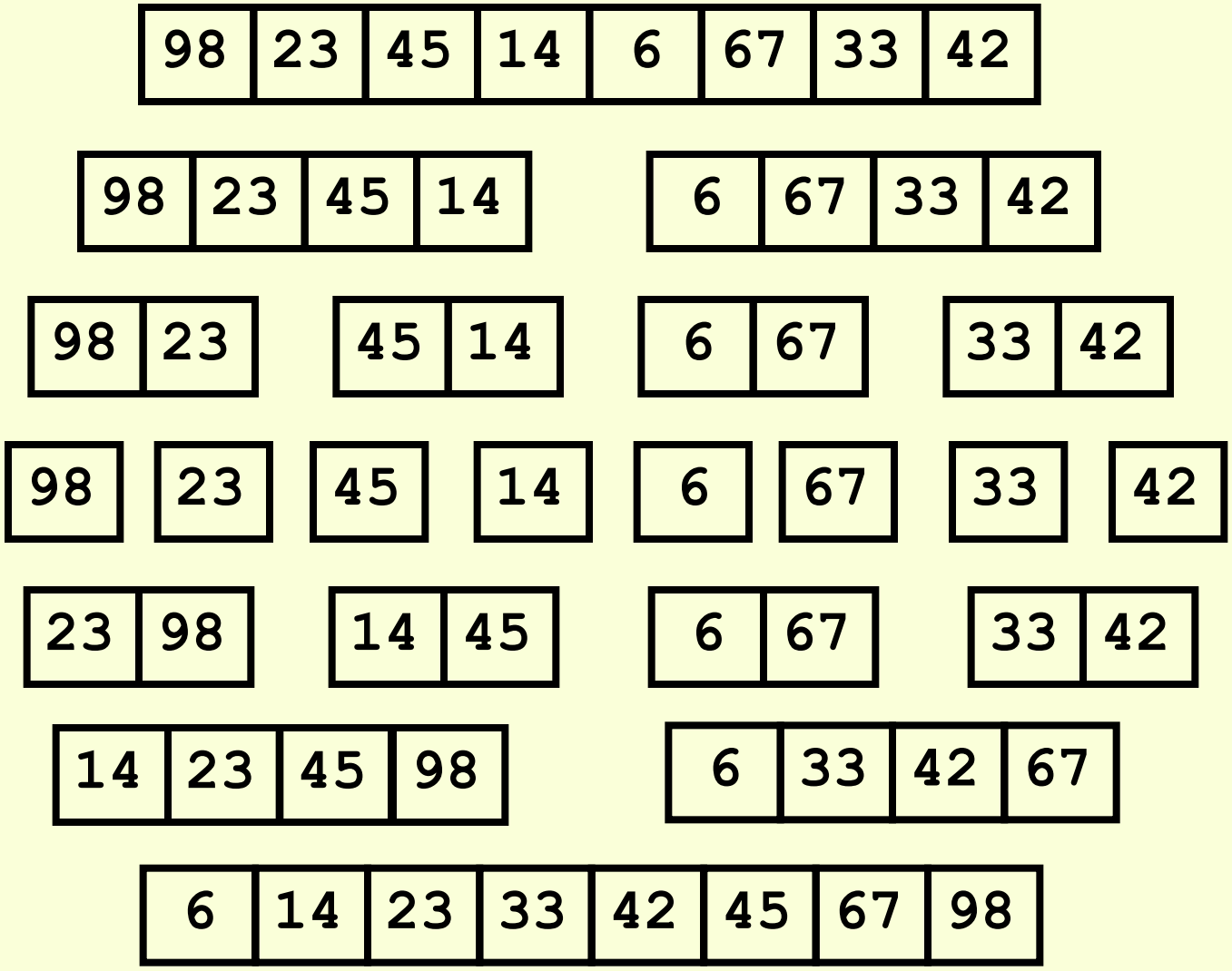

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23
Merge

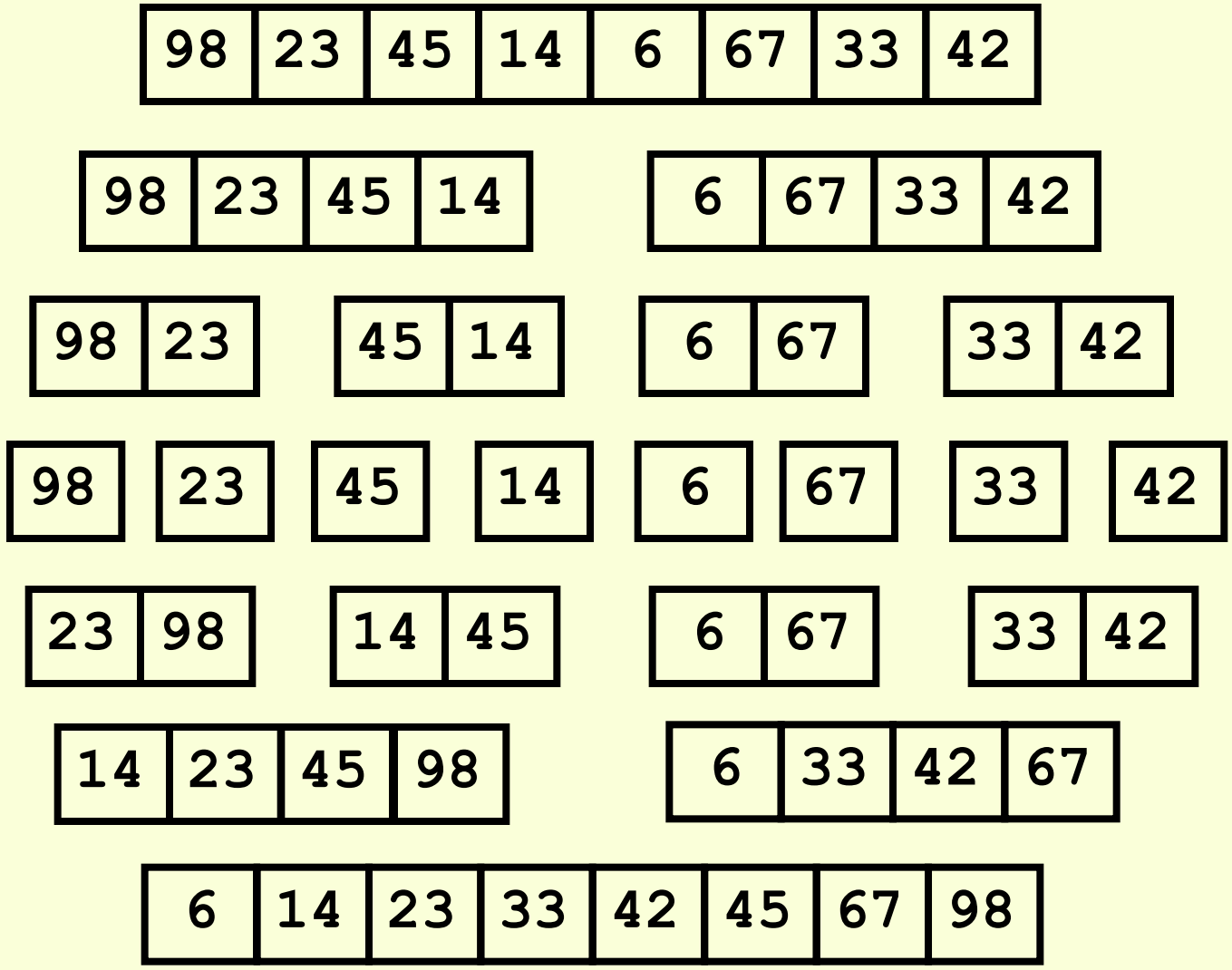

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33
Merge

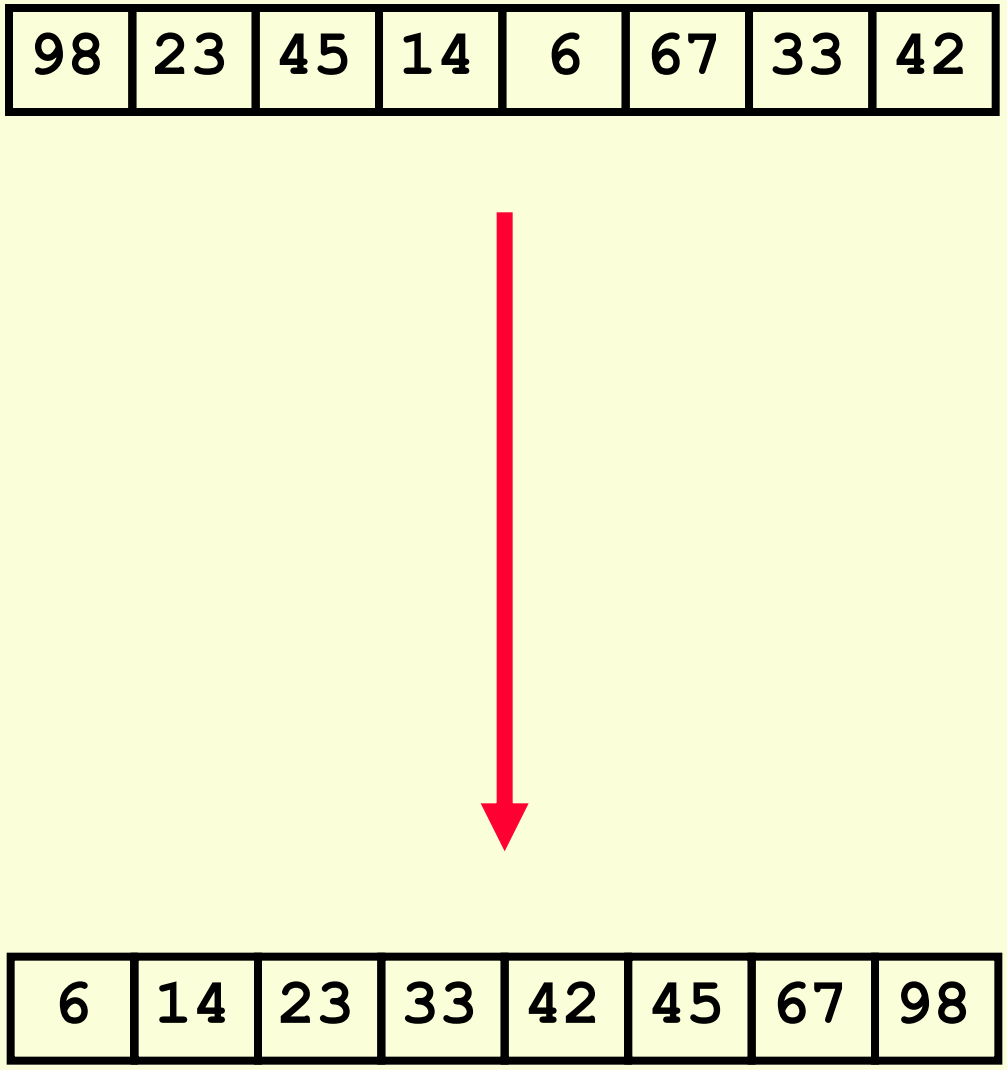

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42
Merge


98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45
Merge


98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45 67
Merge


98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45 67 98
Merge


98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45 67 98


98 23 45 14 6 67 33 42
6 14 23 33 42 45 67 98

Summary

- Divide the unsorted collection into two
- Until the sub-arrays only contain one element
- Then merge the sub-problem solutions together

Questions?

Review?